

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

Trịnh Hoàng An – Lâm Thiều Huy

XÂY DỰNG CÔNG CỤ KIỂM TRA ĐỘ
TƯƠNG ĐỒNG VĂN BẢN XUYỀN NGỮ

KHÓA LUẬN TỐT NGHIỆP CỬ NHÂN CNTT
CHƯƠNG TRÌNH CHẤT LƯỢNG CAO

Tp. Hồ Chí Minh, tháng 03/2026

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

Trịnh Hoàng An – 21127577

Lâm Thiều Huy – 21127056

**XÂY DỰNG CÔNG CỤ KIỂM TRA ĐỘ
TƯƠNG ĐỒNG VĂN BẢN XUYỀN NGỮ**

KHÓA LUẬN TỐT NGHIỆP CỬ NHÂN CNTT
CHƯƠNG TRÌNH CHẤT LƯỢNG CAO

GIÁO VIÊN HƯỚNG DẪN

TS. Nguyễn Hồng Bửu Long

Tp. Hồ Chí Minh, tháng 03/2026

Lời cảm ơn

Trước hết, chúng em xin bày tỏ lòng biết ơn chân thành và sâu sắc đến Thầy Nguyễn Hồng Bửu Long – người đã tận tình hướng dẫn, đồng hành và hỗ trợ chúng em trong suốt quá trình thực hiện khóa luận này. Những góp ý chuyên môn và định hướng quý báu từ Thầy đã giúp chúng em hoàn thiện đề tài một cách hiệu quả và sâu sắc hơn.

Chúng em cũng xin gửi lời cảm ơn đến quý Thầy Cô trong khoa Công nghệ Thông tin, Trường Đại học Khoa học Tự nhiên - Đại học Quốc gia TP.HCM đã truyền đạt cho tôi những kiến thức nền tảng vững chắc trong suốt quá trình học tập tại trường. Những bài giảng, tài liệu và kinh nghiệm quý báu từ các Thầy Cô là hành trang quan trọng giúp chúng em thực hiện tốt khóa luận này.

Bên cạnh đó, chúng em xin cảm ơn gia đình và bạn bè đã luôn ủng hộ, động viên và tạo điều kiện thuận lợi về tinh thần lẫn vật chất để chúng em yên tâm học tập và nghiên cứu.

Cuối cùng, chúng em xin chân thành cảm ơn tất cả những cá nhân và tổ chức đã cung cấp tài liệu, nguồn dữ liệu, công cụ, hoặc hỗ trợ kỹ thuật cần thiết trong quá trình thực hiện đề tài.

Tuy đã cố gắng hết sức, nhưng chắc chắn khóa luận vẫn không tránh khỏi những thiếu sót. Kính mong nhận được sự góp ý và phản hồi từ quý Thầy Cô và độc giả để đề tài được hoàn thiện hơn.

TP. Hồ Chí Minh, tháng 3 năm 2026

Trịnh Hoàng An - Lâm Thiều Huy



ĐỀ CƯƠNG KHOÁ LUẬN TỐT NGHIỆP
XÂY DỰNG CÔNG CỤ KIỂM TRA ĐỘ
TƯƠNG ĐỒNG VĂN BẢN XUYỀN NGỮ

1. THÔNG TIN CHUNG

Người hướng dẫn:

– TS. Nguyễn Hồng Bửu Long (Khoa Công nghệ Thông tin)

Nhóm sinh viên thực hiện:

1. Lâm Thiều Huy (MSSV: 21127056)
2. Trịnh Hoàng An (MSSV: 21127577)

Loại đề tài: Ứng dụng

Thời gian thực hiện: Từ 11/2025 đến 4/2026

2. NỘI DUNG THỰC HIỆN

2.1 Giới thiệu về đề tài

Trong bối cảnh số lượng tài liệu được số hóa ngày càng tăng nhanh tại các cơ quan báo chí, doanh nghiệp và tổ chức, việc tìm kiếm và khai thác thông tin hiệu quả đang trở thành một thách thức lớn. Các tài liệu này không chỉ đa dạng về định dạng, nội dung mà còn phong phú về mặt ngôn ngữ, đặc biệt là trong các tổ chức có sử dụng song song tiếng Việt và tiếng Anh trong giao tiếp và lưu trữ thông tin. Tuy nhiên, hầu hết các hệ thống hiện nay chỉ hỗ trợ so sánh văn bản trong cùng một ngôn ngữ, chưa đáp ứng được nhu cầu kiểm tra độ tương đồng văn bản xuyên ngữ – một yêu cầu ngày càng phổ biến trong môi trường toàn cầu hóa.

Để giải quyết vấn đề trên, đề tài hướng tới việc xây dựng một công cụ kiểm tra độ tương đồng văn bản xuyên ngữ (cross-lingual text similarity checker), cho phép so sánh một văn bản đầu vào (có thể là tiếng Việt hoặc tiếng Anh) với kho dữ liệu chứa các văn bản thuộc ngôn ngữ khác. Qua đó, hệ thống có thể hỗ trợ người dùng phát hiện nội dung trùng lặp hoặc tương tự về mặt ý nghĩa, ngay cả khi hai văn bản được viết bằng ngôn ngữ khác nhau – điều mà các phương pháp truyền thống thường không xử lý được.

Khóa luận này ứng dụng các kỹ thuật hiện đại trong lĩnh vực xử lý ngôn ngữ tự nhiên (NLP), đặc biệt là các phương pháp biểu diễn ngữ nghĩa văn bản như Multilingual BERT (mBERT), XLM-RoBERTa, và Sentence-BERT đa ngữ – những mô hình đã được huấn luyện trên nhiều ngôn ngữ và chứng minh hiệu quả trong các tác vụ so sánh ngữ nghĩa xuyên ngữ [1], [2]. Các biểu diễn này sau đó được sử dụng để tính toán độ tương đồng bằng các chỉ số như Cosine Similarity. Bên cạnh đó, đề tài còn tích hợp Retrieval-Augmented Generation (RAG) – một hướng tiếp cận tiên tiến kết hợp khả năng truy xuất văn bản liên quan từ kho dữ liệu và sinh nội dung tự động. Nhờ đó, hệ thống không chỉ trả về danh sách các văn bản tương tự, mà còn có thể tóm tắt nội dung,

trả lời câu hỏi, hoặc mô tả mối liên hệ ngữ nghĩa giữa văn bản đầu vào và các tài liệu được tìm thấy.

Về mặt ý nghĩa thực tiễn, hệ thống không chỉ giúp tiết kiệm thời gian tra cứu và phân tích thông tin, mà còn đóng vai trò hỗ trợ quản lý tri thức trong các tổ chức sử dụng đa ngôn ngữ. Đây là tiền đề để phát triển các công cụ AI thông minh hơn, xử lý ngôn ngữ và tổ chức dữ liệu hiệu quả trong kỷ nguyên toàn cầu hóa và tự động hóa thông tin.

2.2 Mục tiêu đề tài

Đề tài hướng đến việc nghiên cứu và xây dựng một công cụ hỗ trợ tìm kiếm thông minh, có khả năng **so sánh mức độ tương đồng giữa một văn bản đầu vào và các tài liệu có sẵn trong kho dữ liệu lớn**, đặc biệt là trong **ngữ cảnh xuyên ngữ** (cross-lingual), nơi văn bản đầu vào và tài liệu có thể thuộc các ngôn ngữ khác nhau (ví dụ: tiếng Việt và tiếng Anh).

Hệ thống đề xuất nhằm:

- **Xác định các tài liệu có nội dung giống hoặc trùng lặp** với văn bản đầu vào.
- **Tìm kiếm các văn bản liên quan theo ngữ nghĩa**, vượt qua giới hạn của các phương pháp tìm kiếm dựa trên từ khóa truyền thống.
- **Sinh thông tin hỗ trợ người dùng hiểu rõ hơn kết quả truy xuất**, thông qua tóm tắt hoặc trả lời tự động bằng mô hình ngôn ngữ.

Để thực hiện mục tiêu này thì việc tích hợp mô hình **Retrieval-Augmented Generation (RAG)** – cho phép hệ thống không chỉ truy xuất văn bản tương đồng mà còn **kết hợp khả năng sinh văn bản tự động**. Nhờ đó, người dùng có thể nhận được các **tóm tắt nội dung, mô tả ngắn gọn**, hoặc câu trả lời trực tiếp cho các câu hỏi liên quan đến văn bản đã được truy xuất.

2.3 Phạm vi của đề tài

Đề tài tập trung vào việc xây dựng một công cụ hỗ trợ **so sánh độ tương đồng giữa một văn bản đầu vào với các tài liệu đã có trong một kho dữ liệu số**, đồng thời kết hợp khả năng **truy xuất và sinh thông tin tự động** dựa trên mô hình **Retrieval-Augmented Generation (RAG)**. Phạm vi nghiên cứu bao gồm các nội dung chính sau:

- **Đối tượng nghiên cứu:**

- Các tệp tin văn bản lưu trữ dưới các định dạng phổ biến như txt, doc, docx, pdf.
- Kho dữ liệu văn bản có thể là tập hợp các bài báo, tài liệu nội bộ của doanh nghiệp, hoặc các tài liệu chuyên ngành đã được số hóa.

- **Tập dữ liệu sử dụng:**

- Giai đoạn đầu tập trung vào **dữ liệu báo chí tiếng Việt**, cụ thể là các bài viết thu thập từ các nguồn như *Tuổi Trẻ*, *Thanh Niên*, *Kinh Tế Sài Gòn*, *Sài Gòn Giải Phóng*, *Người Lao Động*... với dữ liệu trải dài qua nhiều năm.
- Dữ liệu sẽ được **tiền xử lý** trước khi đưa vào mô hình, bao gồm các bước như: tách từ, chuẩn hóa văn bản, loại bỏ stopwords, chuẩn hóa Unicode...

- **Nội dung xử lý:**

- **Trích xuất đặc trưng văn bản** thông qua các phương pháp biểu diễn như TF-IDF, Word2Vec và Sentence-BERT.
- **Tính toán độ tương đồng** giữa văn bản đầu vào và các văn bản trong kho dữ liệu bằng các chỉ số như **Cosine Similarity**, hoặc kết hợp với các mô hình học sâu.
- **Tích hợp mô hình RAG** để truy xuất các văn bản tương đồng (Retrieval phase),

sau đó sử dụng mô hình sinh ngôn ngữ để **tóm tắt nội dung, mô tả liên quan hoặc trả lời câu hỏi** dựa trên các văn bản đã tìm được (Generation phase).

- **Xếp hạng kết quả truy xuất** theo mức độ liên quan và hỗ trợ hiển thị thông tin sinh ra cho người dùng.

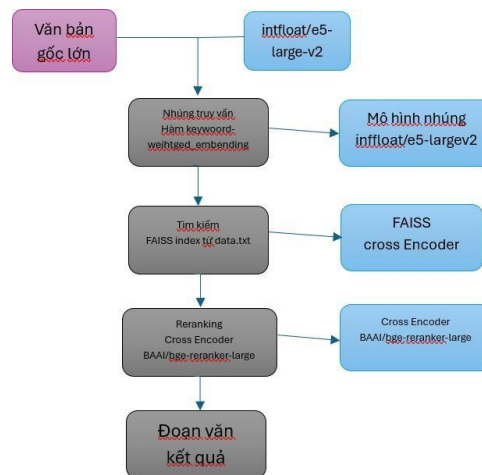
- **Công cụ và ngôn ngữ lập trình:**

- Ngôn ngữ sử dụng chính là **Python** với các thư viện hỗ trợ như scikit-learn, gensim, transformers, sentence-transformers, faiss, NLTK, spaCy...
- Các framework có thể sử dụng cho mô hình RAG bao gồm: **Haystack**, **LlamaIndex**, hoặc tích hợp thủ công qua FAISS + LLM.

- **Giới hạn của đề tài:**

- Chỉ xử lý văn bản thuần (plain text), **chưa bao gồm dữ liệu** như hình ảnh, âm thanh, video.
- Việc sinh nội dung từ mô hình RAG sẽ được thực hiện trên quy mô nhỏ, sử dụng mô hình nhẹ hoặc mã nguồn mở do giới hạn tài nguyên tính toán.
- Tập dữ liệu thử nghiệm giới hạn trong **vài ngàn văn bản**, nhằm đảm bảo hiệu quả triển khai và đánh giá trong khuôn khổ thời gian thực hiện khóa luận.
- Hệ thống **chưa áp dụng học tăng cường từ phản hồi người dùng**, tuy nhiên có thể phát triển thêm trong tương lai.

2.4 Cách tiếp cận dự kiến



2.4.1. Các nghiên cứu và công cụ có liên quan

Trong thực tế hiện nay, nhu cầu tìm kiếm và phân tích các văn bản có nội dung tương đồng ngày càng phổ biến tại các cơ quan báo chí, doanh nghiệp và tổ chức. Việc so sánh và truy xuất thông tin liên quan từ các kho dữ liệu văn bản lớn không chỉ phục vụ mục đích tra cứu mà còn hỗ trợ công tác biên tập, phân tích và ra quyết định. Một số công cụ và hướng tiếp cận đã được triển khai:

- Các hệ thống kiểm tra đạo văn như **Turnitin**, **PlagScan**, **Grammarly's plagiarism checker** sử dụng các kỹ thuật truyền thống như **TF-IDF** kết hợp **Cosine Similarity** để đo lường mức độ trùng lặp giữa các văn bản. Tuy nhiên, các hệ thống này chủ yếu tối ưu cho tiếng Anh và hoạt động chưa hiệu quả với ngôn ngữ tiếng Việt.
- Một số hệ thống nội bộ tại doanh nghiệp lớn như **VnExpress**, **Vietstock** đã triển khai tính năng nhóm bài viết theo chủ đề hoặc sự kiện, nhưng phần lớn vẫn dựa vào việc gán **tag** hoặc **truy vấn từ khóa**, chưa đủ mạnh để hiểu và xử lý nội dung ở mức ngữ nghĩa.

- Các mô hình học sâu như **Sentence-BERT** cho thấy hiệu quả vượt trội trong việc so sánh ngữ nghĩa giữa các đoạn văn bản, đặc biệt là với tiếng Việt nhờ các mô hình đã được huấn luyện như *vinai/phobert* hoặc *keepitreal/vietnamese-sbert*. Những thử nghiệm trên tập dữ liệu báo chí như *Tuổi Trẻ*, *VnExpress* cũng cho kết quả khả quan.
- Gần đây, mô hình **Retrieval-Augmented Generation (RAG)** được đề xuất như một hướng tiếp cận mạnh mẽ kết hợp giữa **khả năng truy xuất văn bản liên quan** (retrieval) và **khả năng sinh ngôn ngữ tự nhiên** (generation), cho phép hệ thống không chỉ tìm kiếm mà còn tự động **tóm tắt nội dung, trả lời câu hỏi**, hoặc mô tả mối liên hệ giữa các tài liệu.

2.4.2 Cách tiếp cận và mô hình triển khai trong đề tài

Đề tài triển khai theo quy trình gồm 5 bước chính:

1. **Tiền xử lý dữ liệu:** Làm sạch, chuẩn hóa văn bản tiếng Việt, tách câu, loại bỏ stopwords bằng *underthesea* hoặc *pyvi*.
2. **Biểu diễn văn bản:** Thử nghiệm các phương pháp embedding gồm TF-IDF, FastText và Sentence-BERT (phiên bản tối ưu cho tiếng Việt, dùng làm phương pháp chính).
3. **So sánh độ tương đồng:** Tính Cosine Similarity giữa văn bản đầu vào và kho dữ liệu để trả về danh sách Top-K văn bản tương tự nhất.
4. **Tích hợp mô hình RAG:** Kết hợp Sentence-BERT và FAISS để truy xuất văn bản liên quan, sau đó sử dụng mô hình sinh ngôn ngữ để tạo tóm tắt, trả lời hoặc mô tả liên kết giữa các văn bản.
5. **Trình bày và đánh giá:** Hiện thị kết quả có kèm phần sinh tự động từ RAG, đánh

giá chất lượng thông qua phản hồi người dùng hoặc kiểm thử trực tiếp.

2.4.3. Sự khác biệt và đóng góp

Khác với các hệ thống hiện tại chủ yếu dựa vào từ khóa hoặc metadata, đề tài này tập trung vào khả năng **hiểu nội dung văn bản ở mức ngữ nghĩa** thông qua embedding và mô hình RAG, mang lại nhiều điểm mới:

- **Tìm kiếm được các văn bản có nội dung tương tự** dù sử dụng cách diễn đạt khác nhau.
- **Hỗ trợ sinh thông tin tự động**: tóm tắt, giải thích, hoặc trả lời dựa trên nội dung truy xuất.
- **Giảm phụ thuộc vào tagging thủ công**, dễ dàng tích hợp vào hệ thống quản lý văn bản, kho báo chí hoặc thư viện tài liệu nội bộ.

2.5 Kết quả dự kiến của đề tài

Đề tài dự kiến xây dựng một **công cụ kiểm tra độ tương đồng văn bản xuyên ngữ**, kết hợp khả năng **truy xuất văn bản liên quan** và **sinh phản hồi tự động** thông qua mô hình **Retrieval-Augmented Generation (RAG)**. Kết quả chính bao gồm:

- **Sản phẩm đầu ra**: Một ứng dụng thử nghiệm có giao diện đơn giản (dòng lệnh hoặc web), hỗ trợ nhập văn bản, tìm kiếm các tài liệu tương tự trong kho dữ liệu và hiển thị kết quả kèm tóm tắt hoặc trả lời tự động.
- **Kết quả học thuật**: Báo cáo khóa luận hoàn chỉnh và tiềm năng phát triển thành **bài báo khoa học** trình bày ứng dụng trong việc xử lý văn bản tiếng Việt và các văn bản xuyên ngữ

2.6 Kế hoạch thực hiện

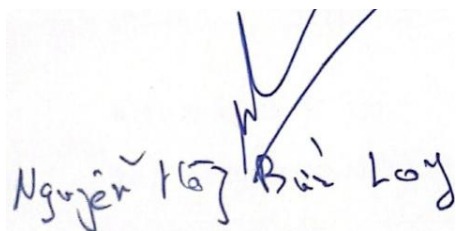
Mốc thời gian	Thời gian bắt đầu	Thời gian kết thúc	Nội dung công việc	Người thực hiện
Tuần 1	03/11/2025	09/11/2025	Thống nhất đề tài, yêu cầu, lập đề cương	An + Huy
Tuần 2 - 3	10/11/2025	23/11/2025	Nghiên cứu cơ sở lý thuyết: embedding, similarity, RAG, FAISS	An
Tuần 4 - 5	24/11/2025	07/12/2025	Khảo sát công cụ hỗ trợ RAG (Haystack, LangChain, LlamaIndex)	Huy
Tuần 6	08/12/2025	14/12/2025	Thu thập dữ liệu báo chí, chuẩn hóa định dạng	Huy
Tuần 7	15/12/2025	21/12/2025	Tiền xử lý (HTML, Unicode, tokenization, stopword)	An
Tuần 8 - 10	22/12/2025	11/01/2026	Cài đặt baseline: TF-IDF + Cosine Similarity	An
Tuần 11 - 13	12/01/2026	01/02/2026	Tích hợp Sentence-BERT, xây dựng FAISS cho tìm kiếm vector	Huy
Tuần 14 - 16	02/02/2026	22/02/2026	Phát triển mô hình RAG (Retrieval + Generation)	An + Huy
Tuần 17 - 18	23/02/2026	08/03/2026	Xây dựng giao diện demo (Flask/Streamlit)	An
Tuần 19 - 20	09/03/2026	22/03/2026	Đánh giá kết quả: độ chính xác và tốc độ phản hồi	Huy
Tuần 21 - 22	23/03/2026	05/04/2026	Viết báo cáo khóa luận và tài liệu kỹ thuật	An + Huy
Tuần 23	06/04/2026	10/04/2026	Báo cáo thử, chỉnh sửa theo góp ý GVHD, hoàn thiện	An + Huy

3. Tài liệu

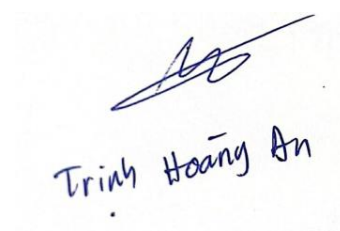
- [1] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in *Proc. of EMNLP*, 2019. [Online]. Available: <https://arxiv.org/abs/1908.10084>
- [2] VinAI Research, “PhoBERT: Pretrained language models for Vietnamese,” 2020. [Online]. Available: <https://github.com/VinAIRsearch/PhoBERT>
- [3] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in *Proc. of EMNLP*, 2019. [Online]. Available: <https://arxiv.org/abs/1908.10084>
- [4] VinAI Research, “PhoBERT: Pretrained language models for Vietnamese,” 2020. [Online]. Available: <https://github.com/VinAIRsearch/PhoBERT>
- [5] Keepitreal, “Vietnamese Sentence-BERT for semantic search,” 2022. [Online]. Available: <https://huggingface.co/keepitreal/vietnamese-sbert>

XÁC NHẬN
CỦA NGƯỜI HƯỚNG DẪN
(Ký và ghi rõ Họ tên)

TP. Hồ Chí Minh, ngày tháng năm
NHÓM SINH VIÊN THỰC HIỆN
(Ký và ghi rõ họ tên)


Nguyễn Hoàng Bách Long


Lâm Thiên Huy


Trịnh Hoàng An

Mục lục

Lời cảm ơn	i
Đề cương chi tiết	
Mục lục	ii
Danh sách hình	vii
Danh sách bảng	ix
Tóm tắt	x
1 Tổng quan	1
1.1 Giới thiệu	1
1.2 Lý do lựa chọn đề tài	3
1.3 Đối tượng và phạm vi nghiên cứu	5
1.3.1 Đối tượng nghiên cứu	5
1.3.2 Phạm vi nghiên cứu	6
1.4 Ý nghĩa khoa học và thực tiễn	7
1.4.1 Về mặt khoa học	7
1.4.2 Về mặt thực tiễn	8
1.5 Các đóng góp chính	9
1.6 Cấu trúc khoá luận	11
2 Cơ sở lý thuyết	13
2.1 Các cơ sở lý thuyết liên quan	13
2.1.1 Biểu diễn văn bản bằng Embedding	14
2.1.2 Embedding đa ngữ và truy hồi xuyên ngữ	15
2.1.3 Đo độ tương đồng bằng Cosine Similarity	15
2.1.4 Tìm kiếm lân cận gần nhất (Nearest Neighbor Search)	15
2.1.5 FAISS và chỉ mục vector	16

2.1.6	Kiến trúc truy hồi hai giai đoạn (Retrieve–Rerank)	16
2.1.7	Sinh câu hỏi–trả lời bằng mô hình ngôn ngữ lớn . .	16
2.2	Các công trình liên quan	17
2.2.1	Các công trình trên thế giới	17
2.2.2	Các công trình trong nước	20
3	Xây dựng và kiểm định bộ dữ liệu QA	24
3.1	Thu thập và tiền xử lý dữ liệu	26
3.2	Sinh câu hỏi–trả lời (QA) từ tài liệu	28
3.2.1	Mục tiêu và chiến lược sinh QA	28
3.2.2	Chuẩn bị nội dung đầu vào	29
3.2.3	Sinh QA bằng các mô hình ngôn ngữ lớn	29
3.2.4	Thiết kế prompt và kiểm soát đầu ra	30
3.2.5	Xử lý kết quả và đảm bảo tính hợp lệ	30
3.2.6	Lưu trữ và mở rộng dữ liệu	31
3.2.7	Tóm tắt	31
3.3	Kiểm định chất lượng dữ liệu QA (QA Validation Pipeline)	31
3.3.1	Chuẩn hoá và ánh xạ QA về nguồn PDF	32
3.3.2	Kiểm định ngữ nghĩa bằng Bi-Encoder (Sentence-BERT)	33
3.3.3	Kiểm định chặt bằng Cross-Encoder	34
3.3.4	Kiểm định suy luận bằng NLI (Natural Language Inference)	36
3.3.5	Nếu chỉ dùng một hoặc hai tầng kiểm định	37
3.3.6	Lọc theo ngưỡng 0.70 và 0.80	38
3.3.7	Tính uy tín và tính tương thích thực tế	38
3.4	Phân tích bộ dữ liệu thực nghiệm	39
3.4.1	Tổng quan bộ dữ liệu ở ngưỡng 0.70	40
3.4.2	Tổng quan bộ dữ liệu ở ngưỡng 0.80	41
3.4.3	Phân tích GPT-5.2 – ngưỡng 0.70	42
3.4.4	Phân tích GPT-5.2 – ngưỡng 0.80	43
3.4.5	Phân tích chi tiết Gemini 2.5 Flash – ngưỡng 0.70 .	44
3.4.6	Phân tích Gemini 2.5 Flash – ngưỡng 0.80	45
3.4.7	Phân tích DeepSeek R1T2 – ngưỡng 0.70	46

3.4.8	Phân tích DeepSeek R1T2 – ngưỡng 0.80	47
3.4.9	So sánh tổng thể giữa GPT-5.2, Gemini và DeepSeek	47
3.4.10	So sánh định lượng giữa các mô hình	49
3.4.11	Đánh giá bổ sung sau khi áp dụng NLI	49
4	Phương pháp nghiên cứu	52
4.1	Tổng quan hệ thống tìm kiếm ngữ nghĩa	52
4.1.1	Kiến trúc tổng thể	52
4.1.2	Hai cấu hình tìm kiếm song song	53
4.1.3	Nguyên lý truy hồi dựa trên embedding	54
4.1.4	Phân tách trách nhiệm giữa các thành phần	54
4.1.5	Định hướng thiết kế	54
4.2	Mô hình biểu diễn ngữ nghĩa	55
4.2.1	Yêu cầu đối với mô hình embedding	55
4.2.2	Mô hình MiniLM đa ngữ	56
4.2.3	Mô hình BGE-M3	56
4.2.4	Chuẩn hóa vector và metric tương đồng	57
4.2.5	So sánh hai mô hình	57
4.2.6	Vai trò của embedding trong toàn hệ thống	57
4.3	Xây dựng chỉ mục FAISS	58
4.3.1	Bài toán truy hồi trong không gian vector	58
4.3.2	Lựa chọn cấu trúc chỉ mục	58
4.3.3	Hai không gian embedding và hai chỉ mục độc lập	59
4.3.4	Quy trình xây dựng chỉ mục	59
4.3.5	Tổ chức lưu trữ và tải nạp chỉ mục	60
4.3.6	Khả năng mở rộng	60
4.3.7	Vai trò của FAISS trong toàn hệ thống	60
4.4	Cơ chế truy hồi và xếp hạng	61
4.4.1	Truy hồi sơ cấp (Initial Retrieval)	61
4.4.2	Chuyển từ Top- N sang Top- K	62
4.4.3	Cơ chế xếp hạng trong cấu hình MiniLM	62
4.4.4	Cơ chế xếp hạng trong cấu hình BGE	63
4.4.5	So sánh hai chiến lược xếp hạng	63
4.4.6	Ảnh hưởng của kích thước Top- N	64

4.4.7	Vai trò của cơ chế hai tầng	64
4.5	Tìm kiếm xuyên ngữ	64
4.5.1	Không gian embedding đa ngữ chung	65
4.5.2	Vấn đề alignment giữa các ngôn ngữ	65
4.5.3	Hiện tượng hubness trong không gian chiều cao . .	66
4.5.4	Chuẩn hóa truy vấn bằng dịch máy	66
4.5.5	Vai trò của cross-encoder trong truy vấn xuyên ngữ	67
4.5.6	So sánh hai chiến lược xuyên ngữ	67
4.5.7	Nhận xét	68
4.6	Triển khai hệ thống và tích hợp thực tế	68
4.6.1	Kiến trúc triển khai tổng thể	68
4.6.2	Thiết kế API truy vấn	69
4.6.3	Quản lý hai backend tìm kiếm	69
4.6.4	Quản lý tài nguyên và tối ưu hiệu năng	70
4.6.5	Cơ chế tải nạp chỉ mục	70
4.6.6	Khả năng mở rộng và tích hợp	71
4.6.7	Đánh giá tổng thể về triển khai	71
4.7	Thiết kế giao diện người dùng	71
4.7.1	Tổng quan kiến trúc frontend	71
4.7.2	Nền tảng công nghệ frontend	73
4.7.3	Cấu trúc module của hệ thống	74
4.7.4	Module Experiment Playground	74
4.7.5	Module Query Test Results	75
4.7.6	Module Dataset Management	75
4.7.7	Module QA Validation	76
4.7.8	Module QA Evaluation	76
4.7.9	Module đánh giá mô hình truy hồi	77
4.7.10	Comparative Analysis	77
4.7.11	Indexing và Chunking	77
4.7.12	Vector Database Evaluation	78
4.7.13	Luồng dữ liệu frontend	78
4.7.14	Đánh giá kiến trúc frontend	78
4.7.15	Kết luận	79

5	Thực nghiệm và đánh giá	80
5.1	Ngữ liệu	80
5.2	Thiết lập thực nghiệm	84
5.2.1	Môi trường phần cứng	84
5.2.2	Môi trường phần mềm	84
5.2.3	Dữ liệu kiểm thử	85
5.2.4	Cấu hình truy hồi	85
5.2.5	Chỉ số đánh giá	86
5.3	Đánh giá hiệu năng truy hồi	87
5.3.1	Mục tiêu đánh giá	87
5.3.2	Thiết lập thực nghiệm	87
5.3.3	Kết quả thực nghiệm với Backend 1 (MiniLM) . . .	88
5.3.4	Phân tích trực quan theo Top-K (Backend 1) . . .	90
5.3.5	So sánh hiệu năng ở mức mô hình	95
5.3.6	Kết quả thực nghiệm với Backend 2 (BGE-M3) . .	96
5.3.7	Phân tích trực quan theo Top-K (Backend 2) . . .	97
5.3.8	Model-Level Comparison (Backend 2)	102
5.3.9	Kết luận	102
6	Kết luận và hướng phát triển	104
6.1	Kết luận	104
6.2	Hướng phát triển	106
	Tài liệu tham khảo	110

Danh sách hình

3.1	Pipeline xây dựng và kiểm định bộ dữ liệu QA song ngữ	25
3.2	Tổng quan bộ dữ liệu với ngưỡng $\tau = 0.70$	40
3.3	Tổng quan bộ dữ liệu với ngưỡng $\tau = 0.80$	41
3.4	Phân tích GPT-5.2 với $\tau = 0.70$	42
3.5	Phân tích GPT-5.2 với $\tau = 0.80$	43
3.6	Phân tích Gemini 2.5 Flash với $\tau = 0.70$	44
3.7	Phân tích Gemini 2.5 Flash với $\tau = 0.80$	45
3.8	Phân tích DeepSeek R1T2 với $\tau = 0.70$	46
3.9	Phân tích DeepSeek R1T2 với $\tau = 0.80$	47
5.1	Biểu đồ tổng quan các chỉ số truy hồi Top-K khi sử dụng mô hình paraphrase-multilingual-MiniLM-L12-v2	90
5.2	Biểu đồ kết quả truy hồi Top-1 giữa các bộ dataset khi sử dụng mô hình paraphrase-multilingual-MiniLM-L12-v2	91
5.3	Biểu đồ so sánh kết quả truy hồi Top-1 giữa truy vấn tiếng Việt và tiếng Anh khi sử dụng mô hình paraphrase-multilingual-MiniLM-L12-v2	91
5.4	Biểu đồ kết quả truy hồi Top-3 giữa các bộ dataset khi sử dụng mô hình paraphrase-multilingual-MiniLM-L12-v2	92
5.5	Biểu đồ so sánh kết quả truy hồi Top-3 giữa truy vấn tiếng Việt và tiếng Anh khi sử dụng mô hình paraphrase-multilingual-MiniLM-L12-v2	93
5.6	Biểu đồ kết quả truy hồi Top-5 giữa các bộ dataset khi sử dụng mô hình paraphrase-multilingual-MiniLM-L12-v2	93
5.7	Biểu đồ so sánh kết quả truy hồi Top-5 giữa truy vấn tiếng Việt và tiếng Anh khi sử dụng mô hình paraphrase-multilingual-MiniLM-L12-v2	94
5.8	Biểu đồ kết quả truy hồi Top-10 giữa các bộ dataset khi sử dụng mô hình paraphrase-multilingual-MiniLM-L12-v2	94

5.9	Biểu đồ so sánh kết quả truy hồi Top-10 giữa truy vấn tiếng Việt và tiếng Anh khi sử dụng mô hình paraphrase-multilingual-MiniLM-L12-v2	95
5.10	Biểu đồ radar so sánh hiệu năng truy hồi giữa các bộ dataset theo các chỉ số Top-K	95
5.11	Biểu đồ so sánh tổng thể hiệu năng truy hồi giữa các bộ dataset	96
5.12	Biểu đồ tổng quan các chỉ số Top-K khi sử dụng mô hình BGE-M3	97
5.13	Biểu đồ kết quả truy hồi Top-1 giữa các bộ dataset khi sử dụng mô hình BGE-M3	98
5.14	Biểu đồ so sánh kết quả truy hồi Top-1 giữa truy vấn tiếng Việt và tiếng Anh khi sử dụng mô hình BGE-M3	98
5.15	Biểu đồ kết quả truy hồi Top-3 giữa các bộ dataset khi sử dụng mô hình BGE-M3	99
5.16	Biểu đồ so sánh kết quả truy hồi Top-3 giữa truy vấn tiếng Việt và tiếng Anh khi sử dụng mô hình BGE-M3	99
5.17	Biểu đồ kết quả truy hồi Top-5 giữa các bộ dataset khi sử dụng mô hình BGE-M3	100
5.18	Biểu đồ so sánh kết quả truy hồi Top-5 giữa truy vấn tiếng Việt và tiếng Anh khi sử dụng mô hình BGE-M3	100
5.19	Biểu đồ kết quả truy hồi Top-10 giữa các bộ dataset khi sử dụng mô hình BGE-M3	101
5.20	Biểu đồ so sánh kết quả truy hồi Top-10 giữa truy vấn tiếng Việt và tiếng Anh khi sử dụng mô hình BGE-M3	101
5.21	Biểu đồ radar so sánh hiệu năng truy hồi giữa các bộ dataset theo các chỉ số Top-K	102
5.22	Biểu đồ so sánh tổng thể hiệu năng truy hồi giữa các bộ dataset	102

Danh sách bảng

3.1	So sánh hiệu năng sinh QA giữa các mô hình ở hai ngưỡng $\tau = 0.70$ và $\tau = 0.80$	49
3.2	So sánh kết quả kiểm định QA trước và sau khi áp dụng NLI	50
5.1	So sánh trước và sau lọc (ngưỡng 0.7/0.8) cho tập GEMINI-VI (MiniLM)	88
5.2	So sánh trước và sau lọc (ngưỡng 0.7/0.8) cho tập DEEPSEEK-VI (MiniLM)	89
5.3	So sánh trước và sau lọc (ngưỡng 0.7/0.8) cho tập GPT-VI (MiniLM)	89
5.4	So sánh trước và sau lọc (ngưỡng 0.7/0.8) cho tập GEMINI-ENG (MiniLM)	89
5.5	So sánh trước và sau lọc (ngưỡng 0.7/0.8) cho tập DEEPSEEK-ENG (MiniLM)	89
5.6	So sánh trước và sau lọc (ngưỡng 0.7/0.8) cho tập GPT-EN (MiniLM)	89
5.7	So sánh hiệu quả truy hồi với các ngưỡng lọc (0.7 và 0.8) trên tập GEMINI-VI sử dụng mô hình BGE-m3	96
5.8	So sánh hiệu quả truy hồi với các ngưỡng lọc (0.7 và 0.8) trên tập DEEPSEEK-VI sử dụng mô hình BGE-m3	96
5.9	So sánh trước và sau lọc (ngưỡng 0.7/0.8) cho tập GPT-VI (BGE)	96
5.10	Kết quả đánh giá BGE-m3 trên tập GEMINI-ENG, trong đó ngưỡng 0.7 được sử dụng làm baseline.	96
5.11	Kết quả đánh giá BGE-m3 trên tập DEEPSEEK-ENG, trong đó ngưỡng 0.7 được sử dụng làm baseline.	97
5.12	So sánh trước và sau lọc (ngưỡng 0.7/0.8) cho tập GPT-EN (BGE)	97

Tóm tắt

Trong bối cảnh khối lượng tài liệu số song ngữ ngày càng gia tăng, các phương pháp tìm kiếm và kiểm tra tương đồng dựa trên từ khóa bộc lộ nhiều hạn chế khi xử lý những văn bản khác ngôn ngữ nhưng tương đồng về ngữ nghĩa. Đặc biệt, đối với tài liệu học thuật tiếng Việt và tiếng Anh, nhu cầu phát hiện nội dung liên quan, hỗ trợ truy xuất tri thức và kiểm tra trùng lặp xuyên ngữ ngày càng trở nên cấp thiết. Xuất phát từ yêu cầu đó, đề tài tập trung giải quyết bài toán kiểm tra độ tương đồng văn bản xuyên ngữ trên dữ liệu PDF học thuật.

Mục tiêu của đề tài là xây dựng một công cụ tìm kiếm ngữ nghĩa xuyên ngữ Việt–Anh (Cross-lingual Semantic Search) có khả năng truy hồi và xếp hạng các đoạn văn bản tương đồng về ngữ nghĩa mà không cần dịch toàn bộ dữ liệu sang một ngôn ngữ trung gian.

Hệ thống được xây dựng dựa trên kiến trúc tìm kiếm ngữ nghĩa xuyên ngữ không sử dụng dịch máy. Quy trình bắt đầu bằng việc trích xuất và phân đoạn 1.041 tài liệu PDF bằng kỹ thuật Semantic Chunking nhằm bảo toàn ngữ cảnh và ý nghĩa của văn bản.

Tiếp theo, các mô hình embedding đa ngữ như BGE-M3 (1024 chiều) và MiniLM (384 chiều) được sử dụng để ánh xạ văn bản vào cùng một không gian vector chung. Các vector này được lưu trữ dưới dạng chỉ mục IndexFlatIP trên thư viện FAISS với cấu hình đa luồng (36 threads), giúp tối ưu tốc độ truy hồi trong quá trình tìm kiếm.

Song song đó, một bộ dữ liệu kiểm thử “vàng” được xây dựng bằng cách sử dụng các mô hình Gemini 2.5 Flash, DeepSeek R1T2 Chimera và GPT-5.2 để sinh hơn 70,870 cặp câu hỏi–trả lời (QA). Các cặp QA này được đưa qua quy trình kiểm định nhiều tầng nhằm đảm bảo chất lượng và độ tin cậy, bao gồm:

- Tầng 1 – Bi-Encoder: sử dụng độ tương đồng cosine giữa embedding của câu hỏi, câu trả lời và ngữ cảnh để lọc sơ bộ các cặp QA không phù hợp.

- Tầng 2 – Cross-Encoder: đánh giá mức độ liên quan và tính suy diễn giữa ngữ cảnh và câu trả lời ở cấp độ token, giúp cải thiện độ chính xác so với Bi-Encoder.
- Tầng 3 – Natural Language Inference (NLI): sử dụng mô hình suy luận ngôn ngữ tự nhiên để kiểm tra mối quan hệ logic giữa ngữ cảnh (premise) và câu trả lời (hypothesis), nhằm đảm bảo rằng câu trả lời thực sự được suy ra từ ngữ cảnh, hạn chế hiện tượng suy diễn sai hoặc sinh nội dung không chính xác (hallucination).

Cuối cùng, hệ thống vận hành cơ chế Retrieve & Rerank đa tầng, kết hợp giữa Neural Cross-Encoder (áp dụng cho backend BGE) và thuật toán Heuristic Scoring (áp dụng cho backend MiniLM), nhằm tinh chỉnh thứ hạng kết quả tìm kiếm theo thời gian thực.

Kết quả thực nghiệm cho thấy việc kết hợp embedding đa ngữ, chỉ mục vector và truy hồi hai giai đoạn, cùng với cơ chế kiểm định nhiều tầng (Bi-Encoder, Cross-Encoder và NLI), là một hướng tiếp cận hiệu quả và khả thi cho bài toán kiểm tra độ tương đồng văn bản xuyên ngữ. Hệ thống có tiềm năng ứng dụng trong phát hiện đạo văn xuyên ngữ, tìm kiếm tài liệu song ngữ, xây dựng hệ thống hỏi–đáp dựa trên tài liệu và các nền tảng quản lý tri thức học thuật.

Chương 1

Tổng quan

1.1 Giới thiệu

Trong bối cảnh chuyển đổi số diễn ra mạnh mẽ trên toàn cầu, khối lượng dữ liệu văn bản được tạo ra và lưu trữ mỗi ngày tăng trưởng với tốc độ rất cao. Các nguồn dữ liệu này trải rộng từ báo điện tử, tạp chí khoa học, kho lưu trữ học thuật, nền tảng dữ liệu mở cho đến các hệ thống quản lý nội dung trực tuyến. Đặc biệt trong lĩnh vực nghiên cứu khoa học, số lượng bài báo, báo cáo và tài liệu chuyên ngành được công bố liên tục đã hình thành nên một kho tri thức đồ sộ, đa dạng về lĩnh vực và ngôn ngữ.

Sự gia tăng nhanh chóng của dữ liệu văn bản đặt ra nhu cầu cấp thiết về các hệ thống tìm kiếm và phân tích nội dung thông minh. Không chỉ dừng lại ở việc truy xuất theo từ khóa, các hệ thống hiện đại cần có khả năng hiểu ngữ nghĩa của văn bản, phát hiện các đoạn nội dung tương đồng và hỗ trợ truy vấn ở mức ý nghĩa thay vì mức bề mặt ký tự. Bài toán này có ý nghĩa quan trọng trong nhiều lĩnh vực như phát hiện đạo văn, truy vết thông tin, kiểm tra trùng lặp nội dung, tổng hợp tri thức từ nhiều nguồn, cũng như xây dựng hệ thống hỏi–đáp dựa trên ngữ cảnh tài liệu.

Các phương pháp tìm kiếm truyền thống dựa trên đối sánh từ khóa (keyword-based search) hoặc mô hình không gian vector thưa (TF-IDF, BM25) có hạn chế trong việc nắm bắt quan hệ ngữ nghĩa sâu giữa các văn bản. Khi hai đoạn văn sử dụng từ vựng khác nhau nhưng biểu đạt cùng một nội dung, các phương pháp này thường không phát hiện được mức độ tương đồng thực sự [1]. Hạn chế này trở nên rõ rệt hơn trong môi trường đa ngôn ngữ, nơi cùng một khái niệm có thể được diễn đạt bằng các hệ thống ngôn ngữ khác nhau.

Trong những năm gần đây, sự phát triển của các mô hình ngôn ngữ dựa trên kiến trúc Transformer đã mở ra hướng tiếp cận mới cho bài toán tìm

kiếm ngữ nghĩa. Các mô hình embedding dense cho phép ánh xạ văn bản vào không gian vector liên tục, trong đó các đoạn văn có nội dung tương tự sẽ nằm gần nhau theo metric hình học như cosine similarity [2]. Đặc biệt, các mô hình embedding đa ngữ có khả năng biểu diễn văn bản thuộc nhiều ngôn ngữ trong cùng một không gian vector, tạo điều kiện thuận lợi cho bài toán tìm kiếm xuyên ngữ (cross-lingual retrieval) [3].

Đề tài tập trung xây dựng một hệ thống tìm kiếm ngữ nghĩa xuyên ngữ giữa tiếng Việt và tiếng Anh, cho phép truy vấn và phát hiện các đoạn văn bản có nội dung tương đồng ngay cả khi chúng được viết bằng hai ngôn ngữ khác nhau. Thay vì dịch toàn bộ dữ liệu về một ngôn ngữ trung gian, hệ thống khai thác trực tiếp không gian embedding đa ngữ để bảo toàn ngữ nghĩa gốc và giảm sai lệch do quá trình dịch tự động.

Nguồn dữ liệu tiếng Việt được thu thập từ kho tạp chí khoa học VJOL (Vietnam Journals Online), đảm bảo tính học thuật và đa dạng lĩnh vực. Dữ liệu tiếng Anh được khai thác thông qua Semantic Scholar Graph API, cho phép truy xuất bài báo khoa học và metadata từ nhiều lĩnh vực nghiên cứu khác nhau. Cách tiếp cận này giúp hình thành một tập dữ liệu song ngữ có tính học thuật cao, phản ánh sát nhu cầu thực tế của việc tìm kiếm tài liệu nghiên cứu.

Trên cơ sở dữ liệu đã thu thập, hệ thống xây dựng tập câu hỏi–trả lời (Question–Answer pairs, QA) nhằm tạo ra tập truy vấn phục vụ đánh giá khả năng tìm kiếm ngữ nghĩa. Quá trình sinh QA được thực hiện bằng các mô hình ngôn ngữ lớn hiện đại như Gemini 2.5 Flash, DeepSeek R1T2 Chimera và GPT-5.2, áp dụng cho cả tiếng Việt và tiếng Anh. Các QA sinh tự động sau đó được lọc và kiểm định chất lượng nhằm đảm bảo độ liên quan và tính bao phủ nội dung so với tài liệu gốc.

Về mặt kỹ thuật, hệ thống sử dụng các mô hình embedding đa ngữ như BAAI/bge-m3 và paraphrase-multilingual-MiniLM để biểu diễn văn bản trong không gian vector ngữ nghĩa. Các vector được lưu trữ trong chỉ mục FAISS nhằm tối ưu tốc độ truy hồi trên tập dữ liệu lớn [4]. Bên cạnh truy hồi sơ cấp dựa trên embedding, hệ thống tích hợp cơ chế xếp hạng lại (reranking) để cải thiện độ chính xác của kết quả, bao gồm cả chiến lược heuristic kết hợp và cross-encoder transformer [5].

Toàn bộ hệ thống được xây dựng bằng ngôn ngữ Python theo kiến trúc

pipeline rõ ràng, bao gồm các bước: thu thập dữ liệu, sinh QA, kiểm định chất lượng, xây dựng embedding, lập chỉ mục và truy vấn. Thiết kế này không chỉ phục vụ mục tiêu nghiên cứu trong khuôn khổ luận văn mà còn có tiềm năng ứng dụng thực tế trong các hệ thống tra cứu tài liệu học thuật, công cụ kiểm tra tương đồng xuyên ngữ và các nền tảng khai thác tri thức song ngữ.

1.2 Lý do lựa chọn đề tài

Trong thời đại bùng nổ thông tin và chuyển đổi số mạnh mẽ hiện nay, khối lượng dữ liệu văn bản được tạo ra mỗi ngày đang tăng trưởng với tốc độ rất cao. Các tài liệu học thuật, báo cáo nghiên cứu, bài báo khoa học, tin tức và tài liệu nội bộ doanh nghiệp liên tục được sản sinh và lưu trữ dưới nhiều hình thức khác nhau. Đặc biệt, trong bối cảnh toàn cầu hóa, việc sử dụng song song tiếng Việt và tiếng Anh trong lưu trữ, trao đổi và công bố thông tin ngày càng trở nên phổ biến, nhất là tại các cơ quan nghiên cứu, trường đại học, tổ chức báo chí và doanh nghiệp đa quốc gia.

Tuy nhiên, phần lớn các hệ thống tìm kiếm và công cụ kiểm tra độ tương đồng văn bản hiện nay vẫn hoạt động chủ yếu trong phạm vi một ngôn ngữ, đặc biệt là tiếng Anh. Các công cụ truyền thống thường dựa trên so khớp từ khóa (keyword matching) hoặc các kỹ thuật thống kê đơn giản, do đó gặp nhiều hạn chế khi xử lý dữ liệu đa ngôn ngữ. Trong các trường hợp văn bản được diễn đạt bằng hai ngôn ngữ khác nhau nhưng mang nội dung tương đồng về mặt ngữ nghĩa, các hệ thống này thường không thể phát hiện được mối liên hệ giữa chúng [1].

Điều này gây ra nhiều khó khăn trong thực tiễn, đặc biệt trong các bài toán như phát hiện đạo văn xuyên ngôn ngữ, truy xuất thông tin liên quan giữa các tài liệu song ngữ, hay xây dựng hệ thống hỏi–đáp dựa trên kho tri thức đa ngữ. Ví dụ, một nội dung nghiên cứu được công bố bằng tiếng Anh có thể có phiên bản tương đương hoặc nội dung tương tự bằng tiếng Việt, nhưng nếu không có cơ chế biểu diễn ngữ nghĩa chung, hệ thống tìm kiếm sẽ không thể nhận diện được sự tương đồng này.

Trong bối cảnh đó, việc phát triển một hệ thống tìm kiếm ngữ nghĩa xuyên ngữ (cross-lingual semantic search) trở nên cần thiết và có ý nghĩa

thực tiễn cao. Khác với tìm kiếm truyền thống dựa trên từ khóa, semantic search tập trung vào việc biểu diễn nội dung văn bản dưới dạng vector ngữ nghĩa trong không gian embedding chung, từ đó đo lường mức độ tương đồng dựa trên ý nghĩa thay vì hình thức bề mặt của từ ngữ [2].

Sự phát triển mạnh mẽ của các mô hình học sâu trong lĩnh vực xử lý ngôn ngữ tự nhiên (NLP), đặc biệt là các mô hình đa ngôn ngữ như mBERT, XLM-RoBERTa, Sentence-BERT và các mô hình embedding thế hệ mới, đã mở ra khả năng xây dựng không gian biểu diễn ngữ nghĩa chung cho nhiều ngôn ngữ [3]. Bên cạnh đó, sự xuất hiện của các mô hình ngôn ngữ lớn (LLMs) cùng các kỹ thuật như embedding đa ngôn ngữ, chỉ mục vector (FAISS) và reranking ngữ nghĩa đã tạo điều kiện thuận lợi để hiện thực hóa các hệ thống semantic search có hiệu năng cao và độ chính xác tốt [4].

Tuy nhiên, mặc dù các mô hình embedding đa ngữ đã đạt được nhiều tiến bộ, vẫn tồn tại những thách thức quan trọng trong thực tế triển khai. Thứ nhất, chất lượng căn chỉnh (alignment) giữa các ngôn ngữ trong không gian embedding không phải lúc nào cũng hoàn hảo, đặc biệt với dữ liệu học thuật chứa nhiều thuật ngữ chuyên ngành. Thứ hai, truy hồi dựa thuần vào khoảng cách vector có thể chưa đủ để phân biệt các đoạn văn có nội dung gần nhau nhưng khác trọng tâm. Điều này đặt ra yêu cầu cần kết hợp thêm cơ chế xếp hạng lại (reranking) nhằm nâng cao độ chính xác [5].

Xuất phát từ những yêu cầu thực tiễn nêu trên, nhóm tác giả lựa chọn nghiên cứu và xây dựng một hệ thống semantic search xuyên ngữ Việt – Anh, với mục tiêu khai thác sức mạnh của các mô hình embedding đa ngôn ngữ và kiến trúc truy vấn theo vector. Đề tài không chỉ hướng tới việc đo lường độ tương đồng văn bản mà còn tập trung vào xây dựng một pipeline hoàn chỉnh từ thu thập dữ liệu, sinh và lọc QA, xây dựng embedding, lập chỉ mục vector, đến truy vấn và sắp xếp kết quả.

Đặc biệt, đề tài triển khai song song hai cấu hình tìm kiếm: một cấu hình nhẹ tối ưu cho hiệu năng và một cấu hình tăng cường độ chính xác thông qua cross-encoder reranking. Việc thiết kế hai backend độc lập cho phép đánh giá rõ ràng sự đánh đổi giữa tốc độ xử lý và chất lượng truy hồi, đồng thời tạo cơ sở thực nghiệm để so sánh các chiến lược tìm kiếm ngữ nghĩa hiện đại.

Qua đó, hệ thống có thể được ứng dụng trong các bài toán:

- Phát hiện đạo văn và nội dung trùng lặp xuyên ngôn ngữ.
- Tìm kiếm tài liệu song ngữ theo ngữ nghĩa thay vì từ khóa.
- Hỗ trợ xây dựng hệ thống hỏi–đáp dựa trên kho tài liệu đa ngữ.
- Phân tích và khám phá mối liên hệ ngữ nghĩa giữa các văn bản khác ngôn ngữ.

Với tính cấp thiết, tính ứng dụng cao và khả năng mở rộng trong tương lai, đề tài này không chỉ mang ý nghĩa nghiên cứu học thuật trong lĩnh vực NLP đa ngữ mà còn có tiềm năng triển khai thực tế trong các hệ thống quản lý tri thức, cơ sở dữ liệu học thuật và nền tảng tìm kiếm thông minh.

1.3 Đối tượng và phạm vi nghiên cứu

1.3.1 Đối tượng nghiên cứu

Đối tượng nghiên cứu của đề tài là các tài liệu học thuật ở định dạng PDF thuộc hai ngôn ngữ: tiếng Việt và tiếng Anh. Dữ liệu tiếng Việt được thu thập từ hệ thống Vietnam Journals Online (VJOL), trong khi dữ liệu tiếng Anh được khai thác thông qua Semantic Scholar Graph API.

Các tài liệu này chủ yếu là bài báo khoa học, công trình nghiên cứu và báo cáo học thuật thuộc nhiều lĩnh vực khác nhau. Đặc điểm chung của các tài liệu này là:

- Nội dung mang tính học thuật và chuyên môn cao.
- Câu văn dài, cấu trúc phức tạp.
- Mật độ thuật ngữ chuyên ngành lớn.

Trên tập dữ liệu này, đề tài tập trung nghiên cứu bài toán tìm kiếm ngữ nghĩa xuyên ngữ (cross-lingual semantic search), nhằm xác định mức độ tương đồng nội dung giữa các văn bản khác ngôn ngữ dựa trên ý nghĩa thay vì so khớp từ khóa [2, 3].

Khác với bài toán dịch máy hay tóm tắt văn bản, trọng tâm của đề tài không nằm ở việc sinh nội dung mới mà ở việc đo lường mức độ liên quan ngữ nghĩa giữa một truy vấn và tập tài liệu đa ngữ. Do đó, đối tượng nghiên cứu cốt lõi là cơ chế biểu diễn ngữ nghĩa, truy hồi vector và xếp hạng lại trong môi trường đa ngôn ngữ [6].

Bên cạnh dữ liệu tài liệu gốc, đề tài cũng xây dựng tập câu hỏi–trả lời (Question–Answer pairs, QA) sinh tự động từ nội dung văn bản để phục vụ đánh giá chất lượng hệ thống tìm kiếm. Tập QA này đóng vai trò là tập truy vấn kiểm thử nhằm đo lường khả năng truy hồi của hệ thống trên dữ liệu thực tế.

1.3.2 Phạm vi nghiên cứu

Đề tài tập trung xây dựng một hệ thống semantic search xuyên ngữ Việt – Anh với các nội dung chính sau:

- Thu thập và tiền xử lý dữ liệu PDF từ hai nguồn VJOL và Semantic Scholar.
- Biểu diễn văn bản bằng mô hình embedding đa ngôn ngữ để đưa các văn bản khác ngôn ngữ về cùng một không gian ngữ nghĩa.
- Xây dựng chỉ mục vector (FAISS) và cơ chế truy vấn nhằm tìm kiếm các đoạn văn bản có nội dung tương đồng [4].
- Thiết kế cơ chế xếp hạng lại (reranking) nhằm cải thiện độ chính xác của kết quả truy hồi [5].
- Sinh và kiểm định các cặp câu hỏi–trả lời từ tài liệu nhằm phục vụ đánh giá hệ thống.

Về mặt kỹ thuật, phạm vi nghiên cứu bao gồm:

- Áp dụng các mô hình embedding đa ngữ đã được huấn luyện sẵn.
- So sánh hai chiến lược truy hồi: cấu hình nhẹ (embedding + heuristic) và cấu hình tăng cường độ chính xác (embedding + cross-encoder reranking).

- Phân tích hiệu năng truy hồi dựa trên các chỉ số đánh giá định lượng.

1.4 Ý nghĩa khoa học và thực tiễn

1.4.1 Về mặt khoa học

Đề tài đóng góp một hướng tiếp cận ứng dụng các kỹ thuật hiện đại trong lĩnh vực xử lý ngôn ngữ tự nhiên (NLP) vào bài toán tìm kiếm ngữ nghĩa xuyên ngữ (cross-lingual semantic search). Thay vì dựa trên phương pháp so khớp từ khóa truyền thống, hệ thống được xây dựng dựa trên biểu diễn ngữ nghĩa của văn bản trong không gian vector đa ngôn ngữ, cho phép đo lường mức độ tương đồng nội dung giữa các văn bản tiếng Việt và tiếng Anh dựa trên ý nghĩa thay vì hình thức bề mặt của từ ngữ [2, 3].

Cụ thể, đề tài khai thác mô hình embedding đa ngôn ngữ để biểu diễn các đoạn văn bản trích xuất từ tài liệu PDF vào cùng một không gian ngữ nghĩa thống nhất. Cách tiếp cận này góp phần minh chứng tính hiệu quả của các mô hình embedding đa ngữ trong việc xử lý dữ liệu học thuật song ngữ, đồng thời làm rõ vai trò của biểu diễn ngữ nghĩa trong bài toán truy xuất thông tin hiện đại.

Bên cạnh đó, đề tài ứng dụng mô hình chỉ mục vector (FAISS) kết hợp với cơ chế truy xuất hai giai đoạn (retrieve và rerank) nhằm cân bằng giữa hiệu năng và độ chính xác. Việc kết hợp giữa truy vấn vector gần đúng (approximate nearest neighbor search) và mô hình đánh giá lại (cross-encoder reranking) thể hiện một kiến trúc đang được sử dụng phổ biến trong các hệ thống semantic search tiên tiến [4, 5]. Thông qua việc triển khai song song hai cấu hình tìm kiếm (embedding thuần kết hợp heuristic và embedding kết hợp cross-encoder), đề tài góp phần làm rõ sự đánh đổi giữa tốc độ xử lý và chất lượng truy hồi trong môi trường dữ liệu học thuật đa ngữ.

Ngoài ra, đề tài còn xây dựng một pipeline sinh và kiểm định câu hỏi-trả lời (QA) từ nội dung tài liệu bằng các mô hình ngôn ngữ lớn. Việc sử dụng các LLM để tạo và lọc QA nhằm phục vụ đánh giá hệ thống tìm kiếm góp phần làm rõ khả năng khai thác tri thức tự động từ tài liệu học thuật, đồng thời mở rộng hướng nghiên cứu về đánh giá hệ thống truy

xuất ngữ nghĩa dựa trên dữ liệu sinh tự động thay vì hoàn toàn phụ thuộc vào bộ dữ liệu gán nhãn thủ công.

Về mặt phương pháp luận, đề tài đóng góp một quy trình hoàn chỉnh bao gồm: thu thập dữ liệu song ngữ, tiền xử lý, biểu diễn embedding, lập chỉ mục vector, truy hồi hai tầng và đánh giá định lượng. Việc tích hợp các thành phần này thành một hệ thống thống nhất giúp minh họa rõ tính khả thi của việc triển khai semantic search xuyên ngữ trong môi trường thực tế.

Nhìn chung, hệ thống được xây dựng cho thấy việc kết hợp embedding đa ngôn ngữ, chỉ mục vector và cơ chế reranking có thể cải thiện đáng kể hiệu quả truy hồi ngữ nghĩa. Kết quả thực nghiệm cũng cung cấp cơ sở để so sánh các chiến lược truy hồi khác nhau trong môi trường dữ liệu học thuật song ngữ.

1.4.2 Về mặt thực tiễn

Về mặt ứng dụng, hệ thống semantic search xuyên ngữ Việt – Anh được xây dựng trong đề tài có tiềm năng triển khai trong nhiều lĩnh vực thực tế, đặc biệt trong môi trường học thuật và nghiên cứu.

Trước hết, hệ thống có thể hỗ trợ phát hiện nội dung trùng lặp hoặc đạo văn xuyên ngôn ngữ trong trường hợp nội dung được dịch hoặc diễn đạt lại từ tài liệu nước ngoài. Đây là nhu cầu thực tiễn tại các cơ sở giáo dục và tổ chức nghiên cứu khi số lượng tài liệu song ngữ ngày càng gia tăng và yêu cầu kiểm tra tính nguyên gốc ngày càng nghiêm ngặt.

Thứ hai, trong các cơ quan, tổ chức hoặc doanh nghiệp lưu trữ tài liệu bằng nhiều ngôn ngữ, hệ thống cho phép tìm kiếm thông tin theo ngữ nghĩa mà không phụ thuộc vào ngôn ngữ của truy vấn. Người dùng có thể đặt câu hỏi bằng một ngôn ngữ và nhận được tài liệu liên quan bằng ngôn ngữ khác, qua đó nâng cao hiệu quả khai thác và quản lý tri thức.

Thứ ba, pipeline sinh và kiểm định QA từ tài liệu tạo nền tảng cho việc xây dựng các hệ thống hỏi–đáp dựa trên tài liệu (document-grounded QA) hoặc chatbot học thuật. Việc chuyển đổi nội dung tài liệu thành cấu trúc QA giúp tăng khả năng truy cập, tra cứu và tái sử dụng tri thức trong các hệ thống thông minh.

Ngoài ra, kiến trúc hệ thống được thiết kế theo hướng mô-đun hóa, cho phép dễ dàng mở rộng sang các ngôn ngữ khác, tích hợp thêm nguồn dữ liệu mới hoặc thay thế mô hình embedding và reranking khi có phiên bản 10 cải tiến. Điều này tạo điều kiện thuận lợi cho việc triển khai ở quy mô lớn hơn hoặc tích hợp vào các nền tảng quản lý tri thức hiện có.

Tổng thể, đề tài không chỉ có ý nghĩa nghiên cứu trong lĩnh vực NLP đa ngữ mà còn có giá trị thực tiễn trong việc xây dựng các hệ thống tìm kiếm và quản lý tri thức thông minh trong môi trường song ngữ, góp phần thúc đẩy quá trình chuyển đổi số trong giáo dục và nghiên cứu khoa học.

1.5 Các đóng góp chính

Đề tài đạt được một số đóng góp chính về mặt hệ thống và phương pháp như sau:

- **Đề xuất và triển khai hệ thống tìm kiếm ngữ nghĩa xuyên ngữ Việt–Anh hoàn chỉnh:** Đề tài xây dựng một kiến trúc semantic search end-to-end cho tài liệu học thuật dạng PDF, bao gồm các bước trích xuất nội dung, phân đoạn văn bản (chunking), biểu diễn embedding, xây dựng chỉ mục vector và truy vấn hai giai đoạn (retrieve và rerank). Hệ thống cho phép truy vấn bằng một ngôn ngữ và tìm kiếm nội dung liên quan ở ngôn ngữ khác dựa trên ngữ nghĩa mà không cần dịch máy.
- **Xây dựng pipeline xử lý PDF song ngữ có khả năng truy vết (traceability):** Hệ thống hỗ trợ trích xuất nội dung từ PDF với cơ chế OCR fallback đối với tài liệu scan, đồng thời gán định danh chunk_id cho từng đoạn văn bản. Cơ chế này giúp truy xuất nguồn gốc nội dung, tăng tính minh bạch và khả năng kiểm chứng trong quá trình tìm kiếm và đánh giá.
- **Ứng dụng embedding đa ngữ trong không gian ngữ nghĩa thống nhất:** Đề tài sử dụng các mô hình embedding đa ngôn ngữ để ánh xạ văn bản tiếng Việt và tiếng Anh vào cùng một không gian vector. Cách tiếp cận này cho phép đo lường độ tương đồng ngữ

nghĩa xuyên ngữ trực tiếp mà không cần thông qua bước dịch trung gian.

- **Triển khai cơ chế truy hồi hai giai đoạn kết hợp tối ưu hiệu năng và độ chính xác:** Hệ thống sử dụng FAISS cho truy hồi vector ở giai đoạn đầu nhằm đảm bảo tốc độ, sau đó áp dụng reranking bằng Cross-Encoder ở giai đoạn hai để cải thiện chất lượng xếp hạng. Kiến trúc two-stage retrieval này giúp cân bằng giữa hiệu năng và độ chính xác trong môi trường dữ liệu lớn.
- **Đề xuất pipeline sinh và kiểm định QA đa tầng phục vụ đánh giá:** Đề tài xây dựng quy trình sinh tự động các cặp câu hỏi–trả lời (QA) từ tài liệu bằng các mô hình ngôn ngữ lớn, sau đó áp dụng cơ chế kiểm định ba tầng gồm: (1) Bi-Encoder lọc theo độ tương đồng cosine, (2) Cross-Encoder đánh giá mức độ liên quan ở cấp độ ngữ cảnh, và (3) Natural Language Inference (NLI) kiểm tra tính suy diễn logic giữa ngữ cảnh và câu trả lời. Cơ chế này giúp nâng cao độ tin cậy của tập QA và hạn chế hiện tượng sinh nội dung sai lệch (hallucination).
- **Xây dựng bộ dữ liệu đánh giá xuyên ngữ quy mô lớn:** Một tập dữ liệu gồm hơn 70,000 cặp QA được xây dựng và kiểm định để phục vụ đánh giá khách quan hệ thống semantic search. Bộ dữ liệu này phản ánh đa dạng nội dung học thuật và hỗ trợ đo lường hiệu quả truy hồi trong bối cảnh xuyên ngữ.
- **Thiết kế hệ thống theo hướng module hóa và dễ mở rộng:** Kiến trúc hệ thống được xây dựng theo hướng tách biệt các thành phần (data processing, embedding, retrieval, reranking, evaluation), cho phép dễ dàng mở rộng hoặc thay thế mô hình. Điều này tạo nền tảng cho các ứng dụng nâng cao như phát hiện đạo văn xuyên ngữ, hệ thống hỏi–đáp dựa trên tài liệu (RAG), và quản lý tri thức học thuật song ngữ.

Những đóng góp trên không chỉ chứng minh tính khả thi của việc xây dựng hệ thống tìm kiếm ngữ nghĩa xuyên ngữ trong môi trường học thuật,

mà còn đề xuất một quy trình đánh giá chặt chẽ thông qua pipeline kiểm định đa tầng. Kết quả của đề tài tạo tiền đề cho các nghiên cứu và ứng dụng nâng cao trong lĩnh vực truy xuất thông tin và xử lý ngôn ngữ tự nhiên đa ngữ.

1.6 Cấu trúc khoá luận

Khóa luận được tổ chức thành 6 chương chính, với nội dung cụ thể như sau:

- **Chương 1 – Tổng quan:** Trình bày bối cảnh nghiên cứu, lý do lựa chọn đề tài, mục tiêu, đối tượng và phạm vi nghiên cứu, ý nghĩa khoa học và thực tiễn, các đóng góp chính của đề tài, cũng như giới thiệu cấu trúc tổng thể của khóa luận.
- **Chương 2 – Cơ sở lý thuyết và nghiên cứu liên quan:** Trình bày các kiến thức nền tảng liên quan đến xử lý ngôn ngữ tự nhiên và truy xuất thông tin, bao gồm biểu diễn văn bản bằng embedding, mô hình embedding đa ngôn ngữ, độ đo tương đồng ngữ nghĩa, tìm kiếm vector và chỉ mục FAISS. Đồng thời, chương này tổng hợp và phân tích các công trình nghiên cứu liên quan đến semantic search và truy xuất xuyên ngữ.
- **Chương 3 – Xây dựng bộ dữ liệu và pipeline sinh QA:** Mô tả quá trình thu thập dữ liệu PDF từ VJOL và Semantic Scholar, các bước tiền xử lý, trích xuất nội dung và chia đoạn văn bản. Chương này trình bày chi tiết pipeline sinh câu hỏi–trả lời (QA) bằng các mô hình ngôn ngữ lớn, quy trình lọc và kiểm định QA, cũng như cấu trúc bộ dữ liệu được xây dựng phục vụ cho đánh giá hệ thống.
- **Chương 4 – Phương pháp nghiên cứu và xây dựng hệ thống semantic search:** Đây là chương trung tâm của khóa luận, trình bày chi tiết thiết kế và triển khai hệ thống tìm kiếm ngữ nghĩa xuyên ngữ Việt – Anh. Nội dung chương bao gồm:

- Tổng quan kiến trúc hệ thống tìm kiếm và phân tách giữa giai đoạn xây dựng chỉ mục và giai đoạn truy vấn.
- Phân tích và triển khai các mô hình embedding đa ngôn ngữ (MiniLM và BGE-M3) dùng để biểu diễn văn bản trong không gian vector ngữ nghĩa chung.
- Xây dựng chỉ mục vector bằng FAISS, tổ chức lưu trữ embedding và metadata phục vụ truy hồi hiệu quả.
- Thiết kế cơ chế truy vấn hai giai đoạn gồm truy hồi sơ cấp (retrieve) và xếp hạng lại (rerank), bao gồm cả chiến lược heuristic nhẹ và cross-encoder reranking nhằm nâng cao độ chính xác.
- Phân tích cơ chế tìm kiếm xuyên ngữ và xử lý truy vấn trong môi trường song ngữ.
- Triển khai hệ thống dưới dạng dịch vụ web, quản lý hai backend tìm kiếm và tối ưu tài nguyên tính toán.

Chương này làm rõ cách tích hợp embedding đa ngữ, chỉ mục vector và mô hình reranking thành một pipeline tìm kiếm hoàn chỉnh, đồng thời phân tích các yếu tố ảnh hưởng đến hiệu năng và chất lượng truy hồi.

- **Chương 5 – Thực nghiệm và đánh giá mô hình:** Trình bày thiết lập thực nghiệm, cấu hình hệ thống, phương pháp đánh giá và các chỉ số đo lường hiệu quả. Chương này phân tích kết quả đạt được, so sánh các cấu hình khác nhau và đánh giá ưu điểm, hạn chế của hệ thống semantic search đã xây dựng.
- **Chương 6 – Kết luận và hướng phát triển:** Tổng kết các kết quả đạt được của đề tài, khẳng định những đóng góp chính, đồng thời đề xuất các hướng nghiên cứu và mở rộng trong tương lai.

Ngoài các chương chính, khóa luận còn bao gồm phần tài liệu tham khảo và phụ lục, trong đó cung cấp thông tin bổ sung về cấu trúc dữ liệu, quy trình xử lý và các thành phần kỹ thuật liên quan đến hệ thống.

Chương 2

Cơ sở lý thuyết

2.1 Các cơ sở lý thuyết liên quan

Trong bài toán truy hồi văn bản theo ngữ nghĩa (semantic search), đặc biệt là trong bối cảnh xuyên ngữ (cross-lingual retrieval), việc biểu diễn văn bản sao cho phản ánh được nội dung ngữ nghĩa thay vì chỉ dựa trên từ khóa bề mặt là yếu tố cốt lõi.

Khác với các phương pháp truyền thống như TF-IDF hay BM25 dựa trên thống kê tần suất từ, các hệ thống hiện đại sử dụng biểu diễn vector liên tục (dense representation) để ánh xạ văn bản vào một không gian ngữ nghĩa nhiều chiều [2]. Trong không gian này, các đoạn văn có nội dung tương đồng sẽ có khoảng cách gần nhau theo một độ đo xác định.

Các nền tảng lý thuyết chính được sử dụng trong đề tài bao gồm:

- Biểu diễn văn bản bằng embedding nhằm ánh xạ nội dung ngôn ngữ sang không gian vector liên tục và bảo toàn thông tin ngữ nghĩa.
- Đo độ tương đồng giữa các văn bản bằng cosine similarity, cho phép so sánh mức độ gần nhau của các vector trong không gian embedding.
- Tìm kiếm lân cận gần nhất trong không gian nhiều chiều để truy hồi các đoạn văn có nội dung tương đồng một cách hiệu quả.
- Xây dựng chỉ mục vector với FAISS nhằm tối ưu hóa hiệu năng truy hồi trên tập dữ liệu lớn.
- Áp dụng kiến trúc truy hồi hai giai đoạn (retrieve-rerank) để cân bằng giữa tốc độ và độ chính xác.
- Sử dụng mô hình suy luận ngôn ngữ tự nhiên (Natural Language Inference – NLI) như một tầng kiểm định nhằm đảm bảo tính đúng

đẫn và tính suy diễn logic của kết quả.

2.1.1 Biểu diễn văn bản bằng Embedding

Biểu diễn văn bản (text embedding) là quá trình chuyển đổi một câu hoặc đoạn văn thành một vector số thực có kích thước cố định d , sao cho các văn bản có ý nghĩa tương tự sẽ có vector gần nhau trong không gian R^d [2].

Sự phát triển của phương pháp biểu diễn văn bản có thể chia thành các giai đoạn:

- Biểu diễn thưa (sparse representation) như Bag-of-Words và TF-IDF, trong đó văn bản được biểu diễn dựa trên tần suất xuất hiện của từ.
- Word embedding như Word2Vec và GloVe, cho phép học được biểu diễn ngữ nghĩa ở mức từ thông qua ngữ cảnh [**mikolov2013word2vec**, **pennington2014glove**].
- Sentence embedding dựa trên Transformer như BERT và Sentence-BERT, cho phép biểu diễn ngữ nghĩa ở mức câu hoặc đoạn văn [**devlin2019bert**, 2].

Trong đề tài, nhóm sử dụng mô hình đa ngữ:

`sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2`

Mô hình này cho phép ánh xạ câu tiếng Việt và tiếng Anh vào cùng một không gian embedding, từ đó hỗ trợ trực tiếp cho bài toán truy hồi xuyên ngữ mà không cần dịch trung gian.

Ưu điểm của Sentence-BERT:

- Biểu diễn câu theo ngữ nghĩa thay vì phụ thuộc vào từ khóa bề mặt.
- Hỗ trợ đa ngôn ngữ và cho phép so sánh trực tiếp giữa các ngôn ngữ khác nhau.
- Có thể encode độc lập từng câu (bi-encoder), phù hợp cho hệ thống truy hồi quy mô lớn.

- Tương thích trực tiếp với các độ đo như cosine similarity và inner product.

2.1.2 Embedding đa ngữ và truy hồi xuyên ngữ

Embedding đa ngữ được huấn luyện trên dữ liệu song ngữ hoặc đa ngữ nhằm căn chỉnh (alignment) các ngôn ngữ vào cùng một không gian vector [3]. Với hai câu tương đương về ngữ nghĩa nhưng thuộc hai ngôn ngữ khác nhau, ký hiệu là x_{vi} và x_{en} , ta kỳ vọng giá trị $\cos(f(x_{vi}), f(x_{en}))$ xấp xỉ 1. Khả năng căn chỉnh này là nền tảng của bài toán truy hồi xuyên ngữ, nơi truy vấn và tài liệu có thể thuộc các ngôn ngữ khác nhau nhưng vẫn được so khớp dựa trên nội dung ngữ nghĩa.

2.1.3 Đo độ tương đồng bằng Cosine Similarity

Sau khi văn bản được biểu diễn thành vector, độ tương đồng giữa hai văn bản được tính bằng cosine similarity theo công thức:

$$\text{sim}(A, B) = \frac{A \cdot B}{\|A\| \|B\|}$$

Đây là một trong những độ đo phổ biến nhất trong các hệ thống semantic search do không phụ thuộc vào độ lớn vector mà chỉ phản ánh góc giữa các vector.

2.1.4 Tìm kiếm lân cận gần nhất (Nearest Neighbor Search)

Với tập dữ liệu lớn, việc so sánh tuần tự giữa truy vấn và toàn bộ vector trong kho dữ liệu là không khả thi về mặt tính toán, do đó bài toán được chuyển thành việc tìm k vector gần nhất với truy vấn trong không gian R^d . Hai hướng tiếp cận chính bao gồm tìm kiếm chính xác (exact search) và tìm kiếm xấp xỉ (Approximate Nearest Neighbor – ANN) [malkov2018hnsw], trong đó ANN cho phép đánh đổi một phần độ chính xác để giảm đáng kể chi phí tính toán, đặc biệt trong không gian nhiều chiều.

2.1.5 FAISS và chỉ mục vector

Đề tài sử dụng thư viện FAISS (Facebook AI Similarity Search) [4] để xây dựng chỉ mục vector phục vụ truy hồi hiệu quả trên quy mô lớn. FAISS hỗ trợ nhiều phương pháp như IndexFlatIP cho tìm kiếm chính xác theo inner product, IVF cho phân cụm vector nhằm tăng tốc ANN, HNSW dựa trên cấu trúc đồ thị và Product Quantization để giảm bộ nhớ. Trong hệ thống thực nghiệm, phương pháp IndexFlatIP được lựa chọn nhằm đảm bảo độ chính xác cao, kết hợp với việc chuẩn hóa vector và truy hồi top- k .

2.1.6 Kiến trúc truy hồi hai giai đoạn (Retrieve–Rerank)

Các hệ thống semantic search hiện đại thường áp dụng kiến trúc hai giai đoạn nhằm tối ưu hiệu năng và chất lượng kết quả:

1. Ở giai đoạn retrieve, mô hình bi-encoder được sử dụng để mã hóa truy vấn và tài liệu độc lập, từ đó truy hồi nhanh tập ứng viên top- k bằng FAISS [2].
2. Ở giai đoạn rerank, mô hình cross-encoder được sử dụng để đánh giá chi tiết từng cặp (query, document), giúp cải thiện đáng kể độ chính xác của thứ hạng [nogueira2019bert].

Cách tiếp cận này cho phép tận dụng ưu điểm về tốc độ của bi-encoder và độ chính xác cao của cross-encoder trong cùng một hệ thống.

2.1.7 Sinh câu hỏi–trả lời bằng mô hình ngôn ngữ lớn

Để xây dựng tập truy vấn đánh giá hệ thống, đề tài sử dụng các mô hình ngôn ngữ lớn (LLMs) để sinh dữ liệu Question–Answer từ tài liệu PDF.

Các mô hình được sử dụng gồm:

- Gemini 2.5 Flash được sử dụng thông qua API chính thức nhằm đảm bảo chất lượng sinh ổn định và khả năng mở rộng.
- DeepSeek R1T2 Chimera được triển khai nội bộ nhằm kiểm soát dữ liệu và giảm phụ thuộc vào dịch vụ bên ngoài.

- GPT-5.2 được sử dụng để tăng tính đa dạng trong cách diễn đạt và hỗ trợ sinh QA trong nhiều ngữ cảnh khác nhau.

Quy trình sinh QA bao gồm việc chia văn bản thành các đoạn (chunk), yêu cầu mô hình sinh các cặp câu hỏi–trả lời dựa trên ngữ cảnh cụ thể, và lưu trữ theo cấu trúc chuẩn gồm **title**, **context**, **question**, **answer**.

Do dữ liệu sinh tự động có thể chứa sai lệch ngữ nghĩa hoặc hiện tượng hallucination, đề tài áp dụng pipeline kiểm định đa tầng, trong đó:

- Bi-encoder được sử dụng để lọc sơ bộ các cặp QA dựa trên độ tương đồng embedding.
- Cross-encoder được sử dụng để đánh giá mức độ liên quan giữa câu hỏi, câu trả lời và ngữ cảnh ở mức chi tiết hơn.
- Mô hình NLI được sử dụng ở bước cuối nhằm kiểm tra mối quan hệ suy diễn giữa context (premise) và answer (hypothesis), chỉ giữ lại các cặp thuộc lớp entailment để đảm bảo tính chính xác và độ tin cậy.

2.2 Các công trình liên quan

2.2.1 Các công trình trên thế giới

Trong lĩnh vực kiểm tra độ tương đồng văn bản và truy hồi ngữ nghĩa xuyên ngữ (cross-lingual semantic retrieval), nhiều hướng tiếp cận quan trọng đã được phát triển, từ các hệ thống thương mại truyền thống đến các mô hình học sâu hiện đại dựa trên embedding và kiến trúc nhiều tầng.

- **Turnitin và các hệ thống kiểm tra đạo văn thương mại**

Turnitin là một trong những hệ thống kiểm tra đạo văn phổ biến nhất hiện nay, trong đó các phương pháp chủ yếu dựa trên so khớp chuỗi, n-gram và so sánh văn bản sau khi dịch sang cùng một ngôn ngữ. Mặc dù có khả năng xử lý đa ngôn ngữ ở mức bề mặt, các hệ thống này còn hạn chế trong việc phát hiện tương đồng ngữ nghĩa sâu, đặc biệt khi văn bản được paraphrase hoặc thay đổi cấu trúc cú

pháp. Điều này đã thúc đẩy sự phát triển của các phương pháp dựa trên biểu diễn ngữ nghĩa.

- **Bi-Encoder và embedding ngữ nghĩa**

Sentence-BERT (SBERT) [2] mở rộng BERT bằng kiến trúc Siamese/Triplet nhằm học biểu diễn embedding ở cấp độ câu, cho phép mã hóa độc lập từng câu và tính toán độ tương đồng bằng cosine similarity. Các phiên bản đa ngữ như `paraphrase-multilingual-MiniLM-L12-v2` hỗ trợ hơn 50 ngôn ngữ và được sử dụng rộng rãi trong truy hồi xuyên ngữ.

Bên cạnh đó, các mô hình embedding thế hệ mới như BAAI/bge-m3, e5, GTE và Instructor được tối ưu trực tiếp cho bài toán retrieval, giúp cải thiện đáng kể hiệu năng ở các chỉ số Top- K . Các mô hình này cho phép ánh xạ văn bản đa ngữ vào cùng một không gian vector, đóng vai trò nền tảng cho các hệ thống truy hồi hiện đại.

- **Dense Retrieval và các biến thể nâng cao**

Dense Passage Retrieval (DPR) [6] là một trong những phương pháp tiên phong sử dụng embedding dày cho bài toán truy hồi, với kiến trúc bi-encoder mã hóa độc lập query và document. Phương pháp này đặt nền tảng cho các hệ thống retrieve-then-rerank hiện đại.

Các biến thể nâng cao như ColBERT [`colbert`] đề xuất cơ chế late interaction nhằm giữ lại thông tin ở cấp token, giúp cải thiện chất lượng truy hồi trong các truy vấn phức tạp, tuy nhiên đi kèm với chi phí tính toán cao hơn.

- **Cross-Encoder và kiến trúc reranking**

Kiến trúc retrieve-then-rerank đã trở thành tiêu chuẩn trong các hệ thống tìm kiếm hiện đại. Trong đó, bi-encoder được sử dụng để truy hồi nhanh tập ứng viên Top- k , sau đó cross-encoder đánh giá lại mức độ liên quan giữa query và context.

Các mô hình như `monoBERT`, `MiniLM`, `mMARCO` cho phép mô hình hóa tương tác đầy đủ giữa hai chuỗi văn bản, từ đó cải thiện đáng kể độ

chính xác Top-1. Tuy nhiên, chi phí tính toán cao khiến các mô hình này thường chỉ được sử dụng ở bước reranking.

- **Natural Language Inference (NLI) trong NLP hiện đại**

Các mô hình suy luận ngôn ngữ tự nhiên như XLM-R [3] và mDeBERTa-v3 được huấn luyện trên các tập dữ liệu đa ngữ như MNLI và XNLI, cho phép xác định quan hệ giữa hai câu (entailment, contradiction, neutral). Trong những năm gần đây, NLI được ứng dụng trong việc kiểm định tính đúng đắn của thông tin, đặc biệt trong các hệ thống hỏi–đáp và sinh dữ liệu tự động.

- **Pipeline kiểm định QA đa tầng**

Trong các hệ thống hiện đại, dữ liệu hỏi–đáp (QA) thường được sinh tự động bằng các mô hình ngôn ngữ lớn, dẫn đến các vấn đề như sai lệch ngữ nghĩa hoặc hallucination. Do đó, các nghiên cứu gần đây đề xuất sử dụng pipeline kiểm định đa tầng nhằm đảm bảo chất lượng dữ liệu.

Pipeline này thường bao gồm ba thành phần chính:

- **Bi-Encoder:** sử dụng embedding để đo độ tương đồng giữa question và context, giúp lọc nhanh các cặp không liên quan. Phương pháp này có ưu điểm về tốc độ nhưng hạn chế ở việc không xét tương tác chi tiết.
- **Cross-Encoder:** đánh giá mức độ liên quan bằng cách đưa đồng thời question và context vào mô hình, giúp cải thiện độ chính xác và loại bỏ các trường hợp tương đồng bề mặt.
- **NLI:** kiểm tra quan hệ suy luận giữa context và answer (entailment, contradiction, neutral), đảm bảo câu trả lời có thể được suy ra từ ngữ cảnh.

Trong đề tài, pipeline kiểm định QA được triển khai với ba mô hình cụ thể:

- Bi-Encoder: `paraphrase-multilingual-MiniLM-L12-v2`,

- Cross-Encoder: `cross-encoder/mmarco-mMiniLMv2-L12-H384-v1`,
- NLI: `MoritzLaurer/mDeBERTa-v3-base-mnli-xnli`.

Cách tiếp cận này cho phép kết hợp ưu điểm của từng tầng: tốc độ của bi-encoder, độ chính xác của cross-encoder và khả năng suy luận của NLI, từ đó nâng cao đáng kể chất lượng dữ liệu QA, đặc biệt trong môi trường xuyên ngữ.

- **Hệ thống chỉ mục vector và FAISS**

FAISS [4] là thư viện mã nguồn mở cho phép xây dựng hệ thống tìm kiếm vector quy mô lớn với hiệu năng cao. FAISS hỗ trợ nhiều phương pháp indexing như IndexFlat, IVF và HNSW, giúp cân bằng giữa tốc độ và độ chính xác trong truy hồi.

Tổng hợp lại, các công trình trên thế giới đã hình thành một pipeline truy hồi hiện đại gồm nhiều tầng xử lý, từ embedding, truy hồi, reranking đến kiểm định ngữ nghĩa. Tuy nhiên, phần lớn các nghiên cứu tập trung vào dữ liệu tiếng Anh hoặc benchmark chuẩn, trong khi việc tích hợp đầy đủ pipeline đa tầng trên dữ liệu song ngữ thực tế vẫn còn hạn chế.

Trong bối cảnh đó, đề tài kế thừa các hướng tiếp cận hiện đại, đồng thời đề xuất một pipeline hoàn chỉnh kết hợp Bi-Encoder, Cross-Encoder và NLI nhằm nâng cao độ tin cậy của dữ liệu và hỗ trợ đánh giá hiệu năng truy hồi xuyên ngữ theo các chỉ số Top- K .

2.2.2 Các công trình trong nước

Trong những năm gần đây, cộng đồng nghiên cứu Xử lý Ngôn ngữ Tự nhiên (NLP) tại Việt Nam đã có nhiều đóng góp quan trọng, đặc biệt trong phát triển mô hình ngôn ngữ và xây dựng tài nguyên dữ liệu tiếng Việt. Tuy nhiên, các nghiên cứu chuyên sâu về truy hồi ngữ nghĩa xuyên ngữ và pipeline kiểm định QA quy mô lớn, đa tầng vẫn còn tương đối hạn chế.

- **PhoBERT [7]**

PhoBERT do VinAI Research phát triển là mô hình BERT được huấn luyện riêng cho tiếng Việt trên tập dữ liệu lớn từ báo điện tử

và Wikipedia. PhoBERT đạt kết quả hàng đầu trên nhiều tác vụ chuẩn như phân loại văn bản, nhận diện thực thể (NER), phân tích cú pháp và suy luận ngôn ngữ tự nhiên (NLI).

PhoBERT đặt nền tảng cho các nghiên cứu embedding và fine-tuning mô hình Transformer cho tiếng Việt. Tuy nhiên, PhoBERT chủ yếu được sử dụng ở mức độ sentence classification hoặc token-level tasks, chưa tập trung mạnh vào retrieval xuyên ngữ hoặc các pipeline nhiều tầng phục vụ truy hồi và kiểm định dữ liệu.

- **Vietnamese Sentence-BERT và embedding tiếng Việt**

Một số phiên bản Sentence-BERT đã được huấn luyện hoặc tinh chỉnh cho tiếng Việt, ví dụ như `keepitreal/vietnamese-sbert`. Các mô hình này hỗ trợ biểu diễn câu tiếng Việt trong không gian embedding, phục vụ các tác vụ như:

- So sánh độ tương đồng câu,
- Phân cụm văn bản,
- Semantic search nội ngữ (monolingual retrieval).

Tuy nhiên, phần lớn các nghiên cứu tập trung vào tiếng Việt đơn ngữ, trong khi bài toán truy hồi xuyên ngữ (Việt–Anh) và việc kết hợp embedding với các tầng reranking hoặc kiểm định QA vẫn chưa được khai thác sâu.

- **VLSP và các bộ dữ liệu chuẩn tiếng Việt**

Hội thảo VLSP (Vietnamese Language and Speech Processing) đã tổ chức nhiều shared task liên quan đến:

- Named Entity Recognition (NER),
- Sentiment Analysis,
- Natural Language Inference,
- Machine Reading Comprehension.

Các bộ dữ liệu như **ViQuAD**, **UIT-ViIC**, **UIT-VSFC** đóng vai trò quan trọng trong huấn luyện và đánh giá mô hình tiếng Việt. Tuy nhiên, các benchmark này chủ yếu phục vụ QA hoặc classification, chưa tập trung vào đánh giá retrieval theo Top- K hoặc các hệ thống truy hồi đa tầng trong bối cảnh xuyên ngữ.

- **Các nghiên cứu phân cụm và truy hồi nội dung báo chí**

Một số công trình trong nước nghiên cứu phân nhóm bài viết báo điện tử (VnExpress, Tuổi Trẻ, v.v.) bằng cách kết hợp embedding với các thuật toán như KMeans, DBSCAN hoặc Hierarchical Clustering. Các hướng này có liên hệ với bài toán tương đồng văn bản, nhưng mục tiêu chính thường là clustering thay vì tối ưu thứ hạng Top- K trong truy hồi.

Khác với các nghiên cứu này, các hệ thống retrieval hiện đại yêu cầu kết hợp nhiều tầng xử lý (retrieval, reranking, validation) để đạt hiệu năng cao và ổn định.

- **Hệ thống hỏi–đáp tiếng Việt**

Một số đề tài nghiên cứu tại các trường như UIT, HCMUS, HUST đã xây dựng hệ thống hỏi–đáp tiếng Việt dựa trên BERT hoặc seq2seq. Các hệ thống này tập trung vào:

- Sinh câu trả lời trực tiếp từ ngữ cảnh,
- Fine-tune mô hình đọc hiểu (machine reading comprehension).

Tuy nhiên, đa phần các hệ thống QA này chưa tích hợp đầy đủ kiến trúc retrieve–rerank, cũng như chưa xây dựng pipeline kiểm định dữ liệu QA tự động nhiều tầng (bi-encoder, cross-encoder, NLI) để đảm bảo chất lượng dữ liệu.

Tổng thể, có thể nhận thấy rằng mặc dù Việt Nam đã có nhiều tiến bộ trong phát triển mô hình và dữ liệu NLP tiếng Việt, các nghiên cứu hiện tại chủ yếu tập trung vào bài toán đơn ngữ hoặc các tác vụ riêng lẻ.

Các hướng nghiên cứu về semantic retrieval xuyên ngữ quy mô lớn, kết hợp embedding đa ngữ, reranking và đặc biệt là pipeline kiểm định QA đa tầng vẫn còn tương đối hạn chế.

Do đó, đề tài đóng góp theo hướng:

- Xây dựng hệ thống truy hồi ngữ nghĩa xuyên ngữ Việt–Anh,
- Kết hợp kiến trúc retrieve–rerank hiện đại,
- Đề xuất pipeline kiểm định QA ba tầng (Bi-Encoder, Cross-Encoder, NLI),
- Phân tích hiệu năng theo các chỉ số Top- K và chi phí suy luận.

Cách tiếp cận này góp phần bổ sung một hướng nghiên cứu mới về đánh giá và kiểm định dữ liệu trong các hệ thống truy hồi xuyên ngữ tại Việt Nam.

Chương 3

Xây dựng và kiểm định bộ dữ liệu QA

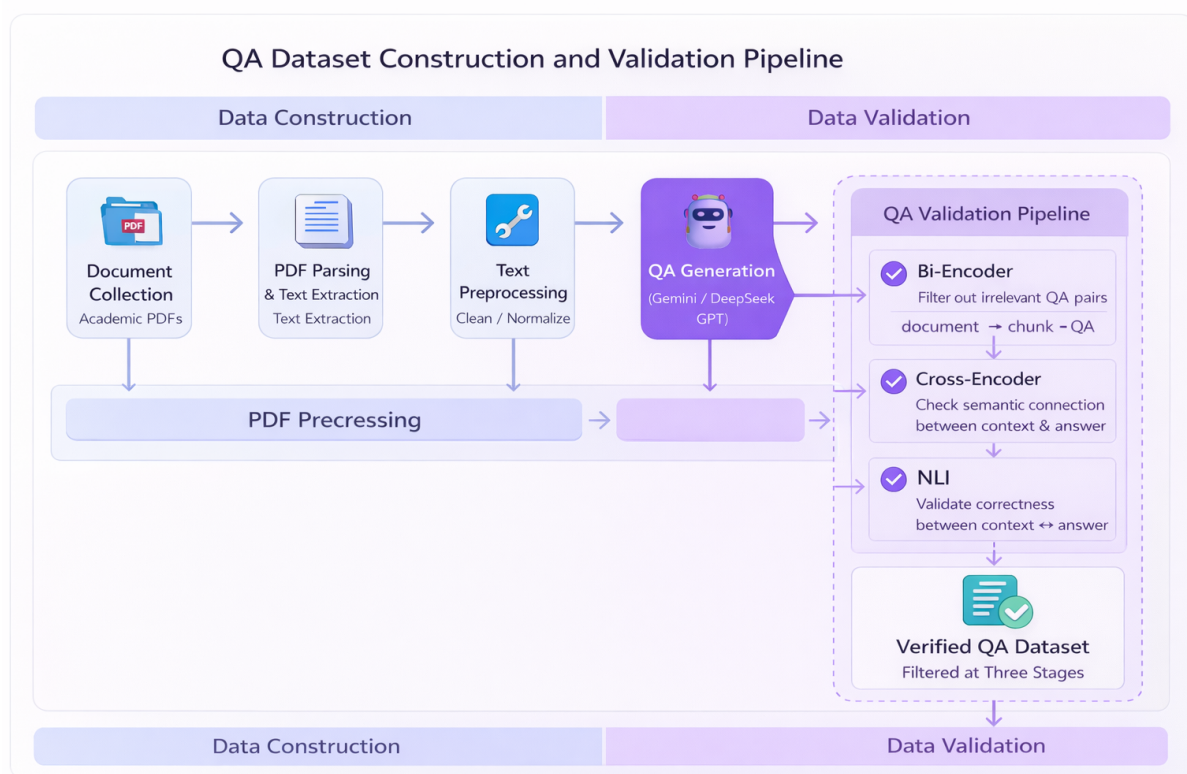
Chương này trình bày toàn bộ quy trình xây dựng bộ dữ liệu câu hỏi–trả lời (QA) và phương pháp kiểm định dữ liệu phục vụ cho hệ thống truy hồi ngữ nghĩa xuyên ngữ Việt–Anh. Khác với các hướng tiếp cận chỉ sử dụng trực tiếp dữ liệu sinh bởi mô hình ngôn ngữ, đề tài xây dựng một pipeline đầy đủ bao gồm: thu thập tài liệu nguồn, tiền xử lý PDF, sinh QA, gán truy vết theo chunk, kiểm định chất lượng đa tầng và phân tích thống kê bộ dữ liệu cuối cùng.

Mục tiêu của chương này gồm:

- Xây dựng một bộ dữ liệu QA song ngữ có khả năng truy vết về tài liệu nguồn.
- Đảm bảo dữ liệu QA sau sinh được lọc theo các tiêu chí ngữ nghĩa rõ ràng.
- Tạo ra tập dữ liệu đủ lớn, đủ sạch và đủ ổn định để phục vụ thực nghiệm semantic search ở các chương tiếp theo.

Quy trình tổng thể được minh họa trong Hình 3.1. Pipeline được chia thành hai khối chính:

- **Data Construction:** xây dựng dữ liệu từ tài liệu PDF gốc.
- **Data Validation:** kiểm định và lọc QA nhằm giữ lại các mẫu có liên kết ngữ nghĩa mạnh.



Hình 3.1: Pipeline xây dựng và kiểm định bộ dữ liệu QA song ngữ

Hình 3.1 cho thấy quy trình nghiên cứu của đề tài bắt đầu từ bước **thu thập tài liệu** từ hai nguồn chính: VJOL đối với tiếng Việt và Semantic Scholar đối với tiếng Anh. Sau đó, các tài liệu PDF được đưa qua bước **trích xuất văn bản và tiền xử lý** để chuẩn hóa nội dung, loại bỏ nhiễu và chia thành các đoạn văn bản nhỏ (chunks).

Từ nội dung đã tiền xử lý, hệ thống sử dụng các mô hình ngôn ngữ lớn để thực hiện **sinh dữ liệu QA**. Mỗi QA sau khi sinh ra không chỉ bao gồm câu hỏi, ngữ cảnh và câu trả lời, mà còn được ánh xạ ngược trở lại đoạn văn nguồn thông qua cơ chế **chunk traceability**. Thành phần này có vai trò quan trọng trong việc đảm bảo khả năng truy vết và kiểm chứng nội dung.

Ở pha tiếp theo, toàn bộ QA được đưa qua **pipeline kiểm định đa tầng**. Trước hết, bi-encoder được sử dụng để đánh giá nhanh mức độ tương đồng ngữ nghĩa giữa câu hỏi, câu trả lời và ngữ cảnh, nhằm loại bỏ các mẫu không liên quan. Tiếp theo, cross-encoder được áp dụng để kiểm tra chi tiết hơn mức độ liên quan giữa các thành phần trong QA. Cuối cùng, một mô hình suy luận ngôn ngữ tự nhiên (NLI) được sử dụng để xác

nhận rằng câu trả lời có thể được suy ra từ ngữ cảnh (entailment), qua đó đảm bảo tính đúng đắn về mặt logic.

Chỉ những QA vượt qua cả ba bước kiểm định mới được giữ lại trong **tập QA xác nhận cuối cùng**.

Một điểm cần lưu ý là pipeline trong Hình 3.1 không chỉ mô tả các bước xử lý tuần tự, mà còn thể hiện rõ hai đặc trưng quan trọng của đề tài:

- **Tính lặp (iterative construction):** nếu một tài liệu chưa đạt đủ số lượng QA xác nhận tối thiểu, hệ thống tiếp tục sinh bổ sung ở các vòng lặp sau.
- **Tính truy vết (traceability):** mỗi QA cuối cùng đều có thể ánh xạ về tài liệu PDF và đoạn văn nguồn tương ứng.

Nhờ đó, bộ dữ liệu thu được không chỉ có số lượng lớn mà còn có chất lượng ngữ nghĩa cao, phù hợp để dùng trong đánh giá hệ thống semantic search xuyên ngữ.

Nội dung chi tiết của chương được tổ chức như sau:

- **Mục 3.1** trình bày quy trình thu thập dữ liệu và tiền xử lý tài liệu PDF, bao gồm trích xuất văn bản, xử lý OCR và phân đoạn văn bản.
- **Mục 3.2** mô tả cách sinh dữ liệu QA bằng các mô hình ngôn ngữ lớn.
- **Mục 3.3** trình bày pipeline kiểm định QA đa tầng dựa trên bi-encoder, cross-encoder và NLI.
- **Mục 3.4** phân tích thống kê và chất lượng của bộ dữ liệu sau kiểm định.

3.1 Thu thập và tiền xử lý dữ liệu

Dữ liệu đầu vào của đề tài bao gồm các tài liệu học thuật ở định dạng PDF, được thu thập từ hai nguồn chính nhằm đảm bảo tính song ngữ Việt–Anh và tính đa dạng về lĩnh vực nghiên cứu. Cụ thể, dữ liệu tiếng Anh

được thu thập từ Semantic Scholar thông qua API học thuật, trong khi dữ liệu tiếng Việt được crawl từ hệ thống Vietnam Journals Online (VJOL). Việc kết hợp hai nguồn dữ liệu này giúp tạo ra một tập dữ liệu đa miền, đồng thời phù hợp cho bài toán truy hồi xuyên ngữ.

Sau khi thu thập, dữ liệu được đưa qua pipeline tiền xử lý nhằm chuẩn hóa nội dung và chuẩn bị cho các bước sinh QA và kiểm định. Pipeline này được thiết kế theo hướng robust để xử lý các trường hợp PDF không đồng nhất trong thực tế.

Quy trình tiền xử lý bao gồm các bước chính sau:

- **Trích xuất văn bản từ PDF:** Hệ thống ưu tiên sử dụng phương pháp trích xuất trực tiếp đối với các tài liệu có lớp văn bản (selectable text). Trong trường hợp không trích xuất được nội dung, cơ chế OCR được kích hoạt để nhận dạng văn bản từ ảnh. Cách tiếp cận này giúp đảm bảo không bỏ sót các tài liệu có giá trị do hạn chế về định dạng.
- **Chuẩn hóa và làm sạch dữ liệu:** Sau khi trích xuất, văn bản được xử lý để loại bỏ nhiễu như ký tự không hợp lệ, xuống dòng không nhất quán và khoảng trắng dư thừa. Việc chuẩn hóa này giúp ổn định dữ liệu đầu vào, giảm sai lệch trong các bước xử lý tiếp theo như sinh QA và tính toán embedding.
- **Phân đoạn văn bản (Chunking):** Do tài liệu học thuật thường có độ dài lớn, nội dung được chia thành các đoạn nhỏ (chunk) nhằm phù hợp với các mô hình ngôn ngữ. Việc chia đoạn được thực hiện theo nguyên tắc giữ nguyên cấu trúc đoạn văn khi có thể, kết hợp với giới hạn độ dài tối đa (khoảng 4000 ký tự) để đảm bảo tính ổn định khi xử lý.
- **Gán định danh và metadata:** Mỗi chunk được gán một mã định danh duy nhất (`chunk_id`) và lưu kèm các thông tin như độ dài văn bản, nội dung tóm tắt và trạng thái sử dụng OCR. Các metadata này hỗ trợ việc kiểm tra chất lượng dữ liệu và phục vụ cho quá trình truy vết.
- **Ánh xạ QA về dữ liệu nguồn (Traceability):** Sau khi sinh QA,

hệ thống sử dụng embedding đa ngữ để ánh xạ mỗi QA về đoạn văn phù hợp nhất trong tài liệu gốc. Cụ thể, embedding của QA (question + context) được so sánh với embedding của các chunk, và chunk có độ tương đồng cao nhất sẽ được chọn. Nhờ đó, mỗi QA đều có thể truy vết về nguồn dữ liệu ban đầu.

Tóm lại, bước thu thập và tiền xử lý dữ liệu không chỉ nhằm chuẩn hóa nội dung mà còn đóng vai trò nền tảng trong việc đảm bảo chất lượng và khả năng truy vết của bộ dữ liệu QA. Việc kết hợp giữa trích xuất linh hoạt, chunking hợp lý và ánh xạ embedding giúp tạo ra một tập dữ liệu có cấu trúc rõ ràng, ổn định và phù hợp cho các bước kiểm định và đánh giá hệ thống truy hồi xuyên ngữ ở các phần tiếp theo.

3.2 Sinh câu hỏi–trả lời (QA) từ tài liệu

Sau bước tiền xử lý và chia đoạn nội dung từ các tài liệu PDF, đề tài tiến hành sinh dữ liệu câu hỏi–trả lời (Question–Answer pairs, QA) nhằm xây dựng tập truy vấn phục vụ đánh giá hệ thống semantic search. Quá trình sinh QA được triển khai bằng cách kết hợp nhiều mô hình ngôn ngữ lớn hiện đại, bao gồm DeepSeek R1T2 Chimera, Gemini 2.5 Flash và GPT-5.2, áp dụng cho cả tiếng Việt và tiếng Anh.

Việc sử dụng nhiều mô hình khác nhau giúp tăng tính đa dạng về cách diễn đạt và mức độ hiểu ngữ nghĩa, đồng thời giảm phụ thuộc vào một nguồn sinh duy nhất. Kết quả, hệ thống xây dựng tổng cộng 6 tập dữ liệu QA tương ứng với 2 ngôn ngữ và 3 mô hình.

3.2.1 Mục tiêu và chiến lược sinh QA

Bộ dữ liệu QA được xây dựng nhằm phục vụ trực tiếp cho bài toán semantic search, với các yêu cầu chính gồm: đảm bảo mỗi câu hỏi có thể truy vết về tài liệu nguồn, hạn chế hiện tượng hallucination và duy trì tính ổn định trong thực nghiệm.

Để đạt được điều này, đề tài áp dụng chiến lược:

- Mỗi tài liệu cần có tối thiểu một số lượng QA nhất định (ví dụ 7 QA), đảm bảo đủ dữ liệu để đánh giá.

- Nếu chưa đạt ngưỡng, hệ thống tự động sinh bổ sung (augment) cho đến khi đủ.
- Các QA đã tồn tại được đưa vào prompt nhằm hạn chế sinh trùng lặp.

3.2.2 Chuẩn bị nội dung đầu vào

Do tài liệu PDF thường có độ dài lớn, hệ thống không sử dụng toàn bộ nội dung mà áp dụng chiến lược chọn mẫu nhằm cân bằng giữa độ bao phủ và giới hạn ngữ cảnh của mô hình:

- Văn bản được chia thành các chunk có độ dài tối đa khoảng 4000 ký tự.
- Một số chunk đại diện (tối đa 8) được chọn trải đều toàn bộ tài liệu.
- Các chunk này được ghép lại thành đầu vào cho mô hình sinh QA.

Cách tiếp cận này giúp đảm bảo QA phản ánh nội dung tổng thể của tài liệu, thay vì chỉ tập trung vào một đoạn nhỏ.

3.2.3 Sinh QA bằng các mô hình ngôn ngữ lớn

Quá trình sinh QA được thực hiện trên ba mô hình với các đặc điểm khác nhau:

- **DeepSeek R1T2 Chimera:** Mô hình được triển khai nội bộ thông qua thư viện `transformers`, cho phép kiểm soát toàn bộ quá trình sinh và không phụ thuộc vào API bên ngoài. Việc self-host giúp đảm bảo tính bảo mật dữ liệu và tối ưu chi phí khi xử lý khối lượng lớn.
- **Gemini 2.5 Flash:** Được sử dụng thông qua API chính thức, mô hình này có tốc độ cao và khả năng sinh nội dung ổn định. Prompt được thiết kế chặt chẽ nhằm đảm bảo đầu ra đúng định dạng và phù hợp với yêu cầu của đề tài.

- **GPT-5.2:** Được sử dụng thông qua giao diện ChatGPT, cho phép kiểm soát tốt nội dung sinh thông qua prompt. Mô hình này đóng vai trò bổ sung, giúp tăng tính đa dạng và chất lượng ngôn ngữ của dữ liệu QA.

Việc kết hợp ba mô hình giúp tận dụng điểm mạnh của từng hệ thống, từ khả năng kiểm soát (DeepSeek), tốc độ (Gemini) đến chất lượng diễn đạt (GPT).

3.2.4 Thiết kế prompt và kiểm soát đầu ra

Để đảm bảo chất lượng dữ liệu, prompt được thiết kế với các ràng buộc nghiêm ngặt:

- Chỉ sử dụng thông tin có trong tài liệu, không suy diễn ngoài nội dung.
- Mỗi QA tập trung vào một ý rõ ràng, tránh mơ hồ.
- Context gồm 3–4 câu đầy đủ để giữ ngữ nghĩa.
- Đầu ra phải đúng định dạng JSON với các trường: `title`, `context`, `question`, `answer`.

Việc kiểm soát đầu ra giúp giảm đáng kể lỗi định dạng và sai lệch nội dung.

3.2.5 Xử lý kết quả và đảm bảo tính hợp lệ

Sau khi sinh QA, hệ thống thực hiện các bước xử lý nhằm đảm bảo tính hợp lệ:

- Trích xuất và parse dữ liệu JSON từ đầu ra của mô hình.
- Lọc các QA không đủ trường hoặc sai định dạng.
- Kiểm tra tính khớp giữa `title` và tài liệu nguồn.

Chỉ các QA hợp lệ mới được đưa vào tập dữ liệu chính thức.

3.2.6 Lưu trữ và mở rộng dữ liệu

Để đảm bảo tính ổn định khi xử lý dữ liệu lớn, hệ thống áp dụng cơ chế lưu incremental, trong đó kết quả được ghi ngay sau mỗi lần sinh thành công. Điều này giúp:

- Tránh mất dữ liệu khi chương trình bị gián đoạn,
- Cho phép tiếp tục xử lý từ trạng thái trước đó,
- Hỗ trợ mở rộng dữ liệu theo thời gian.

3.2.7 Tóm tắt

Tóm lại, bước sinh QA trong đề tài được xây dựng dựa trên sự kết hợp của nhiều mô hình ngôn ngữ lớn, cùng với các cơ chế kiểm soát chặt chẽ về nội dung và định dạng. Pipeline này không chỉ tạo ra dữ liệu đa dạng mà còn đảm bảo tính nhất quán và khả năng truy vết, đóng vai trò nền tảng cho các bước kiểm định và đánh giá hệ thống semantic search xuyên ngữ.

3.3 Kiểm định chất lượng dữ liệu QA (QA Validation Pipeline)

Sau bước sinh dữ liệu câu hỏi–trả lời (QA), tập QA thô thường tồn tại nhiều vấn đề chất lượng. Trong quá trình kiểm tra thủ công một số mẫu, có thể quan sát các lỗi phổ biến như:

- Câu hỏi không thực sự bám sát nội dung đoạn văn.
- Phần **context** chứa thông tin lan man hoặc chưa được cắt đúng trọng tâm.
- Câu trả lời (**answer**) có vẻ hợp lý nhưng không được hỗ trợ trực tiếp bởi nội dung ngữ cảnh (hallucination nhẹ).
- Nhiều QA trùng lặp ý tưởng giữa các lần sinh khác nhau.

Trong thực tế, các hệ thống QA và Retrieval-Augmented Generation (RAG) quy mô lớn như của Meta (Facebook), Google, Microsoft Bing, hay các hệ thống triển khai trên Hugging Face đều không sử dụng trực tiếp dữ liệu sinh ra từ mô hình ngôn ngữ mà luôn có bước lọc, reranking hoặc xác minh ngữ nghĩa nhằm đảm bảo chất lượng đầu ra.

Ví dụ:

- Thư viện **Sentence-Transformers** khuyến nghị pipeline *retrieve-then-rerank*.
- Meta AI áp dụng kiến trúc **two-stage ranking**.
- Các hệ thống RAG hiện đại luôn có bước semantic filtering và reranking.

Tuy nhiên, các pipeline hai tầng truyền thống (bi-encoder + cross-encoder) chủ yếu đảm bảo *relevance* và *precision*, nhưng chưa đảm bảo *tính đúng đắn về mặt suy luận* (logical correctness).

Do đó, đề tài mở rộng pipeline kiểm định thành **ba tầng**, bổ sung thêm bước kiểm định suy luận bằng NLI nhằm nâng cao độ tin cậy của dữ liệu QA.

Pipeline kiểm định gồm năm bước chính:

1. Chuẩn hoá và gán nguồn PDF.
2. Kiểm định ngữ nghĩa bằng bi-encoder (Sentence-BERT).
3. Kiểm định chặt bằng cross-encoder (pairwise scoring).
4. Kiểm định suy luận bằng NLI (entailment verification).
5. Gộp, khử trùng lặp và xuất tập dữ liệu cuối.

3.3.1 Chuẩn hoá và ánh xạ QA về nguồn PDF

Mỗi QA được chuẩn hoá theo schema thống nhất gồm:

```
qa_id, model, language, source_pdf, pdf_match_score,  
chunk_id, title, context, question, answer
```

Tiêu đề được chuẩn hoá (lowercase, loại bỏ punctuation, chuẩn hoá Unicode) trước khi so khớp với danh sách file PDF bằng độ tương đồng chuỗi.

Bước này đảm bảo:

- Mỗi QA có thể truy vết về tài liệu nguồn.
- Dễ dàng phân tích thống kê theo từng PDF.
- Hỗ trợ resume và debug trong các lần chạy nhiều thí nghiệm.

Đây là bước quan trọng để đảm bảo tính reproducibility trong nghiên cứu.

3.3.2 Kiểm định ngữ nghĩa bằng Bi-Encoder (Sentence-BERT)

Đề tài sử dụng mô hình đa ngữ:

`sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2`

Mô hình này được huấn luyện để tối ưu semantic similarity và được sử dụng rộng rãi trong semantic search, question answering và clustering.

Embedding được tính cho:

- Question
- Context
- Answer

Hai độ tương đồng cosine được tính:

$$sim_{qc} = cosine(question, context)$$

$$sim_{ac} = cosine(answer, context)$$

QA được giữ lại nếu:

$$sim_{qc} \geq 0.70 \quad \wedge \quad sim_{ac} \geq 0.70$$

Ý nghĩa hai chiều kiểm định. Việc kiểm tra đồng thời cả (question-context) và (answer-context) giúp:

- Loại bỏ câu hỏi không liên quan đến nội dung.
- Loại bỏ câu trả lời không có bằng chứng trong đoạn văn.
- Giảm nguy cơ giữ lại QA có tính liên quan bề mặt nhưng sai nội dung.

Liên hệ thực tế. Bi-encoder + cosine similarity là nền tảng của semantic search hiện đại. Trong hệ sinh thái Hugging Face, các ví dụ chính thức về semantic search đều dựa trên mô hình embedding và cosine similarity.

Ưu điểm:

- Tốc độ cao.
- Có thể tiền tính embedding.
- Phù hợp xử lý hàng chục nghìn đoạn văn.

Tuy nhiên, bi-encoder không xét tương tác token-level giữa hai văn bản, do đó cần các bước kiểm định sâu hơn ở các tầng tiếp theo để đảm bảo độ chính xác và tính đúng đắn của dữ liệu.

3.3.3 Kiểm định chặt bằng Cross-Encoder

Sau bước lọc ban đầu bằng bi-encoder, đề tài sử dụng cross-encoder nhằm kiểm định chặt chẽ hơn mối quan hệ giữa **context** và **answer**. Mô hình được sử dụng là:

`cross-encoder/mmarco-mMiniLMv2-L12-H384-v1`

Khác với bi-encoder mã hoá độc lập từng văn bản, cross-encoder đưa đồng thời hai chuỗi (**context**, **answer**) vào cùng một mô hình, từ đó cho phép học trực tiếp tương tác ở mức token-level. Điều này giúp mô hình nắm bắt tốt hơn các quan hệ ngữ nghĩa phức tạp.

Xác suất phù hợp được tính theo:

$$p = \sigma(\text{logit})$$

QA được giữ lại nếu:

$$p \geq 0.70$$

Vai trò của cross-encoder. So với bi-encoder, cross-encoder mang lại các lợi ích quan trọng:

- **Đánh giá ngữ nghĩa chi tiết hơn:** Cross-encoder xét toàn bộ tương tác giữa từng token của **context** và **answer**, thay vì chỉ so sánh embedding tổng quát. Nhờ đó, mô hình có thể phát hiện các trường hợp mà câu trả lời chỉ giống về mặt từ vựng nhưng sai về nội dung.
- **Phát hiện mâu thuẫn tinh vi:** Trong nhiều trường hợp, câu trả lời có vẻ hợp lý nhưng thực tế lại mâu thuẫn với nội dung trong context. Cross-encoder có khả năng nhận diện các mâu thuẫn này tốt hơn so với phương pháp cosine similarity.
- **Cải thiện độ chính xác trong reranking:** Do được huấn luyện trực tiếp trên bài toán ranking (ví dụ MS MARCO), cross-encoder giúp tăng đáng kể precision, đặc biệt ở các vị trí Top-1 và Top-*k* thấp.

Liên hệ với các hệ thống thực tế. Chiến lược *retrieve-then-rerank* là kiến trúc chuẩn trong các hệ thống Information Retrieval và RAG hiện đại. Trong đó:

- **Tầng truy hồi (Candidate Generation):** Sử dụng các mô hình nhẹ (bi-encoder hoặc BM25) để lọc nhanh một tập nhỏ ứng viên từ dữ liệu lớn. Tầng này ưu tiên *recall cao* và hiệu năng xử lý.
- **Tầng chấm lại (Neural Re-ranking):** Sử dụng các mô hình sâu hơn như cross-encoder để đánh giá lại từng cặp (query, document)

hoặc (context, answer). Mục tiêu là tối ưu *precision* bằng cách xét tương tác chi tiết giữa hai chuỗi.

Kiến trúc này được áp dụng rộng rãi trong các hệ thống production như Meta, Google, Microsoft Bing và các thư viện NLP trên Hugging Face.

Trong đề tài, bi-encoder đóng vai trò lọc nhanh các QA có liên quan, còn cross-encoder đóng vai trò kiểm định chặt nhằm đảm bảo sự nhất quán giữa **context** và **answer**. Tuy nhiên, việc dừng lại ở hai tầng vẫn chưa đủ để đảm bảo tính đúng đắn về mặt suy luận, do đó cần bổ sung thêm bước kiểm định bằng NLI.

3.3.4 Kiểm định suy luận bằng NLI (Natural Language Inference)

Để đảm bảo tính đúng đắn về mặt logic của dữ liệu QA, đề tài bổ sung tầng kiểm định thứ ba dựa trên mô hình NLI:

MoritzLaurer/mDeBERTa-v3-base-mnli-xnli

Trong thiết lập này:

- **Premise** được định nghĩa là **context**.
- **Hypothesis** được định nghĩa là **answer**.

Mô hình dự đoán mối quan hệ giữa hai câu (entailment, contradiction, neutral), và QA chỉ được giữ lại nếu:

$$label = entailment \quad \wedge \quad p(entailment) \geq 0.70$$

Vai trò của NLI. Nếu bi-encoder và cross-encoder trả lời câu hỏi:

“Có liên quan không?”

thì NLI trả lời:

“Có đúng không?”

Cụ thể, NLI giúp:

- **Loại bỏ QA sai logic:** Một số QA có độ tương đồng cao nhưng câu trả lời không thực sự được suy ra từ context. NLI giúp phát hiện và loại bỏ các trường hợp này.
- **Giảm hallucination:** Trong dữ liệu sinh tự động, hallucination là lỗi phổ biến. NLI đóng vai trò như một bước kiểm chứng cuối cùng để đảm bảo nội dung không bị suy diễn ngoài dữ liệu gốc.
- **Tăng khả năng kiểm chứng:** Mỗi QA sau khi qua NLI đều có thể được truy vết và xác minh trực tiếp từ tài liệu nguồn, nâng cao độ tin cậy của tập dữ liệu.

3.3.5 Nếu chỉ dùng một hoặc hai tầng kiểm định

Việc sử dụng pipeline nhiều tầng là cần thiết do mỗi phương pháp chỉ giải quyết một khía cạnh của bài toán:

- **Chỉ dùng bi-encoder:** Mặc dù có tốc độ xử lý cao, bi-encoder chỉ đo độ tương đồng tổng quát và không thể phát hiện các lỗi logic tinh vi. Điều này dẫn đến việc giữ lại các QA có vẻ liên quan nhưng sai nội dung.
- **Chỉ dùng cross-encoder:** Cross-encoder có độ chính xác cao hơn nhưng chi phí tính toán lớn, khiến việc áp dụng trực tiếp trên toàn bộ dữ liệu trở nên không khả thi. Ngoài ra, nó vẫn chưa đảm bảo hoàn toàn tính suy luận logic.
- **Không sử dụng NLI:** Nếu không có tầng NLI, hệ thống không thể xác minh rằng câu trả lời thực sự được suy ra từ context. Điều này làm tăng nguy cơ tồn tại các QA sai logic trong tập dữ liệu cuối.

Kết luận. Pipeline ba tầng cho phép kết hợp ưu điểm của từng phương pháp:

$$Relevance (Bi-Encoder) \rightarrow Precision (Cross-Encoder) \rightarrow Correctness (NLI)$$

Nhờ đó, dữ liệu QA không chỉ liên quan mà còn chính xác và có thể kiểm chứng.

3.3.6 Lọc theo ngưỡng 0.70 và 0.80

Sau khi qua ba tầng kiểm định, chỉ những QA thỏa:

$$verified_{bi} \wedge verified_{cross} \wedge verified_{nli}$$

mới được giữ lại.

Đề tài xây dựng hai tập dữ liệu với các mức ngưỡng khác nhau:

- **Ngưỡng 0.70 (tập cân bằng):** Được sử dụng cho mục tiêu đánh giá tổng thể hệ thống. Ngưỡng này đảm bảo giữ lại đủ dữ liệu để duy trì recall, đồng thời vẫn loại bỏ phần lớn các QA không đạt yêu cầu.
- **Ngưỡng 0.80 (tập chất lượng cao):** Được sử dụng cho các phân tích yêu cầu độ chính xác cao hoặc trình diễn kết quả. Ngưỡng này giúp loại bỏ tối đa nhiễu và chỉ giữ lại các QA có độ tin cậy cao nhất.

Giải thích lựa chọn ngưỡng. Trong không gian embedding chuẩn hoá, giá trị cosine ≥ 0.70 thường biểu thị sự tương đồng mạnh về nội dung, trong khi ≥ 0.80 thể hiện mức độ gần như trùng khớp về ý nghĩa. Việc lựa chọn hai ngưỡng này giúp cân bằng giữa số lượng và chất lượng dữ liệu.

3.3.7 Tính uy tín và tính tương thích thực tế

Pipeline kiểm định của đề tài tương thích với các kiến trúc phổ biến trong thực tế như semantic search, Retrieval-Augmented Generation (RAG) và two-stage ranking trong Information Retrieval.

Việc mở rộng từ kiến trúc hai tầng sang ba tầng (bi-encoder, cross-encoder và NLI) giúp pipeline không chỉ đảm bảo mức độ liên quan và độ chính xác, mà còn đảm bảo tính đúng đắn về mặt suy luận.

Nhờ đó, tập dữ liệu QA cuối cùng không chỉ là dữ liệu sinh tự động mà là dữ liệu đã qua kiểm định nhiều tầng, có khả năng truy vết nguồn rõ

ràng và đủ tin cậy để phục vụ cho việc xây dựng semantic index cũng như đánh giá hệ thống truy hồi xuyên ngữ.

3.4 Phân tích bộ dữ liệu thực nghiệm

Phần này trình bày phân tích định lượng bộ dữ liệu QA sau khi trải qua pipeline kiểm định hai tầng (bi-encoder + cross-encoder).

Trong giai đoạn đầu, đề tài được xây dựng theo hướng Information Retrieval, do đó pipeline kiểm định tập trung vào hai thành phần chính:

- Bi-encoder để lọc nhanh các QA có liên quan (relevance filtering)
- Cross-encoder để kiểm định chi tiết mối quan hệ giữa context và answer (precision refinement)

Tuy nhiên, qua quá trình thực nghiệm và đánh giá thủ công, nhận thấy rằng pipeline hai tầng này vẫn chưa đảm bảo hoàn toàn tính đúng đắn về mặt suy luận (logical correctness). Do đó, một bước kiểm định bổ sung bằng NLI được áp dụng ở giai đoạn sau nhằm cải thiện chất lượng dữ liệu.

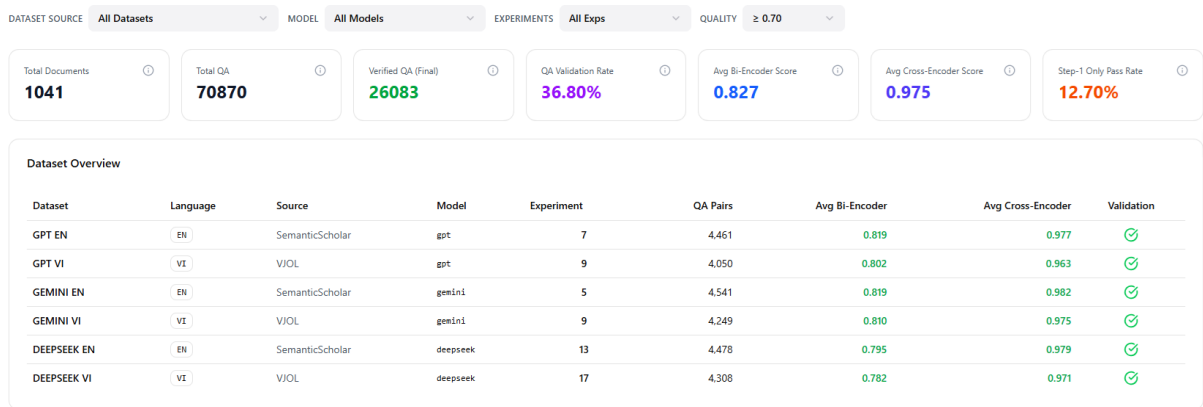
Hai mức ngưỡng được sử dụng trong phân tích:

- $\tau = 0.70$ (tập chuẩn – cân bằng giữa recall và precision)
- $\tau = 0.80$ (tập chất lượng cao – ưu tiên precision)

Tổng số tài liệu nguồn: **1041 PDF** Tổng số QA sinh ban đầu: **70,870 QA**

Các kết quả dưới đây phản ánh hiệu quả của pipeline hai tầng, trước khi áp dụng bước kiểm định NLI.

3.4.1 Tổng quan bộ dữ liệu ở ngưỡng 0.70



Hình 3.2: Tổng quan bộ dữ liệu với ngưỡng $\tau = 0.70$

Tại ngưỡng $\tau = 0.70$:

- QA được xác nhận cuối cùng: **26,083**
- Tỷ lệ validation: **36.80%**
- Avg Bi-Encoder Score: 0.827
- Avg Cross-Encoder Score: 0.975

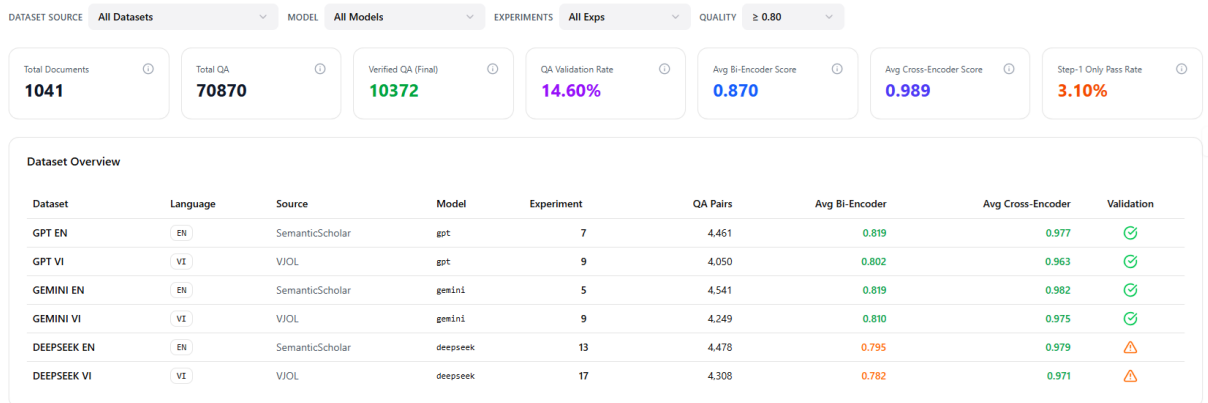
Phân tích chi tiết:

Tỷ lệ 36.80% cho thấy gần 2/5 số QA sinh ra đáp ứng đồng thời cả hai điều kiện kiểm định ngữ nghĩa. Đây là mức giữ lại tương đối cao đối với dữ liệu sinh tự động, chứng tỏ mô hình sinh QA ban đầu đã tạo ra lượng lớn câu hỏi có liên kết nội dung hợp lý với tài liệu nguồn.

Điểm trung bình bi-encoder (0.827) phản ánh mức độ tương đồng ngữ nghĩa khá cao giữa question/context và answer/context. Đặc biệt, điểm trung bình cross-encoder đạt 0.975 cho thấy các QA vượt qua hai tầng kiểm định có mức entailment rất mạnh, tức là câu trả lời gần như luôn được hỗ trợ trực tiếp bởi ngữ cảnh.

Biểu đồ tổng quan cũng cho thấy sự phân bố khá đồng đều giữa các mô hình sinh (GPT, Gemini, DeepSeek) và giữa hai ngôn ngữ (EN, VI), chứng tỏ pipeline kiểm định hoạt động ổn định xuyên ngữ.

3.4.2 Tổng quan bộ dữ liệu ở ngưỡng 0.80



Hình 3.3: Tổng quan bộ dữ liệu với ngưỡng $\tau = 0.80$

Tại ngưỡng $\tau = 0.80$:

- QA được xác nhận cuối cùng: **10,372**
- Tỷ lệ validation: **14.60%**
- Avg Bi-Encoder Score: 0.870
- Avg Cross-Encoder Score: 0.989

Phân tích chi tiết:

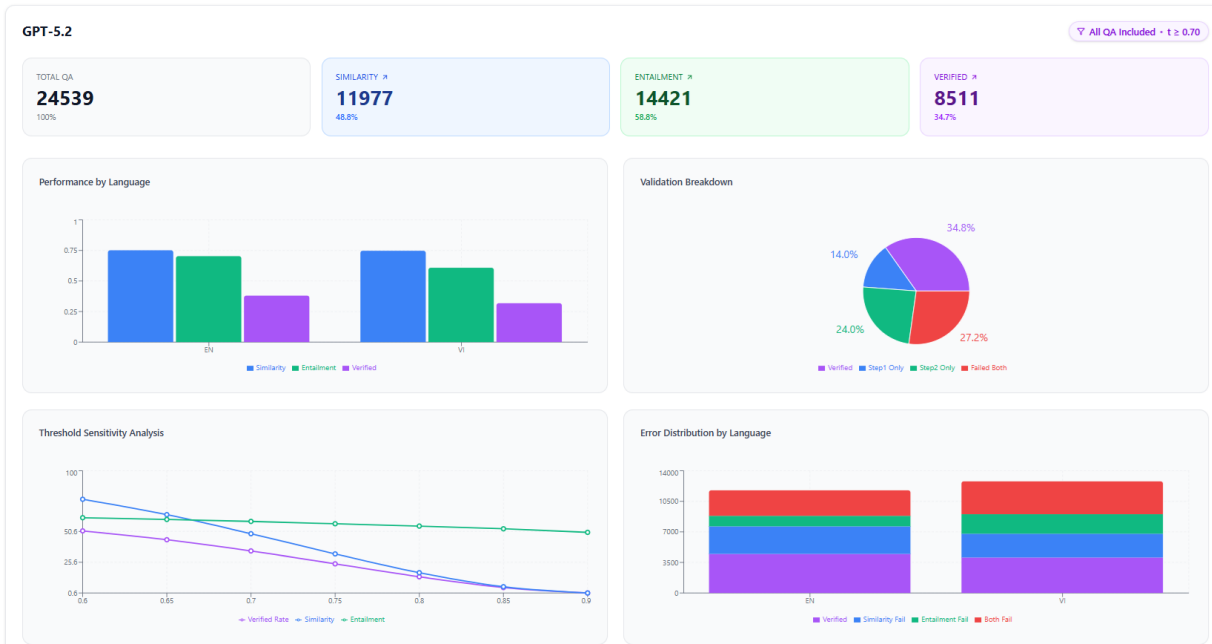
Khi tăng ngưỡng từ 0.70 lên 0.80:

- Số lượng QA giảm khoảng 60%.
- Điểm trung bình bi-encoder tăng từ 0.827 lên 0.870.
- Điểm cross-encoder tăng lên gần mức tối đa (0.989).

Điều này phản ánh đúng đặc tính của cosine similarity và entailment scoring: khi yêu cầu mức tương đồng cao hơn, hệ thống giữ lại ít QA hơn nhưng độ tin cậy ngữ nghĩa tăng đáng kể.

Ngưỡng 0.80 do đó tạo ra một tập dữ liệu có precision rất cao, phù hợp cho các thí nghiệm yêu cầu độ chính xác nghiêm ngặt hoặc dùng làm tập chuẩn (gold-like subset).

3.4.3 Phân tích GPT-5.2 – ngưỡng 0.70



Hình 3.4: Phân tích GPT-5.2 với $\tau = 0.70$

Tổng QA sinh bởi GPT-5.2: 24,539 QA được xác nhận: 8,511 (34.7%)

Phân tích:

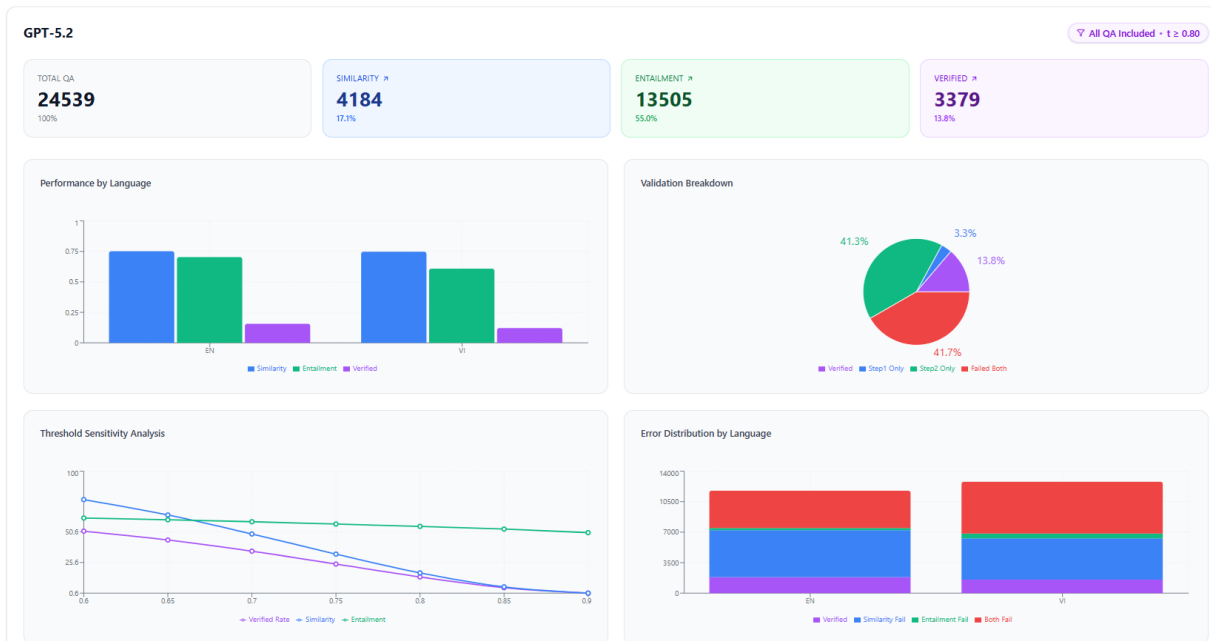
GPT-5.2 có tỉ lệ verified gần với mức trung bình toàn hệ thống (36.80%), thấp hơn Gemini nhưng cao hơn DeepSeek. Điều này cho thấy:

- Chất lượng QA tương đối ổn định, cân bằng giữa độ đa dạng và độ chính xác.
- Tỉ lệ QA vượt qua similarity đạt 48.8%, cho thấy câu hỏi có mức liên kết ngữ nghĩa tốt với context.
- Tuy nhiên, một phần QA không vượt qua bước entailment, phản ánh việc answer chưa luôn được hỗ trợ trực tiếp bởi context.

Biểu đồ Validation Breakdown cho thấy một lượng đáng kể QA chỉ vượt qua một trong hai tầng kiểm định, cho thấy dữ liệu sinh có độ đa dạng cao nhưng vẫn tồn tại nhiều nhất định.

Hiệu năng giữa tiếng Anh và tiếng Việt không có sự chênh lệch đáng kể, chứng tỏ GPT-5.2 hoạt động ổn định trong môi trường xuyên ngữ.

3.4.4 Phân tích GPT-5.2 – ngưỡng 0.80



Hình 3.5: Phân tích GPT-5.2 với $\tau = 0.80$

QA được xác nhận còn lại: 3,379 (13.8%)

Phân tích:

Khi tăng ngưỡng từ 0.70 lên 0.80:

- Tỷ lệ verified giảm đáng kể từ 34.7% xuống 13.8%.
- Số lượng QA vượt qua similarity giảm mạnh.
- Cross-encoder vẫn duy trì mức cao, cho thấy các QA được giữ lại có độ tin cậy cao.

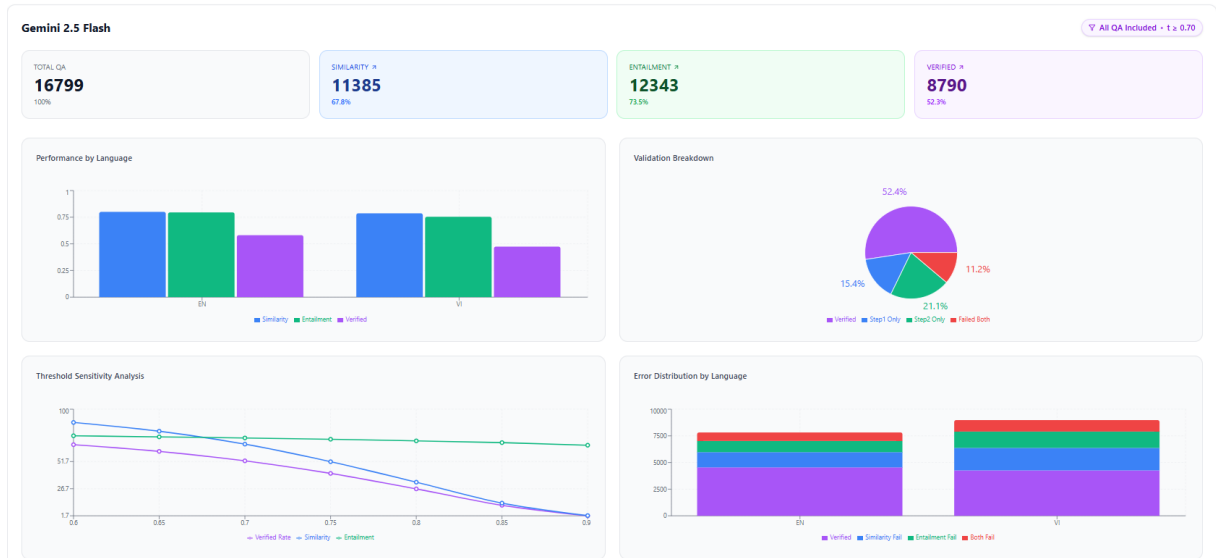
Điều này cho thấy phân bố chất lượng QA của GPT-5.2 tập trung nhiều ở khoảng trung bình (0.70–0.85), tương tự xu hướng chung của các mô hình sinh.

Nhận định:

GPT-5.2 đóng vai trò trung gian giữa Gemini và DeepSeek:

- Ổn định hơn DeepSeek
- Nhưng chưa đạt độ chính xác cao như Gemini
- Phù hợp để bổ sung đa dạng dữ liệu trong pipeline

3.4.5 Phân tích chi tiết Gemini 2.5 Flash – ngưỡng 0.70



Hình 3.6: Phân tích Gemini 2.5 Flash với $\tau = 0.70$

Tổng QA sinh bởi Gemini: 16,799 QA được xác nhận: 8,790 (52.3%)

Nhận định:

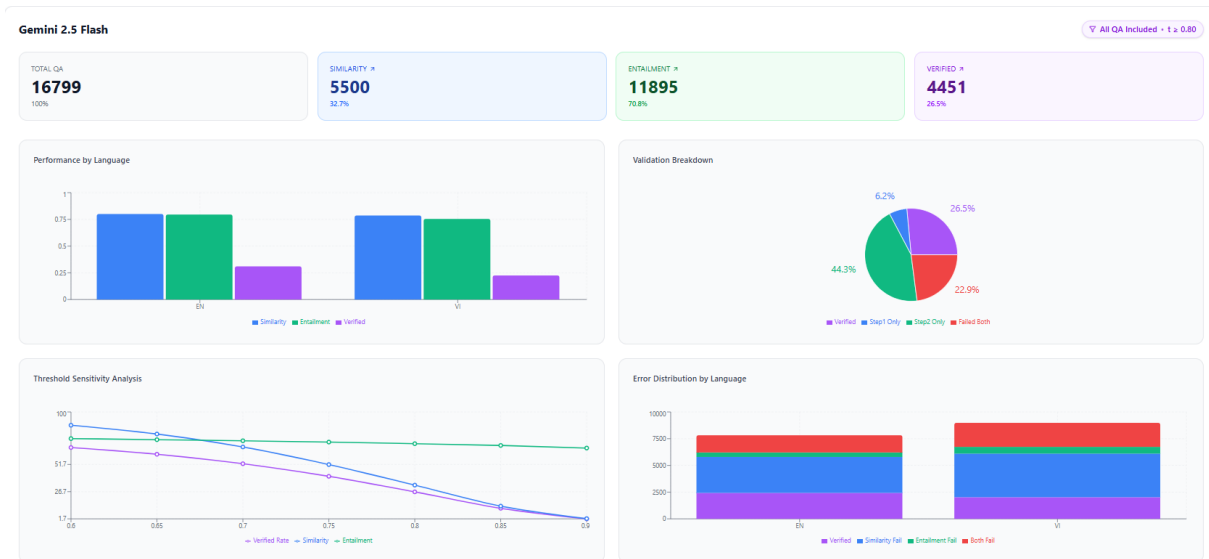
Gemini có tỉ lệ verified cao hơn đáng kể so với mức trung bình toàn hệ thống (37.40%). Điều này cho thấy:

- Câu hỏi sinh ra bám sát nội dung tài liệu.
- Mức độ liên kết answer–context ổn định.
- Tỉ lệ fail cả hai tầng thấp hơn DeepSeek và GPT.

Biểu đồ Threshold Sensitivity cho thấy khi tăng τ , đường Verified giảm tương đối tuyến tính. Điều này phản ánh sự phân bố chất lượng QA tương đối đồng đều: nhiều QA nằm trong khoảng similarity 0.70–0.85.

Đáng chú ý, hiệu năng giữa EN và VI không chênh lệch lớn, chứng tỏ mô hình sinh QA hoạt động ổn định xuyên ngữ.

3.4.6 Phân tích Gemini 2.5 Flash – ngưỡng 0.80



Hình 3.7: Phân tích Gemini 2.5 Flash với $\tau = 0.80$

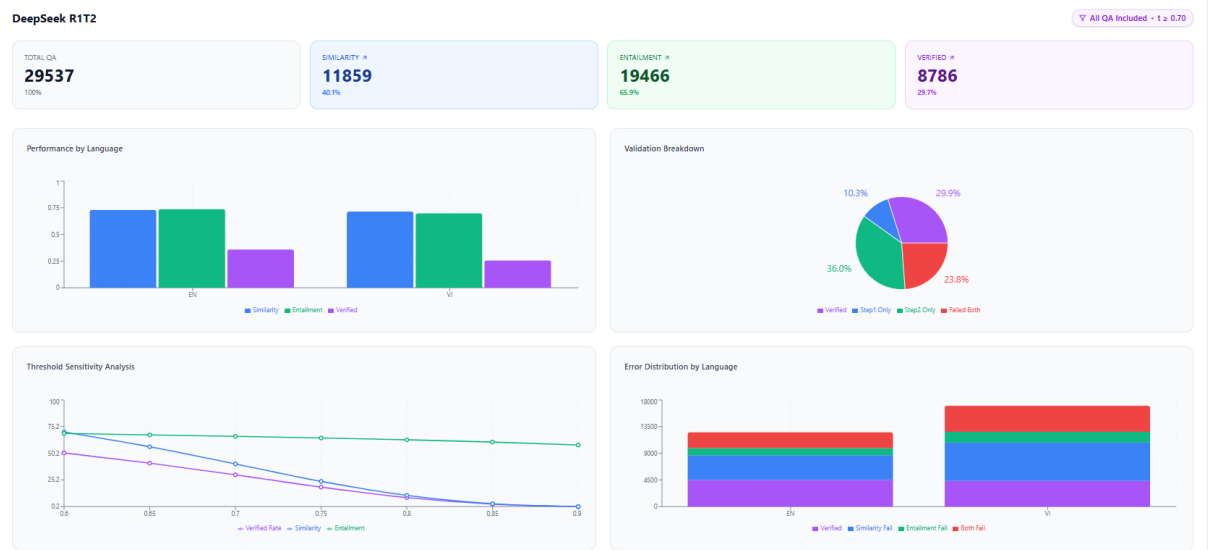
QA được xác nhận còn lại: 4,451 (26.5%)

So với ngưỡng 0.70:

- Tỷ lệ verified giảm nhưng vẫn cao hơn DeepSeek và GPT.
- Cross-encoder vẫn giữ điểm rất cao.

Điều này cho thấy Gemini có độ ổn định cao khi yêu cầu tiêu chuẩn ngữ nghĩa nghiêm ngặt hơn.

3.4.7 Phân tích DeepSeek R1T2 – ngưỡng 0.70



Hình 3.8: Phân tích DeepSeek R1T2 với $\tau = 0.70$

Tổng QA sinh bởi DeepSeek: 29,537 QA được xác nhận: 8,786 (29.7%)

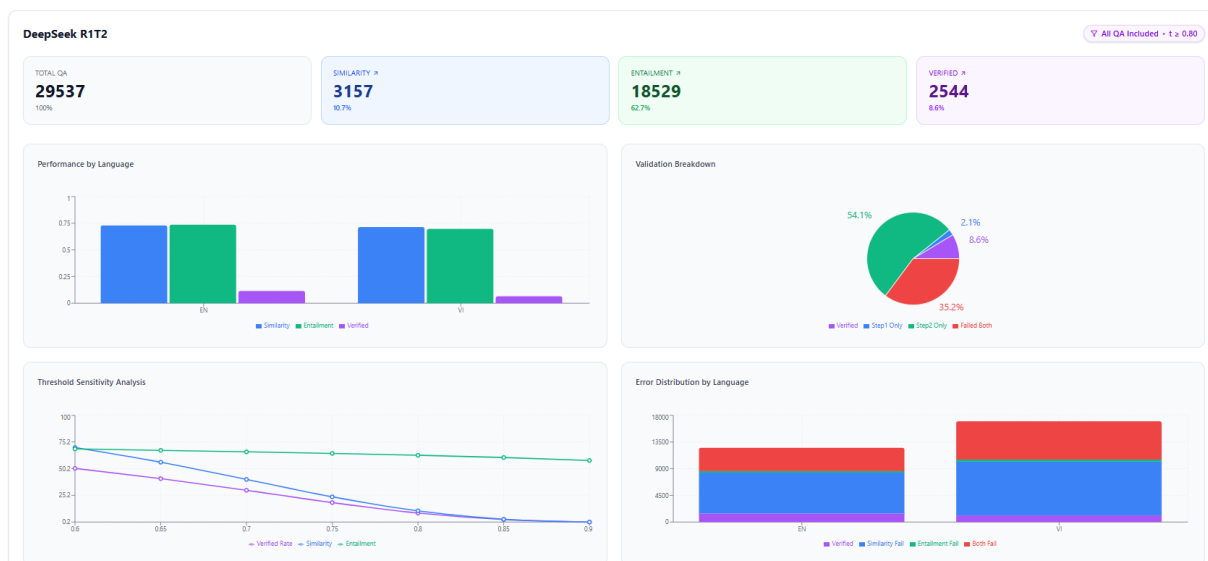
Phân tích:

DeepSeek sinh nhiều QA hơn Gemini và GPT nhưng tỉ lệ vượt qua hai tầng kiểm định thấp hơn. Biểu đồ Validation Breakdown cho thấy:

- Tỉ lệ fail similarity cao hơn.
- Tỉ lệ fail cả hai tầng lớn hơn Gemini và GPT.

Điều này gợi ý rằng DeepSeek có xu hướng sinh QA đa dạng hơn nhưng độ bám sát context không chặt bằng Gemini và GPT.

3.4.8 Phân tích DeepSeek R1T2 – ngưỡng 0.80



Hình 3.9: Phân tích DeepSeek R1T2 với $\tau = 0.80$

QA được xác nhận còn lại: 2,544 (8.6%)

Khi tăng ngưỡng:

- Tỷ lệ verified giảm mạnh.
- Lượng QA fail cả hai tầng tăng đáng kể.

Điều này cho thấy phân bố similarity của DeepSeek tập trung nhiều ở khoảng 0.65–0.80, ít QA đạt mức tương đồng rất cao.

3.4.9 So sánh tổng thể giữa GPT-5.2, Gemini và DeepSeek

Từ các kết quả thực nghiệm, có thể quan sát rõ sự khác biệt về đặc tính sinh dữ liệu của ba mô hình ngôn ngữ lớn được sử dụng trong đề tài.

- **Gemini 2.5 Flash** đạt tỷ lệ QA vượt qua kiểm định cao nhất ở cả hai mức ngưỡng. Điều này cho thấy khả năng bám sát ngữ cảnh và duy trì mối quan hệ entailment giữa context và answer rất tốt. Việc sử dụng API chính thức với prompt được thiết kế cố định giúp đảm bảo tính nhất quán trong quá trình sinh dữ liệu, từ đó giảm thiểu sai lệch giữa các lần sinh.

- **GPT-5.2** thể hiện hiệu năng trung gian giữa Gemini và DeepSeek. Mặc dù chất lượng QA tương đối ổn định, tỉ lệ vượt qua kiểm định vẫn thấp hơn so với Gemini. Một nguyên nhân quan trọng là quá trình sinh QA được thực hiện thông qua giao diện ChatGPT thay vì API, dẫn đến một số hạn chế:
 - Prompt có thể được điều chỉnh linh hoạt trong quá trình sử dụng, gây ra sự không đồng nhất giữa các lần sinh.
 - Hiện tượng *prompt drift* làm thay đổi cách diễn đạt và cấu trúc QA.
 - Khó kiểm soát chặt chẽ định dạng đầu ra so với quy trình sinh tự động qua API.

Những yếu tố này giúp tăng tính đa dạng của dữ liệu, nhưng đồng thời cũng làm tăng nhiễu và giảm tỉ lệ QA đạt chuẩn.

- **DeepSeek R1T2 Chimera** sinh ra số lượng QA lớn nhất, tuy nhiên tỉ lệ vượt qua kiểm định thấp nhất. Điều này có thể được giải thích bởi:
 - Mô hình được triển khai nội bộ (local) và chưa được fine-tune chuyên biệt cho tác vụ sinh QA.
 - Xu hướng sinh đa dạng nhưng ít ràng buộc chặt với ngữ cảnh đầu vào.
 - Khả năng kiểm soát hallucination và đảm bảo entailment thấp hơn so với các mô hình sử dụng API.

Nhìn chung, ba mô hình thể hiện ba đặc tính bổ sung lẫn nhau:

- **Gemini:** ưu tiên độ chính xác cao (*high precision*)
- **GPT-5.2:** cân bằng giữa độ đa dạng và độ chính xác (*balanced trade-off*)
- **DeepSeek:** sinh dữ liệu đa dạng với số lượng lớn (*high diversity*) nhưng độ chính xác thấp hơn

3.4.10 So sánh định lượng giữa các mô hình

Bảng 3.1: So sánh hiệu năng sinh QA giữa các mô hình ở hai ngưỡng $\tau = 0.70$ và $\tau = 0.80$

Mô hình	$\tau = 0.70$		$\tau = 0.80$		Đặc điểm
	Verified	Tỉ lệ (%)	Verified	Tỉ lệ (%)	
Gemini 2.5 Flash	8,790	52.3%	4,451	26.5%	Độ chính xác cao, ổn định
GPT-5.2	8,511	34.7%	3,379	13.8%	Cân bằng giữa đa dạng và độ chính xác
DeepSeek R1T2	8,786	29.7%	2,544	8.6%	Sinh nhiều nhưng nhiễu cao

Bảng 3.1 cho thấy rõ sự đánh đổi giữa số lượng và chất lượng dữ liệu sinh khi thay đổi ngưỡng kiểm định.

- Khi tăng ngưỡng từ 0.70 lên 0.80, số lượng QA được giữ lại giảm đáng kể ở tất cả các mô hình.
- Gemini vẫn duy trì tỉ lệ giữ lại cao nhất, cho thấy khả năng sinh QA có độ liên kết ngữ nghĩa mạnh và ổn định.
- GPT-5.2 thể hiện xu hướng trung gian, phản ánh sự cân bằng giữa tính đa dạng và độ chính xác.
- DeepSeek bị ảnh hưởng mạnh nhất bởi việc tăng ngưỡng, cho thấy nhiều QA nằm ở vùng similarity trung bình.

3.4.11 Đánh giá bổ sung sau khi áp dụng NLI

Để đánh giá hiệu quả của việc bổ sung tầng kiểm định suy luận (NLI), đề tài tiến hành kiểm tra thủ công trên một tập mẫu gồm 600 QA, được lấy ngẫu nhiên từ 6 tập dữ liệu (DeepSeek, Gemini, GPT với hai ngôn ngữ tiếng Anh và tiếng Việt, mỗi tập 100 QA).

Quy trình đánh giá được thực hiện theo ba bước nhằm đảm bảo tính nhất quán trong việc kiểm định:

- Trước hết, đọc **context** để xác định nội dung chính và phạm vi thông tin được cung cấp.
- Tiếp theo, kiểm tra **question** nhằm đánh giá xem câu hỏi có thực sự bám sát nội dung của context hay không.
- Cuối cùng, đánh giá **answer** dựa trên ba tiêu chí: tính đúng đắn về mặt thông tin (fact), không suy diễn hoặc bổ sung ngoài context, và khả năng trả lời chính xác câu hỏi đã đặt ra.

Kết quả kiểm tra trước và sau khi áp dụng NLI được trình bày trong Bảng 3.2.

Bảng 3.2: So sánh kết quả kiểm định QA trước và sau khi áp dụng NLI

Model	Language	Before NLI	After NLI	Improvement
DeepSeek	EN	94/100 (94%)	96/100 (96%)	+2%
DeepSeek	VI	94/100 (94%)	95/100 (95%)	+1%
Gemini	EN	83/100 (83%)	95/100 (95%)	+12%
Gemini	VI	84/100 (84%)	93/100 (93%)	+9%
GPT	EN	96/100 (96%)	98/100 (98%)	+2%
GPT	VI	90/100 (90%)	95/100 (95%)	+5%
Total		541/600 (90.17%)	572/600 (95.33%)	+5.16%

Nhận xét:

Kết quả trong Bảng 3.2 cho thấy việc bổ sung tầng kiểm định NLI giúp cải thiện đáng kể độ chính xác của bộ dữ liệu QA, với mức tăng tổng thể từ 90.17% lên 95.33%, tương ứng cải thiện hơn 5%. Điều này chứng minh rằng NLI đóng vai trò hiệu quả trong việc loại bỏ các QA có lỗi suy luận hoặc mở rộng ngoài phạm vi ngữ cảnh.

Phân tích chi tiết cho thấy mức độ cải thiện không đồng đều giữa các mô hình. Cụ thể, Gemini là mô hình hưởng lợi nhiều nhất, với mức tăng từ 83–84% lên 93–95%. Điều này cho thấy Gemini có xu hướng sinh câu trả lời đầy đủ nhưng đôi khi vượt quá nội dung được cung cấp, và NLI giúp loại bỏ hiệu quả các trường hợp này.

Trong khi đó, GPT và DeepSeek vốn đã có độ bám sát context cao hơn, nên mức cải thiện thấp hơn nhưng vẫn đáng kể. Đặc biệt, GPT đạt mức

chính xác gần 98% ở tiếng Anh sau khi áp dụng NLI, cho thấy khả năng kiểm soát nội dung tốt ngay từ bước sinh.

Nhìn chung, DeepSeek thể hiện tính ổn định cao giữa hai ngôn ngữ, trong khi GPT đạt hiệu năng cao nhất, còn Gemini là mô hình cải thiện mạnh nhất sau khi bổ sung tầng kiểm định suy luận.

Phân tích lỗi còn lại sau NLI:

Mặc dù NLI giúp loại bỏ phần lớn các lỗi rõ ràng, vẫn tồn tại một số lỗi tinh vi trong dữ liệu:

- Câu trả lời bổ sung một chi tiết nhỏ không xuất hiện trực tiếp trong context.
- Câu trả lời thiếu một phần thông tin trong các câu hỏi nhiều vế.
- Sự suy rộng ý nghĩa vượt nhẹ khỏi phạm vi nội dung của đoạn văn.

Điều này cho thấy NLI là một bước kiểm định rất hiệu quả đối với các lỗi rõ ràng, nhưng trong một số trường hợp phức tạp, việc kiểm tra thủ công vẫn cần thiết để đảm bảo chất lượng tuyệt đối.

Kết luận:

Pipeline kiểm định hai tầng (bi-encoder + cross-encoder) đóng vai trò nền tảng trong việc chuẩn hóa dữ liệu sinh từ nhiều nguồn mô hình ngôn ngữ khác nhau, giúp loại bỏ phần lớn các QA có liên kết ngữ nghĩa yếu và cải thiện đáng kể độ chính xác tổng thể.

Tuy nhiên, kết quả thực nghiệm cho thấy pipeline này vẫn chưa đảm bảo hoàn toàn tính đúng đắn về mặt suy luận. Việc bổ sung tầng kiểm định NLI giúp khắc phục hạn chế này bằng cách loại bỏ các câu trả lời mang tính suy diễn hoặc không được hỗ trợ trực tiếp từ context, từ đó nâng độ chính xác của dữ liệu từ 90.17% lên 95.33%.

Do đó, pipeline ba tầng (bi-encoder, cross-encoder và NLI) cho phép kiểm định dữ liệu theo ba mức độ:

$$Relevance \rightarrow Precision \rightarrow Correctness$$

Nhờ đó, bộ dữ liệu QA cuối cùng không chỉ đảm bảo tính liên quan và độ chính xác, mà còn có khả năng kiểm chứng cao, phù hợp cho các bước xây dựng semantic index và đánh giá hệ thống truy hồi xuyên ngữ.

Chương 4

Phương pháp nghiên cứu

4.1 Tổng quan hệ thống tìm kiếm ngữ nghĩa

Hệ thống tìm kiếm trong đề tài được xây dựng nhằm giải quyết bài toán truy hồi văn bản theo ngữ nghĩa và hỗ trợ tìm kiếm xuyên ngữ giữa tiếng Việt và tiếng Anh. Khác với các phương pháp dựa trên đối sánh từ khóa truyền thống, hệ thống khai thác biểu diễn vector liên tục (dense embedding) để ánh xạ văn bản vào không gian ngữ nghĩa chung, từ đó đo lường mức độ tương đồng bằng các phép toán hình học trong không gian vector.

Kiến trúc hệ thống được thiết kế theo hướng mô-đun, bao gồm hai thành phần xử lý chính:

- Giai đoạn xây dựng chỉ mục (offline).
- Giai đoạn xử lý truy vấn (online).

Hai giai đoạn này tách biệt rõ ràng về chức năng và tài nguyên sử dụng, giúp hệ thống tối ưu hiệu năng khi vận hành thực tế.

4.1.1 Kiến trúc tổng thể

Về mặt tổng quát, hệ thống bao gồm các thành phần:

- Mô hình biểu diễn ngữ nghĩa (embedding models).
- Chỉ mục vector FAISS.
- Cơ chế truy hồi và xếp hạng lại.
- Tầng giao tiếp (API) phục vụ truy vấn.

Toàn bộ pipeline tìm kiếm có thể được mô tả như sau:

1. Văn bản tài liệu được chuyển thành các đoạn (chunk).
2. Mỗi đoạn được mã hóa thành vector embedding.
3. Các vector được lưu trữ trong chỉ mục FAISS.
4. Khi nhận truy vấn, hệ thống mã hóa truy vấn thành embedding.
5. Thực hiện truy hồi các ứng viên gần nhất.
6. Xếp hạng lại và trả về Top- K kết quả.

Thiết kế này cho phép hệ thống hoạt động hiệu quả ngay cả khi số lượng tài liệu lớn, đồng thời duy trì tính linh hoạt trong lựa chọn chiến lược xếp hạng.

4.1.2 Hai cấu hình tìm kiếm song song

Hệ thống triển khai hai cấu hình tìm kiếm độc lập nhằm đáp ứng hai mục tiêu vận hành khác nhau:

- **Cấu hình nhẹ (MiniLM + Hybrid Heuristic):** ưu tiên khả năng phục vụ đồng thời nhiều truy vấn với chi phí tính toán thấp.
- **Cấu hình chính xác cao (BGE-M3 + Cross-Encoder Reranker):** ưu tiên chất lượng xếp hạng và độ chính xác ngữ nghĩa.

Hai cấu hình này sử dụng hai không gian embedding khác nhau và duy trì hai chỉ mục FAISS riêng biệt. Khi người dùng lựa chọn cấu hình tìm kiếm, toàn bộ quá trình từ mã hóa truy vấn đến xếp hạng lại sẽ diễn ra trong không gian embedding tương ứng.

Việc thiết kế hai backend song song giúp hệ thống linh hoạt thích ứng với nhiều kịch bản sử dụng:

- Môi trường tải cao, yêu cầu độ trễ thấp.
- Môi trường nghiên cứu, yêu cầu độ chính xác truy hồi cao.

4.1.3 Nguyên lý truy hồi dựa trên embedding

Giả sử mỗi đoạn văn bản được ánh xạ thành vector $v_i \in R^d$ và truy vấn được ánh xạ thành vector $q \in R^d$. Mức độ tương đồng giữa truy vấn và đoạn văn được xác định bằng:

$$\text{sim}(q, v_i) = q \cdot v_i$$

Trong đó các vector đã được chuẩn hóa L2 để phép nhân vô hướng tương đương với cosine similarity.

Bài toán truy hồi khi đó được chuyển thành bài toán tìm kiếm lân cận gần nhất trong không gian vector. FAISS được sử dụng để thực hiện thao tác này một cách hiệu quả.

4.1.4 Phân tách trách nhiệm giữa các thành phần

Thiết kế hệ thống tuân theo nguyên tắc phân tách trách nhiệm:

- Mô hình embedding chỉ đảm nhiệm biểu diễn ngữ nghĩa.
- FAISS chịu trách nhiệm truy hồi gần đúng.
- Thành phần reranking chịu trách nhiệm tinh chỉnh thứ hạng.
- Tầng API điều phối toàn bộ quy trình và giao tiếp với người dùng.

Cách tổ chức này giúp việc thay thế mô hình embedding, thay đổi chiến lược xếp hạng hoặc mở rộng chỉ mục có thể thực hiện độc lập mà không ảnh hưởng đến cấu trúc tổng thể của hệ thống.

4.1.5 Định hướng thiết kế

Toàn bộ hệ thống được xây dựng theo ba nguyên tắc chính:

- Tính nhất quán hình học giữa truy vấn và chỉ mục.
- Khả năng mở rộng theo số lượng tài liệu và số lượng người dùng.
- Linh hoạt trong lựa chọn chiến lược cân bằng giữa hiệu năng và độ chính xác.

Nhờ các nguyên tắc này, hệ thống không chỉ phù hợp cho mục đích thực nghiệm trong luận văn mà còn có khả năng triển khai thực tế trong các môi trường có yêu cầu khác nhau về tài nguyên và chất lượng truy hồi.

4.2 Mô hình biểu diễn ngữ nghĩa

Trọng tâm của hệ thống tìm kiếm ngữ nghĩa là khả năng chuyển đổi văn bản tự nhiên thành biểu diễn vector liên tục sao cho các đoạn văn có nội dung tương đồng sẽ có vị trí gần nhau trong không gian hình học. Thành phần này được gọi là mô hình biểu diễn ngữ nghĩa (semantic embedding model).

Hệ thống triển khai hai mô hình embedding độc lập nhằm phục vụ hai cấu hình tìm kiếm khác nhau:

- Mô hình đa ngữ MiniLM.
- Mô hình BGE-M3.

Hai mô hình này được sử dụng để xây dựng hai không gian embedding riêng biệt và hai chỉ mục FAISS tương ứng.

4.2.1 Yêu cầu đối với mô hình embedding

Đối với bài toán tìm kiếm xuyên ngữ trên dữ liệu học thuật, mô hình embedding cần đáp ứng các yêu cầu sau:

- Hỗ trợ đa ngôn ngữ (tiếng Việt và tiếng Anh).
- Khả năng ánh xạ các câu tương đương về cùng vùng trong không gian vector.
- Ổn định với câu dài và văn bản có mật độ thông tin cao.
- Tốc độ suy luận phù hợp cho môi trường truy vấn thời gian thực.

Việc lựa chọn mô hình embedding là yếu tố quyết định đến chất lượng truy hồi và hành vi của toàn hệ thống.

4.2.2 Mô hình MiniLM đa ngữ

Mô hình đầu tiên được sử dụng là `sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2`. Đây là mô hình dựa trên kiến trúc Transformer nhẹ, được huấn luyện theo hướng tối ưu cho bài toán so sánh câu.

Đặc điểm chính:

- Kích thước embedding: 384 chiều.
- Tối ưu cho cosine similarity.
- Hỗ trợ nhiều ngôn ngữ trong cùng không gian biểu diễn.
- Tốc độ suy luận nhanh trên CPU.

MiniLM phù hợp trong môi trường cần xử lý nhiều truy vấn đồng thời, do chi phí tính toán thấp và bộ nhớ tiêu thụ nhỏ hơn so với các mô hình lớn.

Tuy nhiên, do kích thước mô hình và chiều embedding tương đối nhỏ, khả năng phân biệt các đoạn văn có nội dung gần nhau nhưng khác trọng tâm có thể hạn chế trong một số trường hợp phức tạp.

4.2.3 Mô hình BGE-M3

Mô hình thứ hai được sử dụng là `BAAI/bge-m3`. Đây là mô hình embedding được thiết kế chuyên biệt cho bài toán truy hồi ngữ nghĩa và được huấn luyện trên tập dữ liệu quy mô lớn với mục tiêu tối ưu chất lượng tìm kiếm.

Đặc điểm chính:

- Khả năng biểu diễn ngữ nghĩa mạnh hơn trên văn bản dài.
- Tối ưu cho tác vụ retrieval.
- Hỗ trợ đa ngữ và khả năng căn chỉnh tốt giữa các ngôn ngữ.

So với MiniLM, BGE-M3 thường cho độ phân giải ngữ nghĩa cao hơn, đặc biệt trên dữ liệu học thuật chứa nhiều thuật ngữ chuyên ngành. Tuy nhiên, chi phí suy luận lớn hơn và yêu cầu tài nguyên cao hơn.

4.2.4 Chuẩn hóa vector và metric tương đồng

Trong cả hai cấu hình, embedding được chuẩn hóa theo chuẩn L2:

$$\hat{v} = \frac{v}{\|v\|}$$

Khi đó, độ tương đồng giữa hai vector được tính bằng tích vô hướng:

$$\text{sim}(q, v) = \hat{q} \cdot \hat{v}$$

Điều này cho phép sử dụng chỉ mục FAISS với metric Inner Product trong khi vẫn giữ ý nghĩa của cosine similarity.

4.2.5 So sánh hai mô hình

Sự khác biệt giữa hai mô hình có thể tóm lược như sau:

- MiniLM: nhẹ, nhanh, phù hợp tải lớn.
- BGE-M3: biểu diễn mạnh hơn, phù hợp khi ưu tiên chất lượng truy hồi.

Về mặt hình học, hai mô hình tạo ra hai không gian embedding khác nhau. Do đó, truy vấn phải được mã hóa bằng đúng mô hình đã sử dụng để xây dựng chỉ mục tương ứng.

4.2.6 Vai trò của embedding trong toàn hệ thống

Embedding đóng vai trò là lớp trừu tượng hóa ngữ nghĩa, chuyển đổi văn bản tự nhiên thành cấu trúc toán học có thể xử lý bằng thuật toán tìm kiếm. Chất lượng embedding quyết định:

- Mức độ chính xác của truy hồi sơ cấp.
- Số lượng ứng viên cần rerank.
- Độ ổn định trong tìm kiếm xuyên ngữ.

Do đó, việc triển khai song song hai mô hình embedding giúp hệ thống linh hoạt cân bằng giữa hiệu năng và chất lượng truy hồi trong các kịch bản sử dụng khác nhau.

4.3 Xây dựng chỉ mục FAISS

Sau khi mỗi đoạn văn bản được biểu diễn dưới dạng vector embedding, bước tiếp theo là tổ chức tập vector này thành một cấu trúc cho phép truy hồi hiệu quả theo độ tương đồng. Hệ thống sử dụng thư viện FAISS (Facebook AI Similarity Search) để xây dựng chỉ mục vector phục vụ tìm kiếm lân cận gần nhất trong không gian nhiều chiều.

4.3.1 Bài toán truy hồi trong không gian vector

Giả sử tập tài liệu sau khi chunking tạo ra n đoạn văn, mỗi đoạn được ánh xạ thành vector $v_i \in R^d$. Khi người dùng gửi truy vấn, hệ thống cần tìm tập con các vector có độ tương đồng lớn nhất với vector truy vấn q .

Bài toán truy hồi được mô hình hóa như sau:

$$\text{Top-}N(q) = \arg \max_{v_i \in E} (q \cdot v_i)$$

Trong đó E là tập embedding của toàn bộ đoạn văn và các vector đã được chuẩn hóa L2.

Nếu thực hiện tìm kiếm tuần tự (brute-force), độ phức tạp là $O(n \cdot d)$ cho mỗi truy vấn. Khi n lớn, cách tiếp cận này trở nên không khả thi trong môi trường thời gian thực. FAISS được sử dụng để tối ưu quá trình tìm kiếm này.

4.3.2 Lựa chọn cấu trúc chỉ mục

Trong hệ thống hiện tại, chỉ mục được xây dựng theo dạng *Flat Index* với metric Inner Product. Lý do lựa chọn cấu trúc này bao gồm:

- Tập dữ liệu ở quy mô trung bình, chưa yêu cầu phân cụm hoặc lượng tử hóa.
- Cần đảm bảo tính chính xác cao nhất cho truy hồi sơ cấp.
- Đơn giản hóa quá trình xây dựng và tái nạp chỉ mục.

Vì embedding đã được chuẩn hóa L2, việc sử dụng Inner Product tương đương với cosine similarity. Điều này cho phép FAISS thực hiện tìm kiếm chính xác theo độ tương đồng ngữ nghĩa.

4.3.3 Hai không gian embedding và hai chỉ mục độc lập

Hệ thống triển khai hai mô hình embedding (MiniLM và BGE-M3), do đó tồn tại hai tập vector khác nhau:

$$E_{MiniLM} \in R^{n \times d_1}$$

$$E_{BGE} \in R^{n \times d_2}$$

Trong đó d_1 và d_2 có thể khác nhau tùy theo cấu hình mô hình.

Vì hai không gian embedding là độc lập về mặt hình học, hệ thống xây dựng hai chỉ mục FAISS riêng biệt. Khi người dùng chọn backend, hệ thống sẽ truy cập đúng chỉ mục tương ứng. Điều này đảm bảo tính nhất quán giữa biểu diễn truy vấn và tập vector được tìm kiếm.

4.3.4 Quy trình xây dựng chỉ mục

Quy trình xây dựng chỉ mục bao gồm các bước:

1. Sinh embedding cho toàn bộ chunk của tập tài liệu.
2. Chuẩn hóa vector theo chuẩn L2.
3. Khởi tạo cấu trúc FAISS với kích thước d và metric Inner Product.
4. Nạp toàn bộ vector vào chỉ mục.
5. Lưu chỉ mục xuống đĩa để tái sử dụng.

Bên cạnh đó, hệ thống lưu metadata tương ứng với mỗi vector, bao gồm:

- Nội dung chunk.
- Tên tài liệu nguồn.

- Vị trí câu bắt đầu và kết thúc.
- Danh sách câu của tài liệu.

FAISS chỉ lưu trữ vector số học; việc ánh xạ từ chỉ số vector sang nội dung văn bản được thực hiện thông qua cấu trúc metadata lưu trữ song song.

4.3.5 Tổ chức lưu trữ và tái nạp chỉ mục

Sau khi xây dựng, chỉ mục FAISS và metadata được lưu xuống đĩa dưới dạng các tệp riêng biệt. Khi hệ thống khởi động, chỉ mục được nạp lại vào bộ nhớ thay vì tái xây dựng từ đầu.

Cơ chế này mang lại hai lợi ích:

- Giảm đáng kể thời gian khởi động hệ thống.
- Cho phép cập nhật dữ liệu theo lô và tái tạo chỉ mục khi cần.

Hệ thống cũng hỗ trợ cơ chế tái nạp chỉ mục khi có thay đổi trong tập tài liệu, giúp đảm bảo tính cập nhật mà không ảnh hưởng đến kiến trúc tổng thể.

4.3.6 Khả năng mở rộng

Mặc dù hiện tại hệ thống sử dụng chỉ mục dạng Flat để đảm bảo độ chính xác tối đa, FAISS cung cấp nhiều cấu trúc chỉ mục gần đúng như IVF (Inverted File Index) hoặc HNSW. Khi quy mô dữ liệu tăng lên hàng triệu đoạn văn, hệ thống có thể chuyển sang các cấu trúc này để giảm độ trễ truy vấn.

Thiết kế hiện tại giữ nguyên tính linh hoạt để mở rộng trong tương lai mà không cần thay đổi logic truy hồi ở tầng trên.

4.3.7 Vai trò của FAISS trong toàn hệ thống

FAISS đóng vai trò là thành phần trung gian giữa embedding và reranking. Chất lượng truy hồi sơ cấp phụ thuộc vào:

- Độ phân giải của embedding.
- Cấu trúc chỉ mục.
- Số lượng ứng viên Top- N được chọn.

Một chỉ mục được xây dựng chính xác giúp giảm số lượng ứng viên cần rerank, từ đó tối ưu hiệu năng tổng thể của hệ thống.

Nhờ vậy, bước xây dựng chỉ mục không chỉ là thao tác lưu trữ vector mà là nền tảng quyết định hành vi truy hồi của toàn bộ hệ thống tìm kiếm ngữ nghĩa.

4.4 Cơ chế truy hồi và xếp hạng

Sau khi chỉ mục FAISS được xây dựng, hệ thống thực hiện truy hồi theo hai tầng: (i) truy hồi sơ cấp dựa trên khoảng cách trong không gian embedding và (ii) xếp hạng lại nhằm tinh chỉnh thứ tự kết quả.

Thiết kế hai tầng này cho phép cân bằng giữa hiệu năng và độ chính xác, đặc biệt trong bối cảnh dữ liệu học thuật có mật độ thông tin cao.

4.4.1 Truy hồi sơ cấp (Initial Retrieval)

Với truy vấn $q \in R^d$, hệ thống thực hiện tìm kiếm Top- N vector gần nhất trong chỉ mục FAISS:

$$C_N = \arg \max_{v_i \in E} (q \cdot v_i)$$

Trong đó:

- E là tập embedding của toàn bộ chunk.
- N là kích thước tập ứng viên ban đầu.

Giá trị N được chọn lớn hơn K (số kết quả cuối cùng trả về) để tạo dư địa cho bước xếp hạng lại. Nếu chọn $N = K$, hệ thống sẽ phụ thuộc hoàn toàn vào thứ tự do FAISS trả về, có thể làm giảm độ chính xác tổng thể.

4.4.2 Chuyển từ Top- N sang Top- K

Sau khi có tập ứng viên C_N , hệ thống thực hiện xếp hạng lại và chọn ra Top- K phần tử có điểm cao nhất:

$$R_K = \text{Top-}K(\text{Score}(q, p_i)), \quad p_i \in C_N$$

Trong đó $\text{Score}(q, p_i)$ phụ thuộc vào backend được lựa chọn.

Thiết kế này cho phép hệ thống:

- Tăng chất lượng kết quả cuối cùng.
- Giảm ảnh hưởng của sai lệch hình học trong không gian embedding.

4.4.3 Cơ chế xếp hạng trong cấu hình MiniLM

Trong backend MiniLM, hệ thống áp dụng chiến lược hybrid heuristic. Điểm số cuối cùng được tính như sau:

$$\text{Score}_{\text{hybrid}} = \alpha S_{\text{semantic}} + \beta S_{\text{lexical}} + \gamma S_{\text{fact}}$$

Trong đó:

- S_{semantic} : cosine similarity giữa embedding truy vấn và đoạn văn.
- S_{lexical} : độ tương đồng token (ví dụ Jaccard).
- S_{fact} : mức độ khớp các dữ kiện số như năm, ngày tháng, giá trị định lượng.

Cách tiếp cận này khai thác đồng thời tín hiệu ngữ nghĩa và tín hiệu bề mặt. Điều này đặc biệt hữu ích khi truy vấn chứa nhiều dữ kiện cụ thể, ví dụ năm nghiên cứu hoặc giá trị thống kê.

Độ phức tạp của bước rerank trong cấu hình này là:

$$O(N)$$

vì chỉ bao gồm các phép toán vector và so khớp token.

4.4.4 Cơ chế xếp hạng trong cấu hình BGE

Trong backend BGE, hệ thống sử dụng cross-encoder BAAI/bge-reranker-v2-m3 để tính điểm liên quan trực tiếp cho từng cặp (q, p_i) .

Điểm số được xác định bởi:

$$Score_{CE}(q, p_i) = CE(q, p_i)$$

Trong đó CE là mô hình transformer xử lý đồng thời hai chuỗi văn bản.

Khác với embedding-based similarity, cross-encoder có khả năng mô hình hóa tương tác token-level giữa truy vấn và đoạn văn. Điều này giúp:

- Phân biệt các đoạn có embedding gần nhau nhưng khác trọng tâm.
- Giảm ảnh hưởng của hiện tượng hubness.
- Cải thiện thứ hạng cho truy vấn dài hoặc phức tạp.

Tuy nhiên, độ phức tạp của bước này xấp xỉ:

$$O(N \cdot T)$$

trong đó T là chi phí suy luận transformer cho mỗi cặp.

4.4.5 So sánh hai chiến lược xếp hạng

Hai cấu hình có hành vi khác nhau về hiệu năng và tài nguyên:

- Hybrid rerank:
 - Nhanh và ổn định.
 - Phù hợp môi trường nhiều truy vấn đồng thời.
 - Phụ thuộc mạnh vào chất lượng embedding sơ cấp.
- Cross-encoder rerank:
 - Độ chính xác cao hơn.
 - Phân biệt tốt các đoạn gần ngữ nghĩa.
 - Tốn tài nguyên tính toán, đặc biệt khi N lớn.

4.4.6 Ảnh hưởng của kích thước Top- N

Giá trị N ảnh hưởng trực tiếp đến:

- Độ bao phủ của tập ứng viên.
- Thời gian rerank.
- Mức tiêu thụ tài nguyên GPU (với cross-encoder).

Nếu N quá nhỏ, hệ thống có thể bỏ sót đoạn văn phù hợp. Nếu N quá lớn, độ trễ tăng đáng kể, đặc biệt với cấu hình BGE.

Do đó, lựa chọn N là một tham số điều chỉnh quan trọng trong thực nghiệm.

4.4.7 Vai trò của cơ chế hai tầng

Cơ chế truy hồi hai tầng (retrieval + reranking) giúp hệ thống:

- Tận dụng tốc độ của FAISS.
- Khai thác độ chính xác của heuristic hoặc cross-encoder.
- Linh hoạt điều chỉnh giữa hiệu năng và chất lượng.

Thiết kế này là nền tảng cốt lõi giúp hệ thống đạt được hiệu quả trong cả môi trường tải cao lẫn môi trường yêu cầu độ chính xác cao.

4.5 Tìm kiếm xuyên ngữ

Tìm kiếm xuyên ngữ (cross-lingual retrieval) là một trong những yêu cầu quan trọng của hệ thống, cho phép người dùng sử dụng truy vấn bằng một ngôn ngữ để tìm kiếm tài liệu thuộc ngôn ngữ khác. Trong phạm vi đề tài, hệ thống hỗ trợ tìm kiếm giữa tiếng Việt và tiếng Anh.

Khác với tìm kiếm đơn ngữ, bài toán xuyên ngữ đòi hỏi cơ chế căn chỉnh ngữ nghĩa giữa hai hệ thống ngôn ngữ khác nhau. Hệ thống giải quyết bài toán này thông qua hai chiến lược: (i) khai thác không gian embedding đa ngữ chung và (ii) chuẩn hóa truy vấn thông qua dịch máy khi cần thiết.

4.5.1 Không gian embedding đa ngữ chung

Các mô hình embedding được sử dụng trong hệ thống (MiniLM đa ngữ và BGE-M3) đều được huấn luyện trên tập dữ liệu đa ngôn ngữ. Điều này cho phép ánh xạ câu thuộc nhiều ngôn ngữ vào cùng một không gian vector R^d .

Giả sử tồn tại hàm ánh xạ:

$$f : X \rightarrow R^d$$

với X là tập câu đa ngữ. Khi hai câu x_{vi} và x_{en} có nội dung tương đương, ta kỳ vọng:

$$\cos(f(x_{vi}), f(x_{en})) \approx 1$$

Nhờ đặc tính này, hệ thống có thể trực tiếp:

- Mã hóa truy vấn tiếng Việt.
- Truy hồi tài liệu tiếng Anh.
- Và ngược lại.

Cách tiếp cận này không yêu cầu dịch toàn bộ tài liệu, giúp giảm chi phí xử lý và duy trì cấu trúc ngữ nghĩa gốc của văn bản.

4.5.2 Vấn đề alignment giữa các ngôn ngữ

Mặc dù embedding đa ngữ cho phép ánh xạ chung, mức độ căn chỉnh (alignment) giữa hai ngôn ngữ không hoàn toàn hoàn hảo. Sai lệch có thể xuất phát từ:

- Phân phối dữ liệu huấn luyện không cân bằng giữa các ngôn ngữ.
- Sự khác biệt về cấu trúc cú pháp và hình thái học.
- Sự xuất hiện của thuật ngữ chuyên ngành trong miền học thuật.

Trong dữ liệu học thuật, các thuật ngữ chuyên sâu có thể không xuất hiện phổ biến trong tập huấn luyện đa ngữ, làm giảm chất lượng căn chỉnh.

4.5.3 Hiện tượng hubness trong không gian chiều cao

Trong không gian vector nhiều chiều, hiện tượng hubness có thể xảy ra, trong đó một số vector xuất hiện thường xuyên trong danh sách lân cận gần nhất của nhiều truy vấn khác nhau.

Trong bối cảnh xuyên ngữ, hubness có thể do:

- Một số đoạn văn chứa thuật ngữ phổ quát.
- Phân phối embedding giữa hai ngôn ngữ không đồng đều.

Hệ thống giảm ảnh hưởng của hubness thông qua:

- Chuẩn hóa L2 embedding.
- Bước xếp hạng lại bằng hybrid hoặc cross-encoder.

4.5.4 Chuẩn hóa truy vấn bằng dịch máy

Bên cạnh embedding đa ngữ, hệ thống có thể áp dụng thêm bước chuẩn hóa truy vấn bằng mô hình dịch máy `VietAI/envit5-translation`.

Giả sử truy vấn q_{vi} cần tìm trong tập tài liệu tiếng Anh, hệ thống thực hiện:

$$q_{en} = T(q_{vi})$$

trong đó T là hàm dịch máy.

Chiến lược này giúp:

- Tăng độ khớp từ vựng trong cấu hình hybrid.
- Giảm sai lệch alignment trong một số trường hợp đặc biệt.

Tuy nhiên, việc dịch có thể:

- Gây mất mát ngữ nghĩa nếu bản dịch không chính xác.
- Tăng độ trễ xử lý.

4.5.5 Vai trò của cross-encoder trong truy vấn xuyên ngữ

Trong cấu hình BGE, cross-encoder reranker đóng vai trò quan trọng trong việc cải thiện độ chính xác cho truy vấn xuyên ngữ.

Khác với embedding-based similarity, cross-encoder xử lý trực tiếp cặp (q, p) :

$$Score(q, p) = CE(q, p)$$

Cơ chế này cho phép mô hình:

- Mô hình hóa tương tác token-level giữa hai chuỗi.
- Bù đắp sai lệch alignment trong không gian embedding.
- Phân biệt các đoạn văn có nội dung gần nhau nhưng khác trọng tâm.

Do đó, cấu hình BGE thường ổn định hơn trong các truy vấn học thuật dài hoặc chứa nhiều thuật ngữ chuyên ngành.

4.5.6 So sánh hai chiến lược xuyên ngữ

Hai chiến lược có đặc điểm khác nhau:

- Embedding đa ngữ trực tiếp:
 - Nhanh và gọn nhẹ.
 - Không phát sinh chi phí dịch.
 - Phụ thuộc mạnh vào chất lượng alignment.
- Chuẩn hóa bằng dịch máy:
 - Cải thiện độ ổn định trong một số truy vấn đặc biệt.
 - Tăng chi phí xử lý.
 - Phụ thuộc vào chất lượng bản dịch.

Thiết kế hệ thống cho phép linh hoạt kết hợp các chiến lược này tùy theo cấu hình backend và yêu cầu vận hành.

4.5.7 Nhận xét

Cơ chế tìm kiếm xuyên ngữ của hệ thống không dựa vào một kỹ thuật đơn lẻ mà là sự kết hợp giữa:

- Không gian embedding đa ngữ.
- Cơ chế truy hồi hai tầng.
- Xếp hạng lại bằng heuristic hoặc cross-encoder.
- Chuẩn hóa truy vấn bằng dịch máy khi cần.

Thiết kế này giúp hệ thống duy trì sự cân bằng giữa hiệu năng và độ chính xác trong bài toán truy hồi học thuật xuyên ngữ.

4.6 Triển khai hệ thống và tích hợp thực tế

Sau khi hoàn thiện các thành phần biểu diễn ngữ nghĩa, xây dựng chỉ mục và cơ chế truy hồi, hệ thống được triển khai dưới dạng dịch vụ web nhằm phục vụ truy vấn trong môi trường thực tế. Phần này trình bày kiến trúc triển khai, cơ chế vận hành và các yếu tố liên quan đến tài nguyên tính toán.

4.6.1 Kiến trúc triển khai tổng thể

Hệ thống được xây dựng theo mô hình client-server, bao gồm hai tầng chính:

- **Tầng xử lý (Backend):** triển khai bằng Python và Flask, chịu trách nhiệm điều phối toàn bộ quy trình truy vấn.
- **Tầng giao diện (Frontend):** cho phép người dùng nhập truy vấn, lựa chọn cấu hình tìm kiếm và hiển thị kết quả.

Backend đóng vai trò trung tâm, kết nối các thành phần:

- Mô hình embedding (MiniLM hoặc BGE-M3).

- Chỉ mục FAISS tương ứng.
- Cơ chế reranking (hybrid hoặc cross-encoder).

4.6.2 Thiết kế API truy vấn

Hệ thống cung cấp endpoint chính:

`/search`

Yêu cầu gửi đến endpoint bao gồm:

- Nội dung truy vấn.
- Lựa chọn backend tìm kiếm.
- Số lượng kết quả cần trả về (Top- K).

Sau khi xử lý, hệ thống trả về dữ liệu dưới dạng JSON gồm:

- Danh sách kết quả đã xếp hạng.
- Điểm tương đồng.
- Thông tin tài liệu nguồn.
- Đoạn văn bản mở rộng theo ngữ cảnh.

Thiết kế RESTful giúp hệ thống có thể tích hợp với các ứng dụng khác như nền tảng thư viện số, công cụ kiểm tra tương đồng hoặc hệ thống chatbot học thuật.

4.6.3 Quản lý hai backend tìm kiếm

Hệ thống duy trì hai backend độc lập:

- Backend MiniLM + Hybrid.
- Backend BGE-M3 + Cross-Encoder.

Khi người dùng lựa chọn cấu hình, backend sẽ:

1. Nạp chỉ mục FAISS tương ứng (nếu chưa có trong bộ nhớ).
2. Mã hóa truy vấn bằng mô hình embedding phù hợp.
3. Thực hiện truy hồi và xếp hạng theo cấu hình đã chọn.

Thiết kế này đảm bảo mỗi truy vấn được xử lý nhất quán trong cùng một không gian embedding.

4.6.4 Quản lý tài nguyên và tối ưu hiệu năng

Hai backend có đặc điểm tiêu thụ tài nguyên khác nhau:

- MiniLM: chạy hiệu quả trên CPU, phù hợp môi trường nhiều truy vấn đồng thời.
- BGE + Cross-Encoder: yêu cầu tài nguyên tính toán cao hơn, đặc biệt khi rerank nhiều ứng viên.

Để đảm bảo hệ thống vận hành ổn định, backend BGE được cấu hình thêm các biến môi trường nhằm tối ưu xử lý song song và tránh xung đột thư viện:

```
os.environ["KMP_DUPLICATE_LIB_OK"] = "TRUE"
os.environ["OMP_NUM_THREADS"] = "36"
os.environ["TOKENIZERS_PARALLELISM"] = "true"
```

Việc phân tách hai cấu hình giúp hệ thống linh hoạt trong việc phân bổ tài nguyên, đặc biệt khi triển khai trên máy chủ có GPU.

4.6.5 Cơ chế tái nạp chỉ mục

Chỉ mục FAISS được lưu xuống đĩa sau khi xây dựng. Khi hệ thống khởi động, chỉ mục được nạp lại vào bộ nhớ thay vì xây dựng lại từ đầu.

Hệ thống cũng hỗ trợ cơ chế tái nạp (reload) khi tập dữ liệu thay đổi. Điều này giúp:

- Giảm thời gian cập nhật.
- Duy trì tính liên tục của dịch vụ.

4.6.6 Khả năng mở rộng và tích hợp

Kiến trúc hiện tại cho phép mở rộng theo các hướng:

- Thay thế mô hình embedding bằng phiên bản mới mà không thay đổi tầng API.
- Chuyển sang chỉ mục FAISS gần đúng khi quy mô dữ liệu tăng.
- Tách riêng dịch vụ embedding và reranking thành các microservice độc lập.

Do giao tiếp giữa frontend và backend sử dụng HTTP và JSON, hệ thống có thể dễ dàng tích hợp vào các nền tảng lớn hơn hoặc triển khai trên môi trường đám mây.

4.6.7 Đánh giá tổng thể về triển khai

Việc triển khai hệ thống dưới dạng dịch vụ web giúp kiểm nghiệm toàn bộ pipeline trong môi trường gần với thực tế sử dụng. Thiết kế hai backend song song mang lại sự linh hoạt trong vận hành, cho phép lựa chọn giữa hiệu năng và độ chính xác tùy theo yêu cầu.

Cấu trúc mô-đun, tách biệt giữa embedding, chỉ mục và reranking, tạo nền tảng vững chắc cho việc mở rộng và nâng cấp hệ thống trong tương lai.

4.7 Thiết kế giao diện người dùng

4.7.1 Tổng quan kiến trúc frontend

Giao diện người dùng của hệ thống được xây dựng dưới dạng **Single Page Application (SPA)** nhằm cung cấp môi trường trực quan để thực hiện các thí nghiệm semantic search, quản lý dữ liệu và phân tích kết quả. Ứng dụng frontend được phát triển bằng **React** kết hợp với **TypeScript**, sử dụng **Vite** làm công cụ build và **Tailwind CSS** cho hệ thống giao diện.

Không giống các ứng dụng web thông thường sử dụng router để điều hướng trang, hệ thống này áp dụng mô hình *state-based navigation*. Toàn

bộ các module chức năng được tổ chức thành các **tab**, và việc chuyển đổi giữa các tab được điều khiển thông qua biến trạng thái trung tâm `activeTab`.

Cấu trúc khởi tạo của frontend gồm hai thành phần chính:

- **Entry point:** `main.tsx`
- **App shell:** `App.tsx`

Trong đó, `main.tsx` chịu trách nhiệm khởi tạo ứng dụng React và mount root component, còn `App.tsx` đóng vai trò container chính của toàn bộ hệ thống, quản lý sidebar navigation và điều khiển việc hiển thị các module.

Toàn bộ giao diện được tổ chức thành **10 module chức năng** phục vụ cho các mục đích khác nhau trong quá trình nghiên cứu semantic search:

1. Experiment Playground
2. Query Test Results
3. Dataset Management
4. QA Validation
5. QA Evaluation
6. MiniLM Evaluation
7. BGE Evaluation
8. Comparative Analysis
9. Indexing & Chunking
10. Vector Database Evaluation

Các module này tạo thành một dashboard hoàn chỉnh cho phép người dùng thực hiện các thí nghiệm truy hồi văn bản, đánh giá mô hình và phân tích dữ liệu.

4.7.2 Nền tảng công nghệ frontend

Frontend của hệ thống được xây dựng dựa trên các công nghệ web hiện đại nhằm đảm bảo hiệu năng và khả năng mở rộng.

Bundler và môi trường phát triển Hệ thống sử dụng **Vite** làm bundler chính. So với các bundler truyền thống, Vite cung cấp thời gian khởi động nhanh hơn và hỗ trợ *Hot Module Replacement (HMR)* trong quá trình phát triển. Cấu hình build được định nghĩa trong file `vite.config.ts`, trong khi các script phát triển được khai báo trong `package.json`.

Ngôn ngữ và framework Frontend được phát triển bằng **React** kết hợp với **TypeScript**. Việc sử dụng TypeScript giúp tăng tính an toàn kiểu dữ liệu, giảm lỗi runtime và hỗ trợ tốt hơn cho các hệ thống component lớn.

Theo thống kê codebase, phần frontend bao gồm:

- 114 file `.tsx` chứa các component giao diện
- 8 file `.ts` chứa các định nghĩa kiểu dữ liệu và hằng số

Hệ thống styling Styling của hệ thống được xây dựng trên **Tailwind CSS v4**, kết hợp với các file CSS tùy chỉnh như `index.css`, `tailwind.css` và `theme.css`. Cách tiếp cận này cho phép thiết kế giao diện nhanh, giảm CSS dư thừa và đảm bảo tính nhất quán giữa các component.

Thư viện UI nội bộ Frontend sử dụng một thư viện UI component nội bộ gồm khoảng **47 component** nằm trong thư mục `ui`. Các component này bao gồm:

- Card
- Table
- Modal

- Tabs
- Form control

Nhờ hệ thống component hóa, nhiều phần giao diện có thể tái sử dụng giữa các module khác nhau, giúp giảm trùng lặp code và tăng tính nhất quán của giao diện.

4.7.3 Cấu trúc module của hệ thống

Frontend được tổ chức thành hai nhóm thư mục chính:

- **Dashboard modules:** chứa các module nghiệp vụ của hệ thống semantic search.
- **Shared UI components:** chứa các component giao diện dùng chung.

Các module dashboard được thiết kế theo nguyên tắc tách biệt logic nghiệp vụ và giao diện hiển thị, giúp tăng khả năng mở rộng và bảo trì hệ thống.

4.7.4 Module Experiment Playground

Module Experiment Playground đóng vai trò môi trường thử nghiệm truy vấn semantic search theo thời gian thực. Container chính của module là `SearchQATesting.tsx`. Giao diện được chia thành ba khu vực chính:

- **Left Panel:** cấu hình truy vấn, lựa chọn ngôn ngữ, mô hình embedding và giá trị Top-K.
- **Middle Panel:** hiển thị danh sách các đoạn văn được truy hồi cùng điểm similarity.
- **Right Panel:** hiển thị các thông tin runtime như độ trễ truy vấn và số lượng kết quả trả về.

Module này cho phép người dùng thực hiện truy vấn trực tiếp lên hệ thống semantic search thông qua API:

POST <http://localhost:5000/search>

Ngoài ra hệ thống còn hỗ trợ truy vết ngược từ kết quả truy hồi về tài liệu PDF nguồn thông qua endpoint:

[/qa/doc-meta](#)

4.7.5 Module Query Test Results

Module này được sử dụng để đánh giá hiệu năng của hệ thống trên tập truy vấn benchmark. Container chính là `QueryTestResults.tsx` và dữ liệu được lấy từ API:

GET <http://localhost:4000/query-test/query>

Module bao gồm các tab phân tích khác nhau như:

- All Queries
- Query Types
- Language Analysis
- Diagnostics

Nhờ đó người dùng có thể quan sát hiệu năng truy hồi theo loại truy vấn, theo ngôn ngữ và phát hiện các trường hợp truy hồi thất bại.

4.7.6 Module Dataset Management

Module Dataset Management cho phép quản lý kho tài liệu PDF nguồn phục vụ cho semantic search. Container chính là `DatasetManagement.tsx`. Module này cung cấp các chức năng:

- Hiển thị thống kê tổng số tài liệu
- Lọc tài liệu theo ngôn ngữ
- Xem danh sách file PDF

- Mở và tải xuống tài liệu nguồn

Các API chính được sử dụng gồm:

`/dataset/stats`

`/dataset/list`

`/qa/doc-meta`

4.7.7 Module QA Validation

Module QA Validation được sử dụng để kiểm tra độ tin cậy của các cặp câu hỏi–trả lời (QA pair) thông qua pipeline kiểm định hai tầng. Container chính là `QAVValidation.tsx`.

Module này cho phép:

- lọc QA theo dataset, model và experiment
- xem thống kê độ tin cậy của QA
- truy vết QA về đoạn văn và tài liệu nguồn

Các API được sử dụng gồm:

`/summary/qa-list`

`/summary/dataset-overview`

`/summary/get-summary`

4.7.8 Module QA Evaluation

Module QA Evaluation được sử dụng để đánh giá chất lượng QA sinh ra bởi các mô hình ngôn ngữ lớn. Container chính là `QAEvaluation.tsx`.

Module này cung cấp các biểu đồ và bảng so sánh hiệu năng giữa các mô hình như GPT, Gemini và DeepSeek, giúp phân tích mức độ chính xác và độ ổn định của dữ liệu QA.

4.7.9 Module đánh giá mô hình truy hồi

Hai module riêng biệt được xây dựng để đánh giá hiệu năng của hai cấu hình truy hồi:

- **Option1 (MiniLM Evaluation)**
- **Option2 (BGE Evaluation)**

Mỗi module được chia thành các tab phân tích:

- Executive Summary
- Model Comparison
- Language Analysis
- Latency
- Detailed Tables

Các API tương ứng gồm:

```
/evaluation/option1/*  
/evaluation/option2/*
```

4.7.10 Comparative Analysis

Module Comparative Analysis cho phép so sánh trực tiếp hiệu năng giữa MiniLM và BGE. Các biểu đồ và bảng kết quả giúp phân tích sự khác biệt về độ chính xác, tốc độ truy hồi và hiệu quả xử lý giữa hai mô hình.

4.7.11 Indexing và Chunking

Module Indexing & Chunking phân tích ảnh hưởng của chiến lược indexing và kích thước chunk đối với hiệu năng truy hồi. Module này sử dụng API:

```
/add/indexing-chunking
```

Các biểu đồ hiển thị mối quan hệ giữa kích thước chunk, độ trễ truy vấn và độ chính xác truy hồi.

4.7.12 Vector Database Evaluation

Module Vector Database Evaluation được xây dựng để so sánh các chiến lược lưu trữ vector trong FAISS. Các biểu đồ phân tích bao gồm:

- Search latency
- Recall
- Storage cost
- Insert speed

4.7.13 Luồng dữ liệu frontend

Frontend sử dụng mô hình **component-level fetch**. Mỗi module sẽ tự gọi API khi component được mount hoặc khi bộ lọc thay đổi.

Quản lý trạng thái được thực hiện thông qua các hook của React như:

- `useState`
- `useEffect`
- `useMemo`

Một số module sử dụng `AbortController` để tránh tình trạng race condition khi nhiều request được gửi đồng thời.

4.7.14 Đánh giá kiến trúc frontend

Kiến trúc frontend của hệ thống có một số ưu điểm nổi bật:

- Chia module rõ ràng theo nghiệp vụ semantic search
- Visualization phong phú với nhiều loại biểu đồ
- Hỗ trợ truy vết từ kết quả truy hồi về tài liệu nguồn
- Component hóa tốt, dễ mở rộng

Tuy nhiên hệ thống vẫn tồn tại một số hạn chế:

- Endpoint API được hard-code trực tiếp trong code
- Không sử dụng router URL-based
- Một số artifact build nằm trong thư mục source
- Chưa có hệ thống lint hoặc test tự động

4.7.15 Kết luận

Frontend của hệ thống cung cấp một môi trường trực quan cho toàn bộ pipeline semantic search, từ quản lý dữ liệu, sinh QA, kiểm định QA đến đánh giá mô hình truy hồi. Việc tích hợp nhiều module phân tích và visualization giúp hệ thống không chỉ đóng vai trò là công cụ chạy thí nghiệm, mà còn là một dashboard nghiên cứu hoàn chỉnh cho hệ thống truy hồi ngữ nghĩa song ngữ.

Chương 5

Thực nghiệm và đánh giá

5.1 Ngữ liệu

Phần này trình bày chi tiết bộ dữ liệu được sử dụng trong thực nghiệm, bao gồm quy mô tài liệu nguồn, phân bố theo ngôn ngữ và thống kê dữ liệu sinh QA.

Các thống kê trong phần này chủ yếu dựa trên kết quả sau pipeline kiểm định hai tầng (bi-encoder và cross-encoder) trong giai đoạn đầu, trước khi bổ sung bước kiểm định suy luận bằng NLI ở các phần phân tích tiếp theo.

Quy mô tài liệu nguồn

Tổng số tài liệu sử dụng trong nghiên cứu:

- **Tổng số PDF: 1041 tài liệu**
- **Tiếng Anh: 542 tài liệu**
- **Tiếng Việt: 499 tài liệu**
- **Tổng dung lượng dữ liệu: 784.3 MB**

Trung bình mỗi tài liệu có dung lượng khoảng:

$$\frac{784.3}{1041} \approx 0.75 \text{ MB/PDF}$$

Bộ dữ liệu bao gồm các tài liệu học thuật và chuyên ngành ở cả tiếng Việt và tiếng Anh. Việc sử dụng dữ liệu thực tế thay vì các bộ benchmark cố định giúp đánh giá hệ thống trong bối cảnh gần với ứng dụng thực tiễn hơn.

Thống kê sinh QA

Từ 1041 PDF, hệ thống sinh tổng cộng:

70,870 QA

Phân bố theo mô hình sinh:

- Gemini 2.5 Flash: 16,799 QA
- GPT-5.2: 24,539 QA
- DeepSeek R1T2 Chimera: 29,537 QA

Trung bình mỗi PDF sinh ra:

$$\frac{70,870}{1041} \approx 68.1 \text{ QA/tài liệu}$$

Số lượng QA lớn này giúp tạo ra mật độ truy vấn đủ cao để đánh giá hệ thống semantic search trên nhiều tình huống ngữ nghĩa khác nhau.

Thống kê sau kiểm định ngưỡng $\tau = 0.70$

Sau pipeline kiểm định hai tầng (bi-encoder và cross-encoder) trong giai đoạn đầu, tổng số QA được giữ lại là:

26,083 QA

Tỉ lệ validation toàn hệ thống đạt:

$$\frac{26,083}{70,870} \approx 36.8\%$$

Phân theo mô hình:

- Gemini 2.5 Flash: 8,790 / 16,799 (52.3%)
- GPT-5.2: 8,511 / 24,539 (34.7%)
- DeepSeek R1T2 Chimera: 8,786 / 29,537 (29.7%)

Trung bình mỗi tài liệu sau kiểm định còn:

$$\frac{26,083}{1041} \approx 25.1 \text{ QA/PDF}$$

Ngưỡng 0.70 tạo ra một tập dữ liệu có độ bao phủ tương đối tốt, đồng thời vẫn duy trì được mức độ tin cậy ngữ nghĩa phù hợp cho thực nghiệm.

Thống kê sau kiểm định ngưỡng $\tau = 0.80$

Ở ngưỡng nghiêm ngặt hơn, tổng số QA được giữ lại là:

10,372 QA

Tỉ lệ validation toàn hệ thống đạt:

$$\frac{10,372}{70,870} \approx 14.6\%$$

Phân theo mô hình:

- Gemini 2.5 Flash: 4,451 / 16,799 (26.5%)
- GPT-5.2: 3,379 / 24,539 (13.8%)
- DeepSeek R1T2 Chimera: 2,544 / 29,537 (8.6%)

Trung bình mỗi PDF còn lại:

$$\frac{10,372}{1041} \approx 10.0 \text{ QA/PDF}$$

Ngưỡng 0.80 giúp tạo ra tập dữ liệu có độ chính xác ngữ nghĩa cao hơn, nhưng đồng thời làm giảm mạnh số lượng QA giữ lại.

Nhận xét về đặc điểm ngữ liệu

Từ các thống kê trên có thể rút ra một số nhận xét chính:

- Bộ dữ liệu có quy mô tương đối lớn, gồm 1041 tài liệu với tổng dung lượng 784.3 MB, đủ để đánh giá hệ thống trong điều kiện gần với thực tế.

- Việc sinh tổng cộng 70,870 QA tạo ra mật độ truy vấn cao, giúp kiểm tra khả năng semantic search trên nhiều mức độ tương đồng ngữ nghĩa khác nhau.
- Ở ngưỡng $\tau = 0.70$, hệ thống giữ lại 36.8% số QA, cho thấy dữ liệu sinh tự động có chất lượng tương đối tốt sau khi qua pipeline kiểm định.
- Ở ngưỡng $\tau = 0.80$, số lượng QA giảm mạnh nhưng độ tin cậy của tập dữ liệu tăng lên, phù hợp cho các thực nghiệm ưu tiên precision.
- Gemini 2.5 Flash cho tỉ lệ QA đạt chuẩn cao nhất ở cả hai ngưỡng, cho thấy độ ổn định ngữ nghĩa tốt hơn.
- GPT-5.2 cho kết quả ở mức trung gian, phản ánh sự cân bằng giữa độ đa dạng và độ chính xác.
- DeepSeek R1T2 Chimera sinh ra số lượng QA lớn nhất nhưng tỉ lệ giữ lại thấp hơn, cho thấy dữ liệu đa dạng hơn nhưng cần kiểm định chặt hơn.

Ghi chú về bước kiểm định NLI. Các thống kê trong phần này phản ánh chất lượng dữ liệu sau hai tầng kiểm định ban đầu. Trong các phần phân tích tiếp theo, đề tài tiếp tục bổ sung một tầng kiểm định thứ ba dựa trên Natural Language Inference (NLI) nhằm nâng cao độ chính xác và giảm các lỗi suy luận.

Kết quả thực nghiệm cho thấy việc bổ sung NLI giúp cải thiện đáng kể chất lượng dữ liệu QA, đặc biệt trong việc loại bỏ các câu trả lời có xu hướng suy diễn hoặc vượt quá phạm vi context.

Bộ ngữ liệu sau kiểm định có tính truy vết về nguồn PDF, có cấu trúc rõ ràng và đủ mật độ truy vấn để phục vụ đánh giá hệ thống semantic search xuyên ngữ trong các chương tiếp theo.

5.2 Thiết lập thực nghiệm

Phần thiết lập cấu hình thực nghiệm được sử dụng để đánh giá hệ thống semantic search xuyên ngữ Việt – Anh. Mục tiêu của thực nghiệm là đo lường độ chính xác truy hồi (retrieval effectiveness) và hiệu năng xử lý (latency, throughput) của hai cấu hình tìm kiếm đã triển khai.

5.2.1 Môi trường phần cứng

Thực nghiệm được tiến hành trên máy chủ với cấu hình:

- CPU: Intel(R) Xeon(R) E5-2696 v3 @ 2.30GHz (2 processors)
- RAM: 96GB
- GPU: NVIDIA GPU với bộ nhớ 40GB VRAM

Trong quá trình chạy cấu hình sử dụng mô hình BGE và cross-encoder reranking, mức sử dụng GPU dao động khoảng 19% và bộ nhớ GPU sử dụng khoảng 7.2GB trên tổng 40GB. Nhiệt độ GPU duy trì ở mức trung bình 49°C trong suốt quá trình thực nghiệm.

Cấu hình sử dụng MiniLM không yêu cầu GPU và có thể chạy hoàn toàn trên CPU với bộ nhớ RAM tiêu chuẩn.

5.2.2 Môi trường phần mềm

Hệ thống được triển khai bằng ngôn ngữ Python. Các thư viện chính bao gồm:

- `sentence-transformers` để sinh embedding.
- `FAISS` để xây dựng và truy vấn chỉ mục vector.
- `NumPy` để xử lý vector và tính toán số học.

Đối với cấu hình BGE, trước khi khởi tạo các thư viện liên quan đến FAISS và Torch, một số biến môi trường được thiết lập nhằm tối ưu hóa xử lý song song và tránh xung đột thư viện:

```
KMP_DUPLICATE_LIB_OK = TRUE
OMP_NUM_THREADS = 36
TOKENIZERS_PARALLELISM = true
```

Các thiết lập này giúp tận dụng khả năng đa luồng của CPU và giảm thiểu cảnh báo liên quan đến OpenMP khi sử dụng mô hình lớn.

5.2.3 Dữ liệu kiểm thử

Dữ liệu kiểm thử được xây dựng từ tập QA sinh tự động ở Chương 3. Mỗi mẫu dữ liệu bao gồm:

- Câu hỏi (query).
- Tiêu đề tài liệu mong đợi.
- Đoạn nội dung (context) mong đợi chứa câu trả lời.

Tập dữ liệu được lưu dưới định dạng JSON hoặc JSONL và được nạp vào chương trình kiểm thử `test.py`. Mỗi truy vấn được sử dụng để đánh giá khả năng hệ thống truy hồi đúng tài liệu và đoạn văn bản liên quan.

5.2.4 Cấu hình truy hồi

Hai cấu hình truy hồi được sử dụng trong thực nghiệm:

Cấu hình 1 – MiniLM (Bi-Encoder thuần)

Cấu hình này sử dụng mô hình `paraphrase-multilingual-MiniLM-L12-v2` để sinh embedding cho truy vấn và tài liệu. Hệ thống thực hiện:

1. Mã hóa truy vấn thành vector embedding.
2. Truy hồi Top- K vector gần nhất từ chỉ mục FAISS.
3. Trả về kết quả theo thứ tự khoảng cách cosine.

Cấu hình này không sử dụng bước reranking bằng cross-encoder, do đó ưu tiên tốc độ và khả năng phục vụ nhiều truy vấn đồng thời.

Cấu hình 2 – BGE + Cross-Encoder Reranking

Cấu hình này sử dụng embedding model BGE để mã hóa truy vấn và tài liệu, sau đó áp dụng kiến trúc truy hồi hai giai đoạn:

1. Truy hồi sơ cấp Top- N ứng viên từ FAISS.
2. Xếp hạng lại (rerank) bằng cross-encoder để đánh giá mức độ liên quan sâu giữa truy vấn và từng ứng viên.

Cấu hình này yêu cầu tài nguyên tính toán cao hơn do bước reranking có độ phức tạp lớn hơn, nhưng kỳ vọng đạt độ chính xác truy hồi cao hơn.

5.2.5 Chỉ số đánh giá

Hiệu năng hệ thống được đánh giá theo các mức Top- K với:

$$K \in \{1, 3, 5, 10\}$$

Đối với mỗi truy vấn, ba thành phần điểm số được tính:

- **Title score:** đánh giá mức độ trùng khớp tiêu đề.
- **Context score:** đánh giá mức độ trùng khớp nội dung.
- **Total score:** trung bình của hai thành phần trên.

Một truy vấn được xem là truy hồi thành công nếu tổng điểm đạt ngưỡng ≥ 0.7 .

Ngoài độ chính xác, hệ thống còn ghi nhận:

- Thời gian xử lý mỗi truy vấn (latency).
- Độ trễ trung bình (Average latency).
- P50, P90, P95 latency.
- Throughput (số truy vấn xử lý trên giây).

Các chỉ số này cho phép so sánh toàn diện giữa hai cấu hình về cả chất lượng truy hồi và hiệu năng xử lý.

5.3 Đánh giá hiệu năng truy hồi

5.3.1 Mục tiêu đánh giá

Phần này trình bày các kết quả thực nghiệm nhằm đánh giá hiệu năng của hệ thống truy hồi văn bản trong bối cảnh dữ liệu học thuật song ngữ Việt–Anh. Mục tiêu của các thí nghiệm là đo lường khả năng của hệ thống trong việc truy hồi chính xác các đoạn văn có chứa thông tin liên quan đến truy vấn đầu vào.

Trong các hệ thống hỏi đáp dựa trên truy hồi (Retrieval-Augmented Generation – RAG), chất lượng của bước truy hồi đóng vai trò quan trọng vì các đoạn văn được truy hồi sẽ được sử dụng làm ngữ cảnh cho mô hình ngôn ngữ sinh câu trả lời. Do đó, khả năng đưa đoạn văn chứa thông tin cần thiết vào các vị trí đầu của danh sách kết quả là yếu tố quan trọng quyết định chất lượng của toàn bộ hệ thống.

Trong nghiên cứu này, hiệu năng truy hồi được đánh giá thông qua các chỉ số phổ biến trong lĩnh vực Information Retrieval:

- **Top-1:** đoạn chứa đáp án xuất hiện ở vị trí đầu tiên.
- **Top-3:** đoạn chứa đáp án nằm trong ba kết quả đầu.
- **Top-5:** đoạn chứa đáp án nằm trong năm kết quả đầu.
- **Top-10:** đoạn chứa đáp án nằm trong mười kết quả đầu.

Các chỉ số này phản ánh mức độ hiệu quả của hệ thống trong việc đưa các đoạn văn liên quan lên những vị trí ưu tiên, từ đó cải thiện khả năng sinh câu trả lời chính xác trong hệ thống hỏi đáp.

5.3.2 Thiết lập thực nghiệm

Toàn bộ các thí nghiệm được thực hiện thông qua script `test.py`. Trong quá trình chạy thực nghiệm, các tham số của hệ thống được giữ cố định nhằm đảm bảo tính nhất quán giữa các cấu hình thử nghiệm. Các thay đổi chủ yếu nằm ở backend truy hồi và môi trường thực thi.

Hai backend truy hồi được sử dụng trong nghiên cứu này gồm:

- **Backend 1:** sử dụng mô hình embedding đa ngôn ngữ `MiniLM-L12-v2` (thuộc họ `paraphrase-multilingual`) để biểu diễn các đoạn văn bản và truy vấn dưới dạng vector. Độ tương đồng giữa truy vấn và các đoạn văn được tính bằng cosine similarity. Backend này được triển khai và thực thi trên CPU.
- **Backend 2:** sử dụng mô hình embedding `BAAI/bge-m3` kết hợp với mô hình reranker `BAAI/bge-reranker-v2-m3`. Trong cấu hình này, hệ thống thực hiện truy hồi sơ bộ bằng embedding, sau đó áp dụng mô hình reranker để đánh giá lại mức độ liên quan giữa truy vấn và các đoạn văn nhằm cải thiện thứ hạng kết quả. Backend này được triển khai trên GPU.

Các truy vấn đánh giá được sinh từ pipeline tạo dữ liệu hỏi đáp đã được mô tả trong các chương trước. Tập truy vấn được phân chia theo hai nguồn sinh câu hỏi:

- GPT-5.2
- Gemini 2.5 Flash
- DeepSeek R1T2 Chimera

Ngoài ra, các thí nghiệm cũng được phân tách theo hai ngôn ngữ gồm tiếng Việt và tiếng Anh nhằm đánh giá khả năng truy hồi đa ngôn ngữ của hệ thống.

5.3.3 Kết quả thực nghiệm với Backend 1 (MiniLM)

Kết quả trên tập truy vấn tiếng Việt

Filter	Total Items	Retention	Top1	Top3	Top5	Top10	Avg Latency (ms)	Throughput (qps)
raw	8963	1.0000	0.7362	0.8207	0.8463	0.8740	6546.81	0.14
0.7	4256	0.4749	0.7966	0.8724	0.8931	0.9182	5902.81	0.16
0.8	2023	0.2257	0.8434	0.9067	0.9216	0.9411	7001.23	0.14

Bảng 5.1: So sánh trước và sau lọc (ngưỡng 0.7/0.8) cho tập GEMINI-VI (MiniLM)

Filter	Total Items	Retention	Top1	Top3	Top5	Top10	Avg Latency (ms)	Throughput (qps)
raw	16595	1.0000	0.6765	0.7692	0.8013	0.8354	5985.60	0.16
0.7	4252	0.2562	0.7732	0.8547	0.8821	0.9084	5781.52	0.17
0.8	1087	0.0655	0.8536	0.9134	0.9297	0.9450	7854.93	0.12

Bảng 5.2: So sánh trước và sau lọc (ngưỡng 0.7/0.8) cho tập DEEPSEEK-VI (MiniLM)

Filter	Total Items	Retention	Top1	Top3	Top5	Top10	Avg Latency (ms)	Throughput (qps)
raw	12559	1.0000	0.7130	0.7970	0.8253	0.8535	6250.40	0.15
0.7	4017	0.3198	0.7864	0.8628	0.8864	0.9073	5840.30	0.16
0.8	1523	0.1213	0.8488	0.9100	0.9255	0.9433	7420.60	0.13

Bảng 5.3: So sánh trước và sau lọc (ngưỡng 0.7/0.8) cho tập GPT-VI (MiniLM)

Hai bảng trên trình bày kết quả thực nghiệm của backend MiniLM trên tập truy vấn tiếng Việt. Có thể quan sát rằng khi áp dụng các mức lọc QA với ngưỡng cao hơn, số lượng truy vấn giảm xuống. Tuy nhiên, các chỉ số Top-K có xu hướng được cải thiện, cho thấy rằng các truy vấn được giữ lại có chất lượng cao hơn và giúp hệ thống truy hồi chính xác hơn.

Kết quả trên tập truy vấn tiếng Anh

Filter	Total Items	Retention	Top1	Top3	Top5	Top10	Avg Latency (ms)	Throughput (qps)
raw	7860	1.0000	0.7908	0.8543	0.8708	0.8904	5396.92	0.18
0.7	4579	0.5826	0.8341	0.8901	0.9041	0.9193	5749.29	0.17
0.8	2447	0.3113	0.8645	0.9087	0.9207	0.9303	6200.71	0.15

Bảng 5.4: So sánh trước và sau lọc (ngưỡng 0.7/0.8) cho tập GEMINI-ENG (MiniLM)

Filter	Total Items	Retention	Top1	Top3	Top5	Top10	Avg Latency (ms)	Throughput (qps)
raw	12518	1.0000	0.7319	0.8112	0.8351	0.8609	5033.10	0.19
0.7	4503	0.3597	0.8082	0.8745	0.8921	0.9098	5695.98	0.17
0.8	1433	0.1145	0.8620	0.9069	0.9183	0.9293	6538.58	0.14

Bảng 5.5: So sánh trước và sau lọc (ngưỡng 0.7/0.8) cho tập DEEPSEEK-ENG (MiniLM)

Filter	Total Items	Retention	Top1	Top3	Top5	Top10	Avg Latency (ms)	Throughput (qps)
raw	11569	1.0000	0.7641	0.8303	0.8523	0.8738	5220.40	0.19
0.7	4397	0.3800	0.8215	0.8828	0.8982	0.9142	5718.40	0.17
0.8	1788	0.1545	0.8635	0.9077	0.9195	0.9298	6360.40	0.15

Bảng 5.6: So sánh trước và sau lọc (ngưỡng 0.7/0.8) cho tập GPT-EN (MiniLM)

Xu hướng trên tập truy vấn tiếng Anh tương tự với tiếng Việt. Khi tăng mức lọc dữ liệu, số lượng mẫu giảm nhưng các chỉ số Top-K được cải thiện. Điều này cho thấy hệ thống có khả năng xử lý tốt truy vấn ở cả hai ngôn ngữ.

5.3.4 Phân tích trực quan theo Top-K (Backend 1)

Tổng quan các chỉ số truy hồi

Complete Metrics Table (All Configurations)
Accuracy + Latency distribution (Avg / P50 / P90 / P95)

Model	Lang	Threshold	Top-1	Top-3	Top-5	Top-10	Avg	P50	P90	P95	Throughput
DeepSeek	EN	0.7	80.8%	87.5%	89.2%	91.0%	5696ms	5275ms	5954ms	6577ms	0.17 q/s
DeepSeek	EN	0.8	86.2%	90.7%	91.8%	92.8%	6539ms	5685ms	6976ms	8955ms	0.14 q/s
DeepSeek	EN	All QAs	73.2%	81.1%	83.5%	86.1%	5033ms	4888ms	5526ms	5733ms	0.19 q/s
DeepSeek	VI	0.7	77.3%	85.5%	88.2%	90.8%	5782ms	4975ms	7792ms	8161ms	0.17 q/s
DeepSeek	VI	0.8	85.4%	91.3%	93.0%	94.5%	7855ms	7669ms	8414ms	9255ms	0.12 q/s
DeepSeek	VI	All QAs	67.7%	76.9%	80.1%	83.5%	5986ms	4583ms	11282ms	12285ms	0.16 q/s
Gemini	EN	0.7	81.4%	89.0%	90.4%	91.9%	5749ms	5346ms	6077ms	6724ms	0.17 q/s
Gemini	EN	0.8	86.5%	90.9%	92.1%	93.0%	6201ms	5523ms	6667ms	7339ms	0.15 q/s
Gemini	EN	All QAs	79.1%	85.4%	87.1%	89.0%	5397ms	5207ms	5795ms	6213ms	0.18 q/s
Gemini	VI	0.7	79.7%	87.2%	89.3%	91.8%	5903ms	5091ms	7934ms	8578ms	0.16 q/s
Gemini	VI	0.8	84.3%	90.7%	92.2%	94.1%	7001ms	7385ms	8682ms	9746ms	0.14 q/s
Gemini	VI	All QAs	73.6%	82.1%	84.6%	87.4%	6547ms	5004ms	11361ms	12210ms	0.14 q/s
GPT	EN	0.7	82.2%	88.3%	89.8%	91.4%	5718ms	5309ms	6006ms	6648ms	0.17 q/s
GPT	EN	0.8	86.4%	90.8%	92.0%	93.0%	6360ms	5550ms	6820ms	8100ms	0.15 q/s
GPT	EN	All QAs	76.4%	83.0%	85.2%	87.4%	5220ms	5030ms	5650ms	5970ms	0.19 q/s
GPT	VI	0.7	78.6%	86.3%	88.6%	90.7%	5840ms	5032ms	7810ms	8255ms	0.16 q/s
GPT	VI	0.8	84.9%	91.0%	92.5%	94.3%	7421ms	7520ms	8550ms	9500ms	0.13 q/s
GPT	VI	All QAs	71.3%	79.7%	82.5%	85.4%	6250ms	4700ms	11250ms	12180ms	0.15 q/s

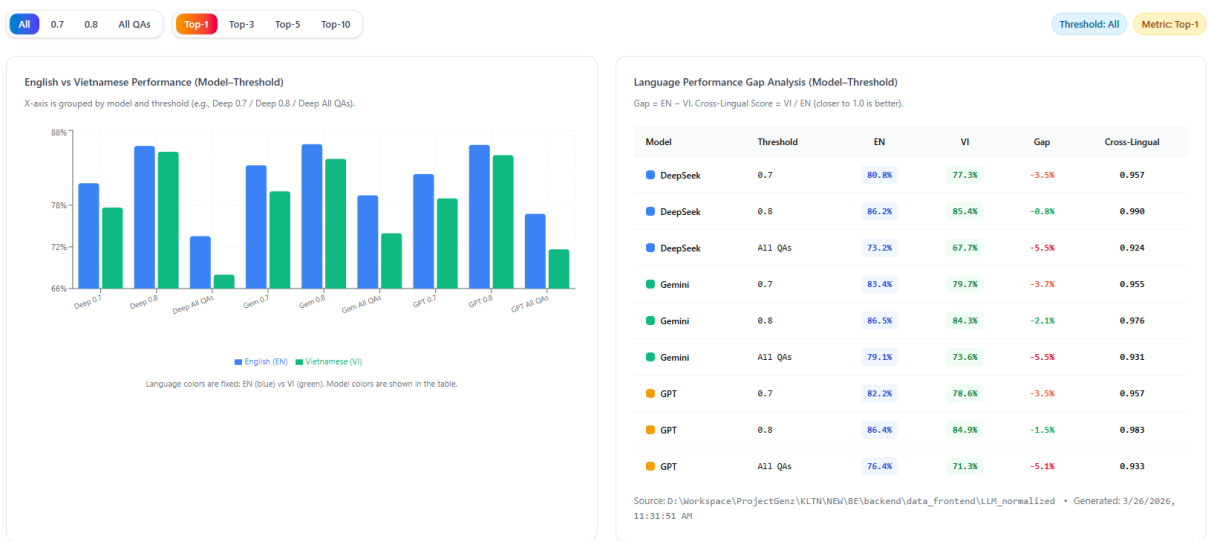
Hình 5.1: Biểu đồ tổng quan các chỉ số truy hồi Top-K khi sử dụng mô hình paraphrase-multilingual-MiniLM-L12-v2

Biểu đồ trên thể hiện tổng quan hiệu năng truy hồi của hệ thống theo các chỉ số Top-K. Có thể nhận thấy rằng khi giá trị K tăng lên, xác suất đoạn văn chứa đáp án xuất hiện trong danh sách kết quả cũng tăng theo.

Top-1



Hình 5.2: Biểu đồ kết quả truy hồi Top-1 giữa các bộ dataset khi sử dụng mô hình paraphrase-multilingual-MiniLM-L12-v2



Hình 5.3: Biểu đồ so sánh kết quả truy hồi Top-1 giữa truy vấn tiếng Việt và tiếng Anh khi sử dụng mô hình paraphrase-multilingual-MiniLM-L12-v2

Top-3



Hình 5.4: Biểu đồ kết quả truy hồi Top-3 giữa các bộ dataset khi sử dụng mô hình paraphrase-multilingual-MiniLM-L12-v2

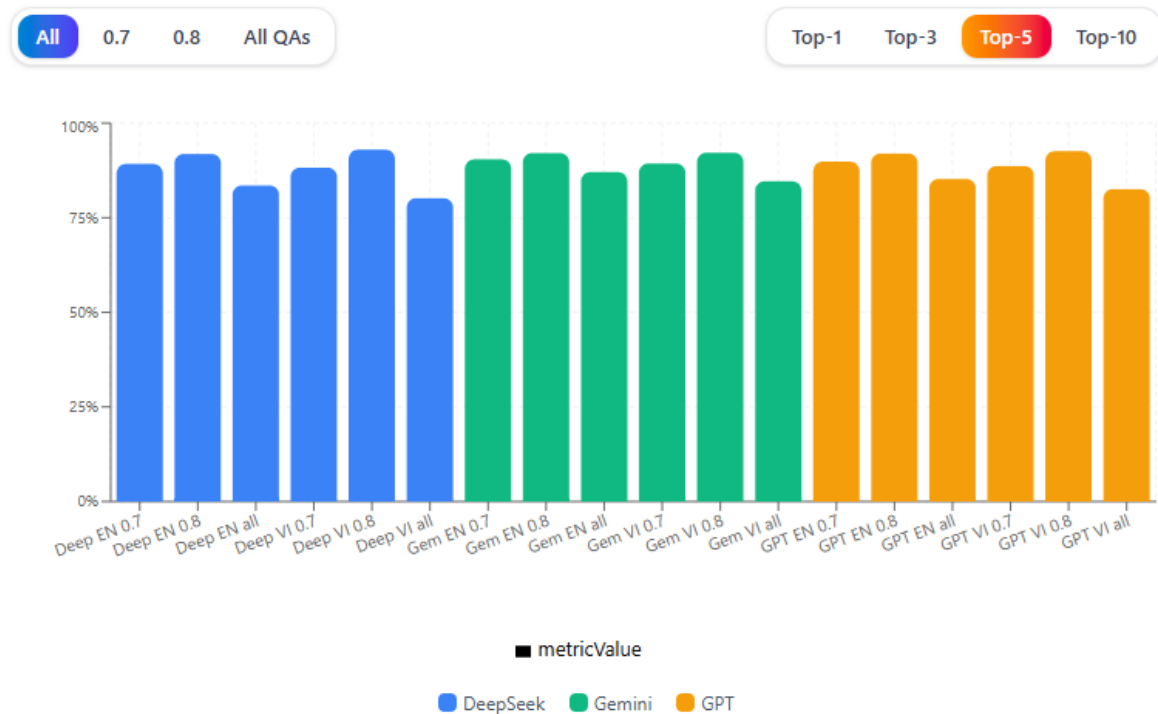


Hình 5.5: Biểu đồ so sánh kết quả truy hồi Top-3 giữa truy vấn tiếng Việt và tiếng Anh khi sử dụng mô hình paraphrase-multilingual-MiniLM-L12-v2

Top-5

Metric Across Configurations

Filter by threshold, then choose Top-k metric to compare.



If Top-5 shows "—" in tooltip/table, backend may not provide that metric yet. Bar chart renders missing values as 0 for stability.

Hình 5.6: Biểu đồ kết quả truy hồi Top-5 giữa các bộ dataset khi sử dụng mô hình paraphrase-multilingual-MiniLM-L12-v2



Hình 5.7: Biểu đồ so sánh kết quả truy hồi Top-5 giữa truy vấn tiếng Việt và tiếng Anh khi sử dụng mô hình paraphrase-multilingual-MiniLM-L12-v2

Top-10

Metric Across Configurations

Filter by threshold, then choose Top-k metric to compare.

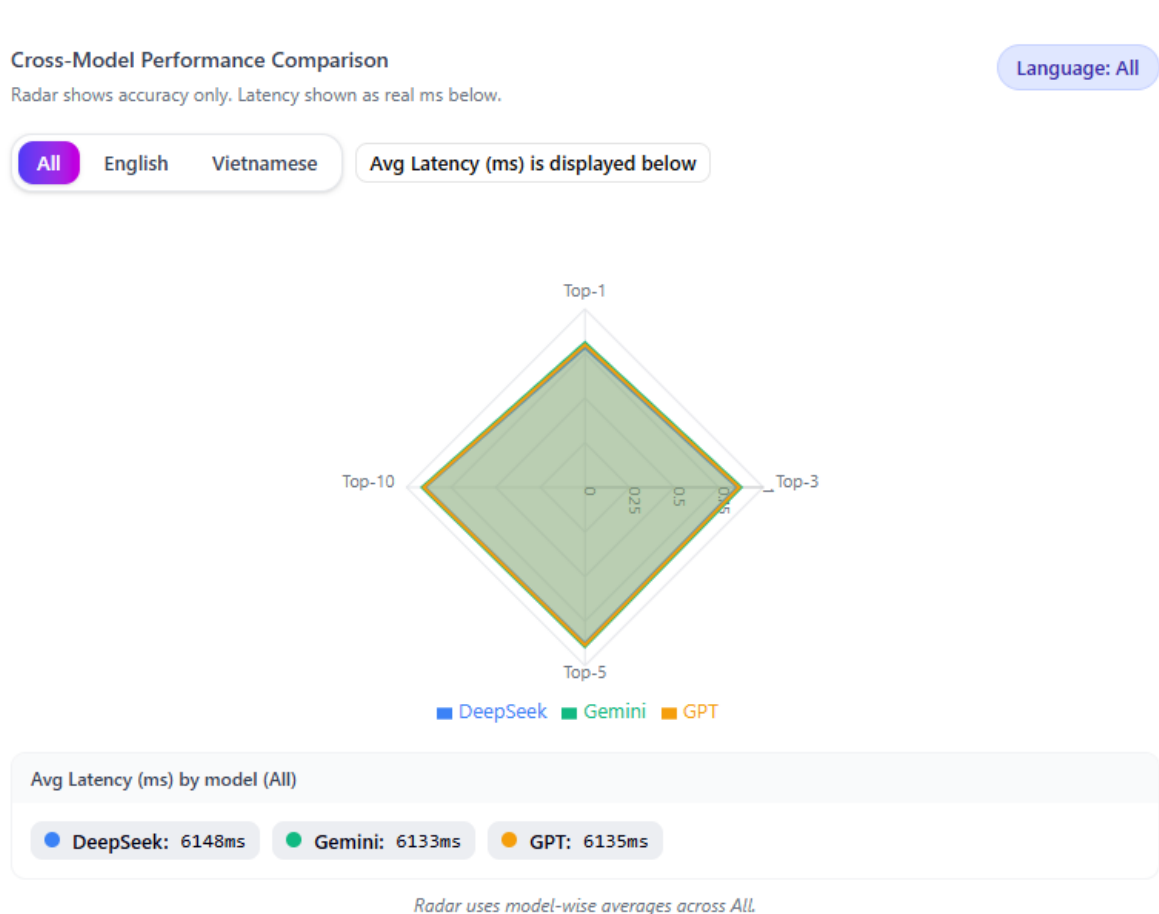


Hình 5.8: Biểu đồ kết quả truy hồi Top-10 giữa các bộ dataset khi sử dụng mô hình paraphrase-multilingual-MiniLM-L12-v2



Hình 5.9: Biểu đồ so sánh kết quả truy hồi Top-10 giữa truy vấn tiếng Việt và tiếng Anh khi sử dụng mô hình paraphrase-multilingual-MiniLM-L12-v2

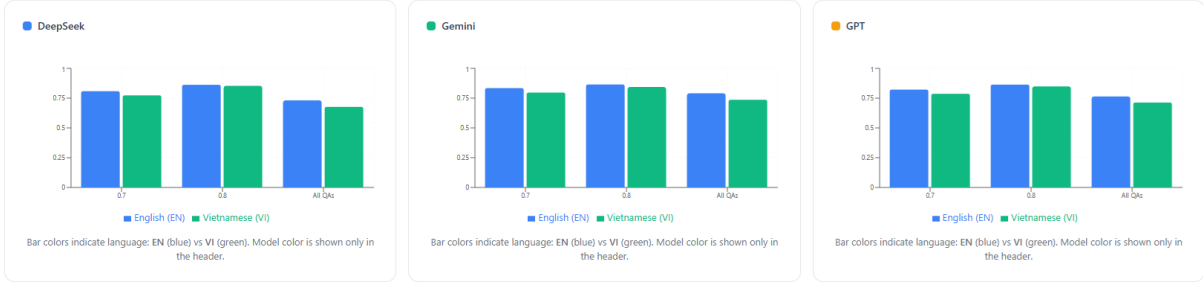
5.3.5 So sánh hiệu năng ở mức mô hình



Hình 5.10: Biểu đồ radar so sánh hiệu năng truy hồi giữa các bộ dataset theo các chỉ số Top-K

4.1 Model-Level Comparison

Performance breakdown by model, language, and threshold setting



Hình 5.11: Biểu đồ so sánh tổng thể hiệu năng truy hồi giữa các bộ dataset

5.3.6 Kết quả thực nghiệm với Backend 2 (BGE-M3)

Kết quả trên tập truy vấn tiếng Việt

Filter	Total Items	Top1	Top3	Top5	Top10	Avg Latency (ms)	Throughput (qps)
0.7	4256	0.8953	0.9301	0.9361	0.9413	2979.65	0.32
0.8	0.4753	0.9076	0.9419	0.9468	0.9509	2959.65	0.29

Bảng 5.7: So sánh hiệu quả truy hồi với các ngưỡng lọc (0.7 và 0.8) trên tập GEMINI-VI sử dụng mô hình BGE-m3

Filter	Total Items	Top1	Top3	Top5	Top10	Avg Latency (ms)	Throughput (qps)
0.7	4252	0.9245	0.9579	0.9654	0.9699	2912.35	0.32
0.8	1087	0.9455	0.9702	0.9766	0.9789	3266.68	0.29

Bảng 5.8: So sánh hiệu quả truy hồi với các ngưỡng lọc (0.7 và 0.8) trên tập DEEPSEEK-VI sử dụng mô hình BGE-m3

Filter	Total Items	Retention	Top1	Top3	Top5	Top10	Avg Latency (ms)	Throughput (qps)
raw	12559	1.0000	0.8450	0.8925	0.9050	0.9153	3380.20	0.29
0.7	4017	0.3198	0.9098	0.9450	0.9528	0.9583	2945.40	0.31
0.8	1523	0.1213	0.8989	0.9397	0.9463	0.9532	3100.20	0.28

Bảng 5.9: So sánh trước và sau lọc (ngưỡng 0.7/0.8) cho tập GPT-VI (BGE)

Kết quả trên tập truy vấn tiếng Anh

Filter	Total Items	Top1	Top3	Top5	Top10	Avg Latency (ms)
0.7	4579	0.9014	0.9307	0.9357	0.9402	2689.47
0.8	2447	0.9103	0.9362	0.9399	0.9431	2660.66

Bảng 5.10: Kết quả đánh giá BGE-m3 trên tập GEMINI-ENG, trong đó ngưỡng 0.7 được sử dụng làm baseline.

Filter	Total Items	Top1	Top3	Top5	Top10	Avg Latency (ms)
0.7	4503	0.8839	0.9196	0.9271	0.9338	2762.15
0.8	1433	0.9010	0.9298	0.9376	0.9411	2752.71

Bảng 5.11: Kết quả đánh giá BGE-m3 trên tập DEEPSEEK-ENG, trong đó ngưỡng 0.7 được sử dụng làm baseline.

Filter	Total Items	Retention	Top1	Top3	Top5	Top10	Avg Latency (ms)	Throughput (qps)
raw	11569	1.0000	0.8300	0.8825	0.8965	0.9078	3450.20	0.27
0.7	4397	0.3800	0.8930	0.9258	0.9328	0.9378	2840.20	0.33
0.8	1788	0.1545	0.9055	0.9335	0.9390	0.9422	2720.40	0.34

Bảng 5.12: So sánh trước và sau lọc (ngưỡng 0.7/0.8) cho tập GPT-EN (BGE)

5.3.7 Phân tích trực quan theo Top-K (Backend 2)

Tổng quan các chỉ số truy hồi

Complete Metrics Table (All Configurations)
Accuracy + Latency distribution (Avg / P50 / P90 / P95)

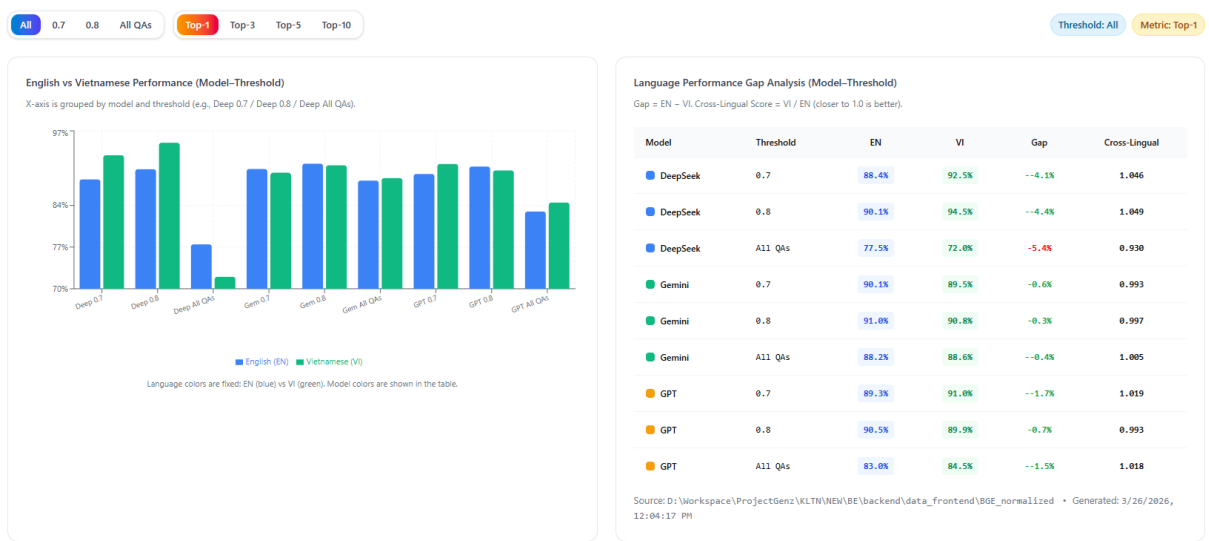
Model	Lang	Threshold	Top-1	Top-3	Top-5	Top-10	Avg	P50	P90	P95	Throughput
DeepSeek	EN	0.7	88.4%	92.0%	92.7%	93.4%	2762ms	2476ms	3793ms	4491ms	0.34 q/s
DeepSeek	EN	0.8	90.1%	93.0%	93.8%	94.1%	2953ms	3010ms	3200ms	4300ms	0.34 q/s
DeepSeek	EN	All QAs	77.5%	85.4%	87.8%	90.2%	3433ms	3880ms	3526ms	3733ms	0.19 q/s
DeepSeek	VI	0.7	92.5%	95.8%	96.5%	97.0%	2912ms	2710ms	3777ms	4476ms	0.32 q/s
DeepSeek	VI	0.8	94.5%	97.0%	97.7%	97.9%	3267ms	2780ms	4575ms	5715ms	0.29 q/s
DeepSeek	VI	All QAs	72.0%	81.3%	84.6%	87.9%	3343ms	3099ms	4216ms	4880ms	0.30 q/s
Gemini	EN	0.7	90.1%	93.1%	93.6%	94.0%	2789ms	2576ms	3693ms	4591ms	0.34 q/s
Gemini	EN	0.8	91.0%	93.6%	94.0%	94.3%	2661ms	2310ms	3300ms	3700ms	– q/s
Gemini	EN	All QAs	88.2%	91.4%	92.2%	92.8%	3343ms	3126ms	4284ms	4994ms	0.28 q/s
Gemini	VI	0.7	89.5%	93.0%	93.6%	94.1%	2900ms	2810ms	3956ms	4556ms	0.32 q/s
Gemini	VI	0.8	90.8%	94.2%	94.7%	95.1%	3060ms	2580ms	3875ms	4015ms	0.29 q/s
Gemini	VI	All QAs	88.6%	92.3%	93.1%	93.7%	3412ms	3200ms	4351ms	5050ms	0.28 q/s
GPT	EN	0.7	89.3%	92.6%	93.3%	93.8%	2840ms	2520ms	3901ms	4651ms	0.33 q/s
GPT	EN	0.8	90.5%	93.3%	93.9%	94.2%	2720ms	2510ms	3600ms	4300ms	0.34 q/s
GPT	EN	All QAs	83.0%	88.3%	89.6%	90.8%	3450ms	3200ms	4200ms	4900ms	0.27 q/s
GPT	VI	0.7	91.0%	94.5%	95.3%	95.8%	2945ms	2720ms	3800ms	4500ms	0.31 q/s
GPT	VI	0.8	89.9%	94.0%	94.6%	95.3%	3100ms	2850ms	3900ms	4600ms	0.28 q/s
GPT	VI	All QAs	84.5%	89.3%	90.5%	91.5%	3380ms	3150ms	4200ms	4900ms	0.29 q/s

Hình 5.12: Biểu đồ tổng quan các chỉ số Top-K khi sử dụng mô hình BGE-M3

Top-1

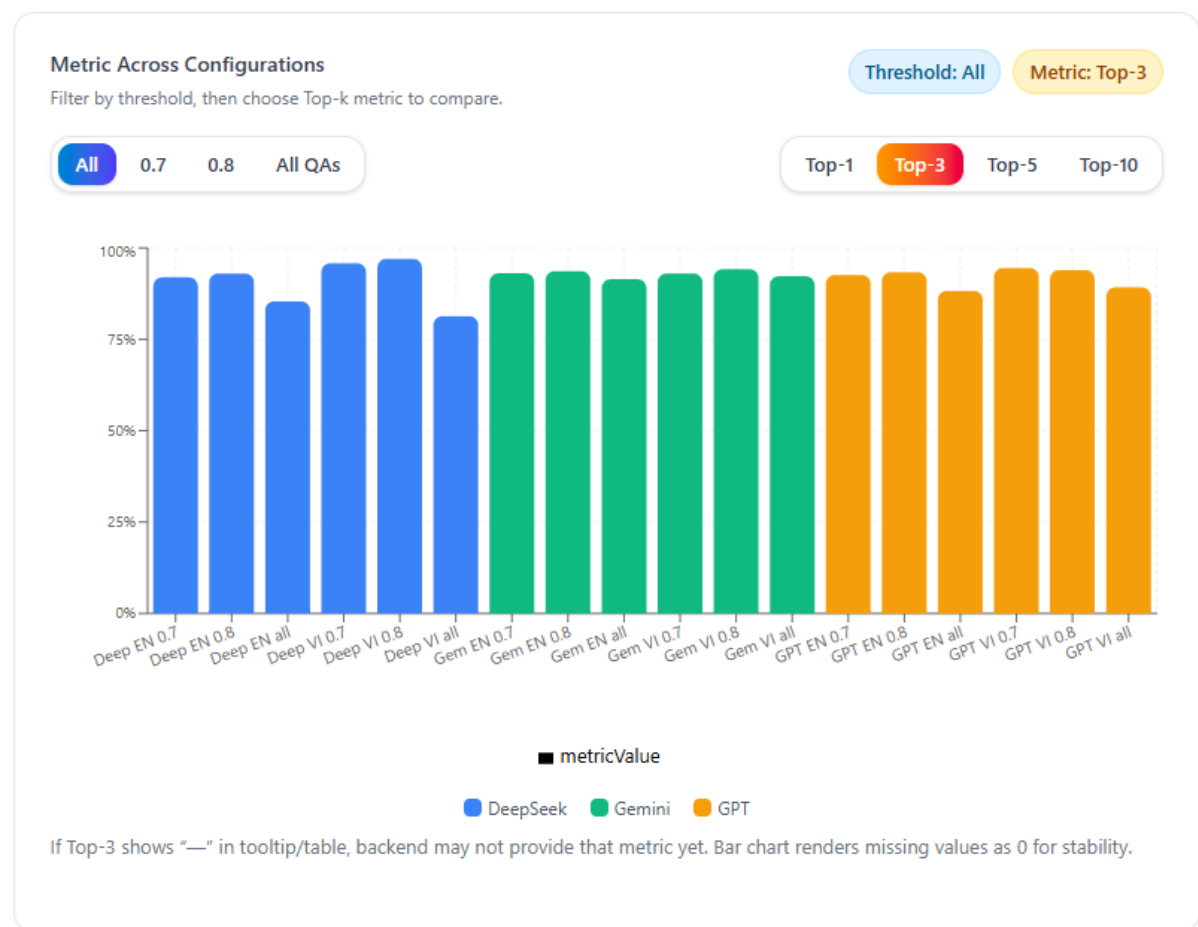


Hình 5.13: Biểu đồ kết quả truy hồi Top-1 giữa các bộ dataset khi sử dụng mô hình BGE-M3

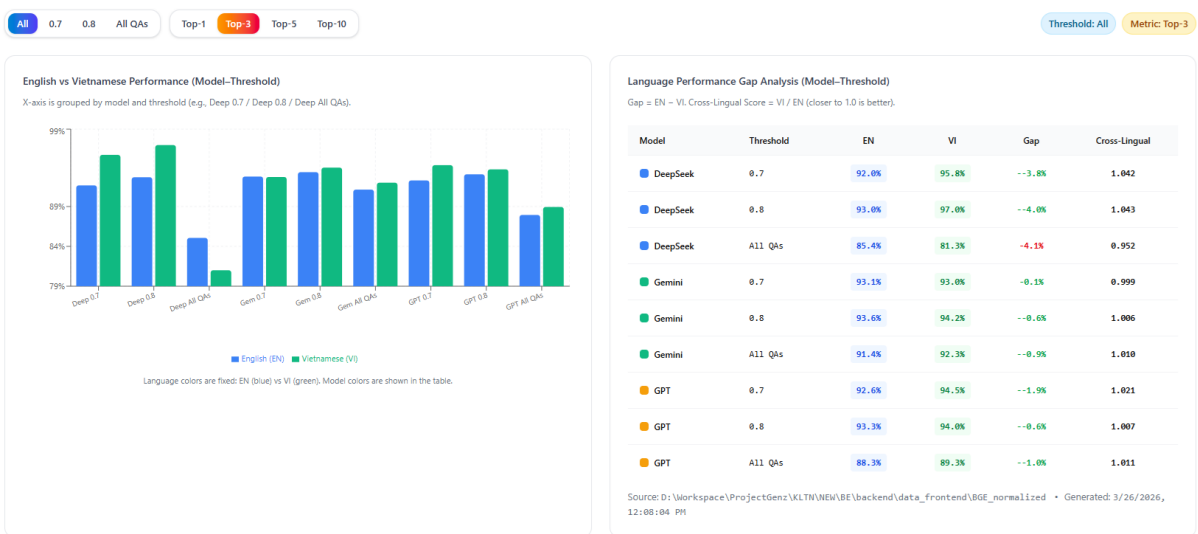


Hình 5.14: Biểu đồ so sánh kết quả truy hồi Top-1 giữa truy vấn tiếng Việt và tiếng Anh khi sử dụng mô hình BGE-M3

Top-3

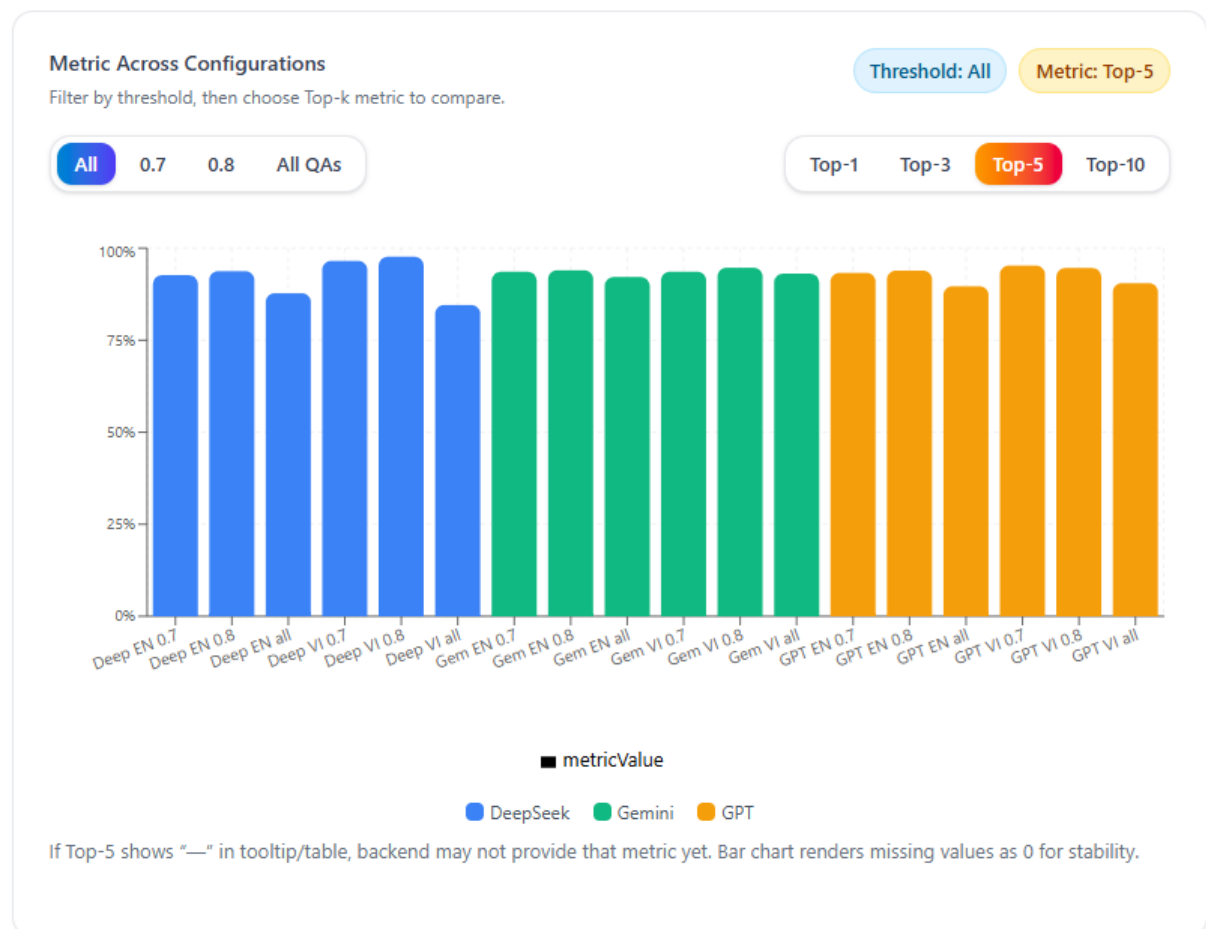


Hình 5.15: Biểu đồ kết quả truy hồi Top-3 giữa các bộ dataset khi sử dụng mô hình BGE-M3

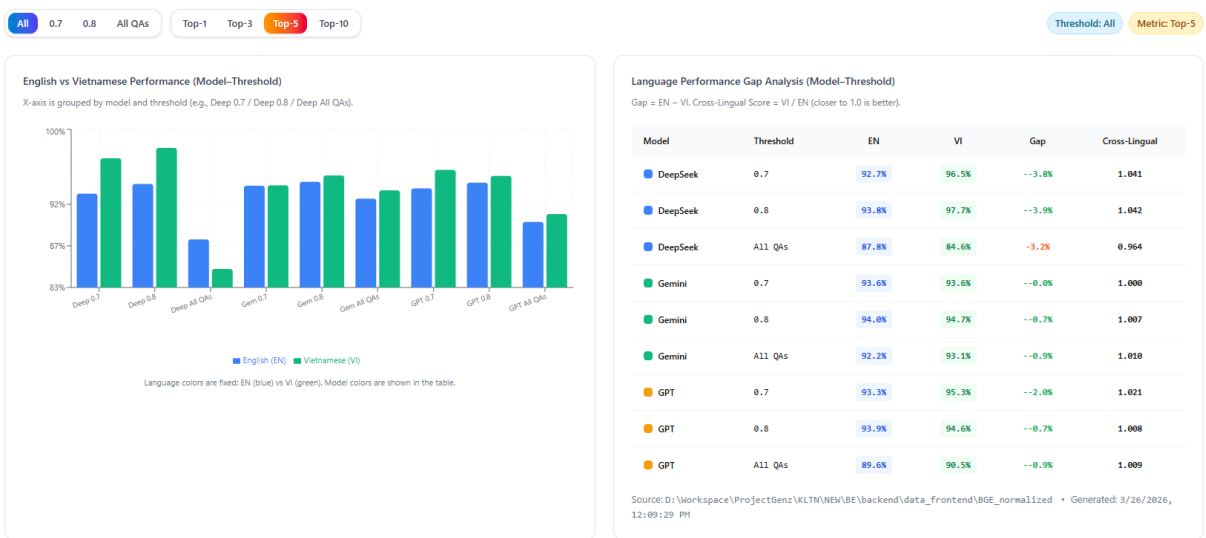


Hình 5.16: Biểu đồ so sánh kết quả truy hồi Top-3 giữa truy vấn tiếng Việt và tiếng Anh khi sử dụng mô hình BGE-M3

Top-5

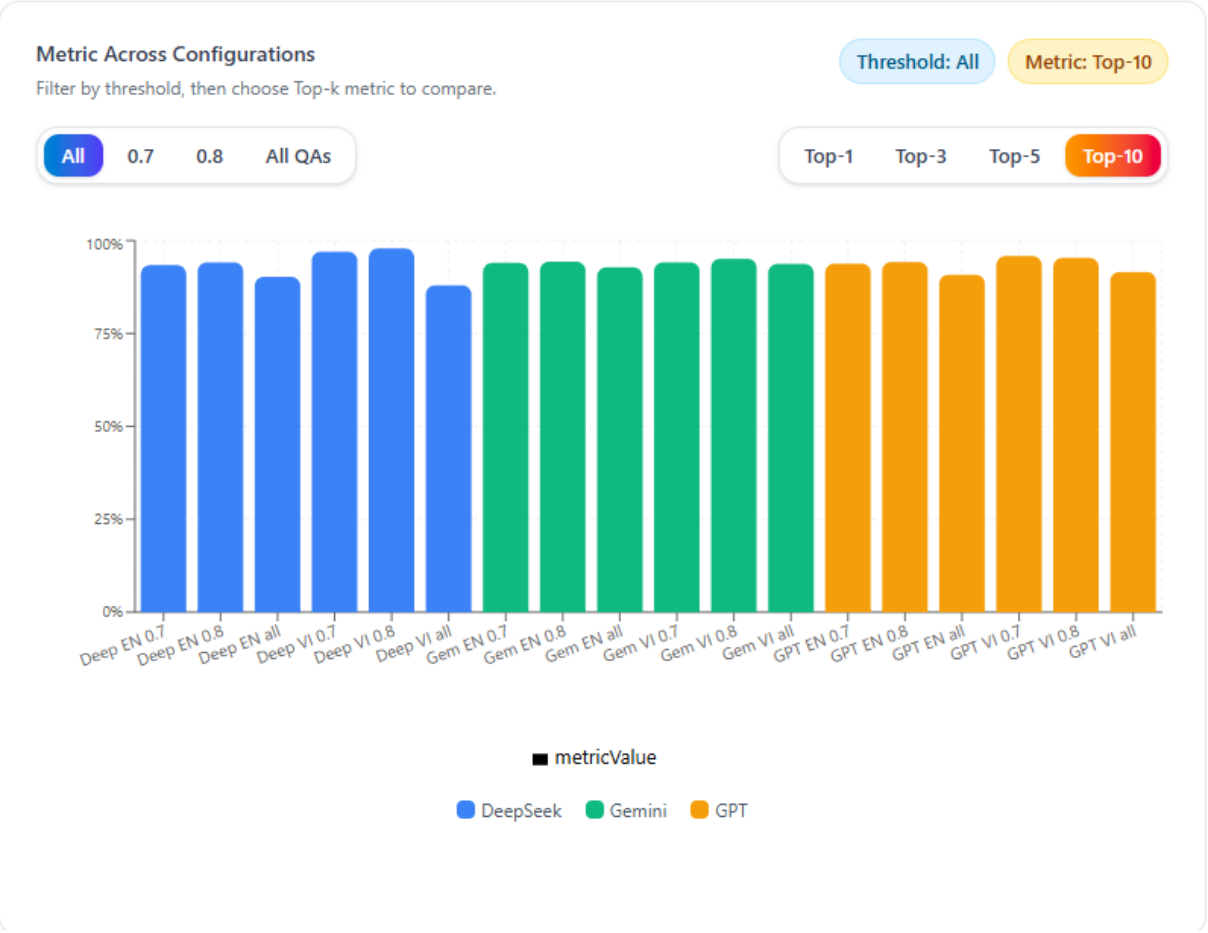


Hình 5.17: Biểu đồ kết quả truy hồi Top-5 giữa các bộ dataset khi sử dụng mô hình BGE-M3

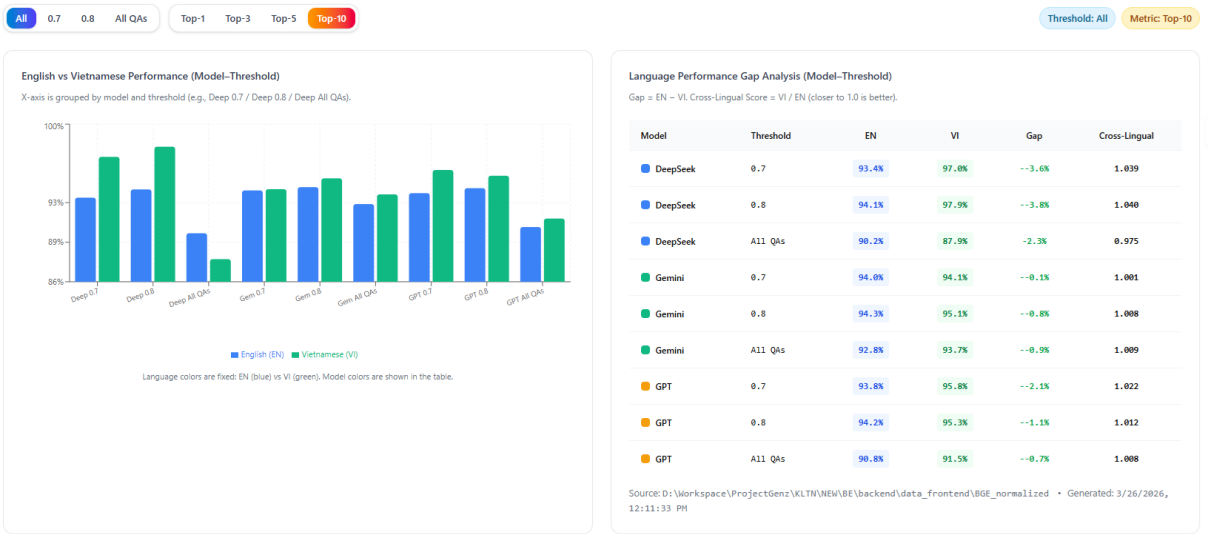


Hình 5.18: Biểu đồ so sánh kết quả truy hồi Top-5 giữa truy vấn tiếng Việt và tiếng Anh khi sử dụng mô hình BGE-M3

Top-10

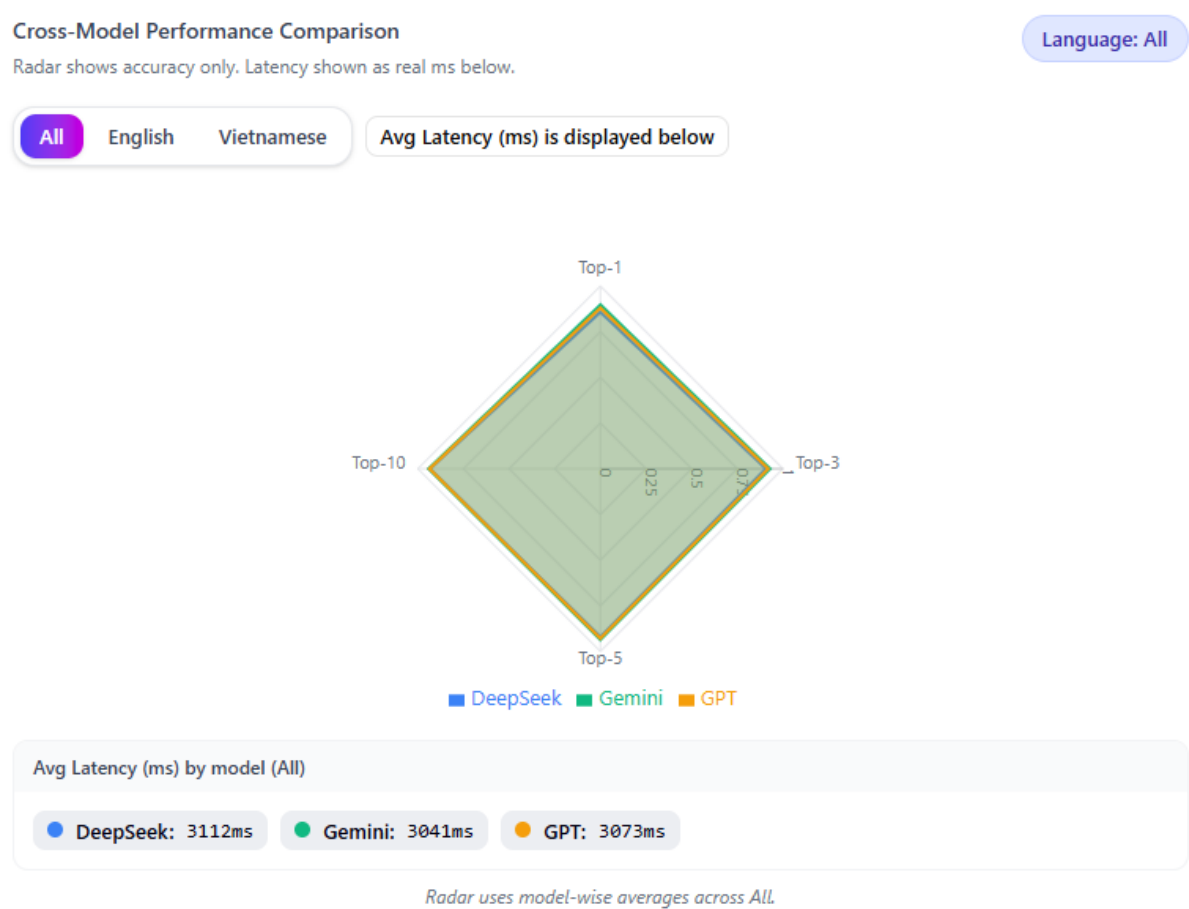


Hình 5.19: Biểu đồ kết quả truy hồi Top-10 giữa các bộ dataset khi sử dụng mô hình BGE-M3

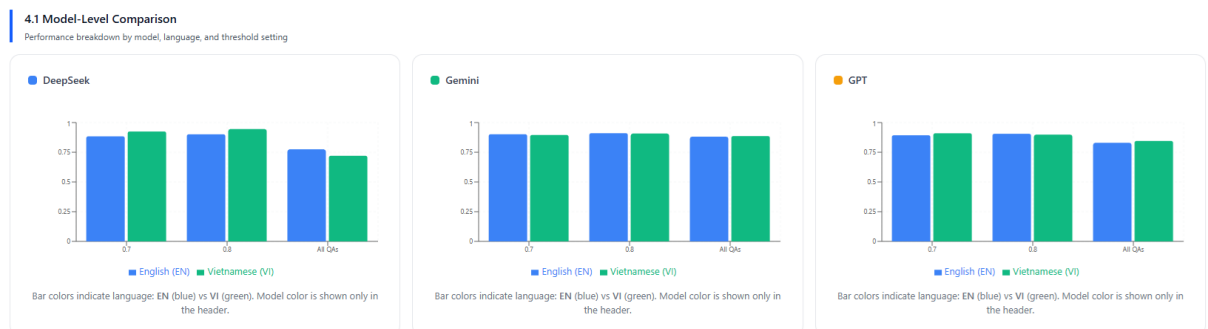


Hình 5.20: Biểu đồ so sánh kết quả truy hồi Top-10 giữa truy vấn tiếng Việt và tiếng Anh khi sử dụng mô hình BGE-M3

5.3.8 Model-Level Comparison (Backend 2)



Hình 5.21: Biểu đồ radar so sánh hiệu năng truy hồi giữa các bộ dataset theo các chỉ số Top-K



Hình 5.22: Biểu đồ so sánh tổng thể hiệu năng truy hồi giữa các bộ dataset

5.3.9 Kết luận

Từ các kết quả thực nghiệm có thể nhận thấy rằng hệ thống truy hồi đạt hiệu năng ổn định trên cả hai ngôn ngữ tiếng Việt và tiếng Anh. Các

chỉ số Top-K cho thấy khả năng đưa đoạn văn chứa thông tin liên quan vào nhóm kết quả đầu với xác suất tương đối cao.

Khi so sánh giữa hai backend, có thể thấy rõ sự khác biệt về hiệu năng giữa MiniLM và BGE-M3. Cụ thể, BGE-M3 consistently đạt kết quả tốt hơn trên tất cả các mức Top-K, đặc biệt là Top-1. Mức cải thiện này thường nằm trong khoảng 8–10%, cho thấy khả năng biểu diễn ngữ nghĩa của BGE vượt trội hơn so với MiniLM.

Ngoài ra, BGE không chỉ cải thiện về độ chính xác mà còn cho thấy hiệu năng xử lý tốt hơn trong nhiều trường hợp, với độ trễ trung bình thấp hơn và throughput cao hơn khi được triển khai trên GPU. Điều này cho thấy việc sử dụng embedding mạnh hơn kết hợp với tài nguyên tính toán phù hợp có thể mang lại lợi ích đáng kể cho hệ thống truy hồi.

Bên cạnh đó, các thí nghiệm cũng cho thấy rằng việc áp dụng các ngưỡng lọc QA (0.7 và 0.8) giúp cải thiện rõ rệt hiệu năng truy hồi ở cả hai backend. Tuy nhiên, BGE tận dụng tốt hơn các tập dữ liệu đã được lọc, đạt mức hiệu năng cao hơn khi dữ liệu có chất lượng tốt.

Tổng hợp lại, có thể rút ra rằng:

- MiniLM phù hợp với các hệ thống yêu cầu tốc độ cao, chi phí thấp và triển khai trên CPU.
- BGE-M3 phù hợp với các hệ thống cần độ chính xác cao, đặc biệt trong các ứng dụng yêu cầu Top-1 chính xác.
- Chất lượng embedding có ảnh hưởng rất lớn đến hiệu năng truy hồi, thậm chí có thể quan trọng ngang với chất lượng dữ liệu QA.

Nhìn chung, việc kết hợp giữa dữ liệu QA chất lượng cao và mô hình embedding mạnh giúp hệ thống đạt được sự cân bằng tốt giữa độ chính xác và hiệu năng, phù hợp cho các hệ thống semantic search và RAG trong thực tế.

Chương 6

Kết luận và hướng phát triển

6.1 Kết luận

Khóa luận đã đề xuất và hiện thực hóa một hệ thống truy hồi và kiểm tra độ tương đồng văn bản xuyên ngữ dựa trên kiến trúc semantic search hiện đại. Hệ thống được xây dựng theo hướng dữ liệu thực nghiệm quy mô lớn, kết hợp nhiều mô hình ngôn ngữ lớn (LLMs) và pipeline kiểm định ngữ nghĩa đa tầng nhằm đảm bảo chất lượng dữ liệu đầu vào.

Các thành phần chính của hệ thống bao gồm:

- Thu thập và xử lý 1041 tài liệu PDF tiếng Việt và tiếng Anh từ các nguồn học thuật và báo chí.
- Sinh tự động 70,870 cặp câu hỏi-trả lời (QA) bằng các mô hình ngôn ngữ lớn, bao gồm Gemini 2.5 Flash (Google API), DeepSeek R1T2 Chimera (triển khai nội bộ) và GPT-5.2 (thông qua giao diện ChatGPT).
- Xây dựng pipeline kiểm định chất lượng QA ba tầng:
 - Bi-encoder đa ngữ (paraphrase-multilingual-MiniLM-L12-v2) để lọc nhanh dựa trên cosine similarity (relevance filtering).
 - Cross-encoder (mmarco-mMiniLMv2-L12-H384-v1) để kiểm tra chặt chẽ mối quan hệ giữa context và answer (precision refinement).
 - Mô hình NLI (mDeBERTa-v3-base-mnli-xnli) để xác nhận tính suy luận logic giữa context và answer (correctness verification).
- Áp dụng hai mức ngưỡng $\tau = 0.70$ và $\tau = 0.80$ nhằm phân tích trade-off giữa recall và precision.

- Xây dựng hệ thống semantic search sử dụng Sentence-BERT đa ngữ và FAISS IndexFlatIP cho truy hồi vector hiệu năng cao.
- Thiết kế dashboard trực quan để phân tích hiệu năng theo mô hình, ngôn ngữ và ngưỡng kiểm định.

Kết quả thực nghiệm cho thấy pipeline hai tầng ban đầu giúp loại bỏ 60–85% QA có liên kết ngữ nghĩa yếu, đồng thời giữ lại tập dữ liệu có độ tin cậy cao (26,083 QA ở ngưỡng 0.70 và 10,372 QA ở ngưỡng 0.80). Điểm trung bình cross-encoder gần đạt mức tối đa (0.976–0.989), chứng tỏ các QA được giữ lại có mức entailment rất mạnh với tài liệu nguồn.

Tuy nhiên, việc bổ sung tầng kiểm định thứ ba bằng NLI giúp cải thiện đáng kể chất lượng dữ liệu, đặc biệt trong việc loại bỏ các câu trả lời có xu hướng suy diễn hoặc vượt quá phạm vi context. Kết quả đánh giá thủ công cho thấy độ chính xác của QA tăng từ 90.17% lên 95.33%, khẳng định vai trò quan trọng của NLI trong việc đảm bảo tính đúng đắn về mặt suy luận.

Phân tích so sánh giữa các mô hình sinh QA cho thấy sự khác biệt rõ rệt về đặc tính sinh dữ liệu. Gemini 2.5 Flash đạt độ ổn định ngữ nghĩa cao nhất ở cả hai ngưỡng, nhờ khả năng bám sát ngữ cảnh và duy trì quan hệ entailment tốt. GPT-5.2 thể hiện hiệu năng trung gian, cân bằng giữa độ đa dạng và độ chính xác, nhưng chịu ảnh hưởng bởi sự không nhất quán trong quá trình sinh thông qua giao diện ChatGPT (prompt drift). Trong khi đó, DeepSeek R1T2 Chimera tạo ra số lượng QA lớn nhất nhưng có tỉ lệ bị loại cao hơn, phản ánh xu hướng sinh dữ liệu đa dạng nhưng ít ràng buộc chặt với ngữ cảnh.

Bên cạnh đó, kết quả thực nghiệm cũng cho thấy sự khác biệt đáng kể giữa các mô hình embedding. Cụ thể, BGE-M3 consistently đạt hiệu năng cao hơn so với MiniLM trên tất cả các mức Top-K, đặc biệt là Top-1 với mức cải thiện khoảng 8–10%. Ngoài ra, BGE còn cho thấy hiệu năng xử lý tốt hơn trong nhiều trường hợp khi được triển khai trên GPU. Điều này cho thấy chất lượng embedding đóng vai trò quan trọng không kém so với chất lượng dữ liệu QA trong hệ thống semantic retrieval.

Về mặt học thuật, đề tài đóng góp:

- Một quy trình xây dựng và kiểm định dữ liệu QA tự động có truy vết nguồn rõ ràng.
- Một pipeline kiểm định ba tầng tương thích với kiến trúc retrieve-rerank hiện đại và mở rộng thêm bước kiểm chứng suy luận.
- Một bộ dữ liệu QA xuyên ngữ đã được lọc ngữ nghĩa nhiều tầng, có thể sử dụng cho đánh giá semantic search hoặc RAG.
- Một phân tích so sánh giữa các mô hình LLM trong bài toán sinh QA xuyên ngữ, làm rõ trade-off giữa độ đa dạng và độ chính xác dữ liệu.

Về mặt ứng dụng, hệ thống có tiềm năng triển khai trong:

- Hệ thống hỏi–đáp dựa trên tài liệu (document QA).
- Semantic search cho môi trường song ngữ.
- Kiểm tra mức độ tương đồng nội dung xuyên ngữ.
- Hệ thống Retrieval-Augmented Generation (RAG) trong thực tế.

Tổng thể, khóa luận chứng minh rằng việc kết hợp embedding đa ngữ, FAISS index và kiến trúc kiểm định đa tầng (bi-encoder + cross-encoder + NLI) là một hướng tiếp cận hiệu quả và có tính thực tiễn cao cho bài toán truy hồi và kiểm tra tương đồng văn bản xuyên ngữ trong bối cảnh dữ liệu lớn. Đồng thời, việc kết hợp nhiều mô hình sinh QA (Gemini, GPT-5.2 và DeepSeek) giúp tận dụng ưu điểm riêng của từng mô hình, nâng cao cả độ bao phủ và độ tin cậy của dữ liệu sau kiểm định.

6.2 Hướng phát triển

Mặc dù hệ thống đã đạt được kết quả khả quan trong việc xây dựng và kiểm định bộ dữ liệu QA xuyên ngữ, vẫn còn nhiều hướng mở rộng có thể triển khai trong tương lai nhằm nâng cao cả độ chính xác lẫn khả năng ứng dụng thực tế.

- **Mở rộng quy mô và đa dạng miền dữ liệu**

Hiện tại hệ thống được xây dựng trên 1041 tài liệu PDF thuộc các lĩnh vực học thuật và báo chí. Trong tương lai, có thể mở rộng sang:

- Tài liệu chuyên ngành (y khoa, luật, kỹ thuật).
- Dữ liệu song ngữ có cấu trúc (parallel corpora).
- Dữ liệu đa miền nhằm đánh giá khả năng tổng quát hóa của embedding đa ngữ.

Việc mở rộng miền dữ liệu sẽ giúp đánh giá khả năng ổn định của pipeline kiểm định QA trong môi trường phức tạp hơn.

- **Nâng cấp mô hình embedding và reranking**

Pipeline hiện tại sử dụng MiniLM đa ngữ và cross-encoder mMARCO. Trong tương lai có thể:

- Thử nghiệm các mô hình embedding lớn hơn (e5-large, GTE, Instructor, BGE-m3).
- Huấn luyện fine-tune embedding theo miền dữ liệu riêng.
- Thay thế cross-encoder bằng mô hình NLI chuyên biệt hoặc LLM-based verifier.
- So sánh thêm các chiến lược reranking như LLM reranker hoặc ColBERT.

Việc nâng cấp này có thể cải thiện độ chính xác semantic alignment và giảm false positive trong kiểm định QA.

- **Tối ưu chiến lược chọn ngưỡng (Adaptive Thresholding)**

Hiện hệ thống sử dụng ngưỡng cố định $\tau = 0.70$ và $\tau = 0.80$. Trong tương lai có thể:

- Áp dụng threshold động theo từng miền dữ liệu.
- Sử dụng calibration hoặc validation set để chọn ngưỡng tối ưu tự động.

- Phân tích ROC curve và precision–recall curve thay vì ngưỡng thủ công.

Điều này giúp hệ thống thích nghi tốt hơn với các tập dữ liệu khác nhau.

- **Kết hợp đánh giá thủ công (Human-in-the-loop)**

Hiện tại toàn bộ QA được kiểm định tự động bằng mô hình. Một hướng phát triển quan trọng là:

- Lấy mẫu QA để đánh giá thủ công.
- So sánh độ tương đồng giữa đánh giá mô hình và đánh giá con người.
- Sử dụng kết quả đánh giá thủ công để tinh chỉnh threshold hoặc fine-tune mô hình.

Cách tiếp cận này giúp tăng độ tin cậy khoa học của bộ dữ liệu.

- **Tích hợp vào hệ thống Retrieval-Augmented Generation (RAG)**

Bộ dữ liệu QA đã kiểm định có thể được sử dụng làm:

- Tập đánh giá retrieval trong RAG.
- Benchmark nội bộ cho cross-lingual document QA.
- Nền tảng xây dựng chatbot học thuật đa ngữ.

Việc tích hợp vào pipeline RAG hoàn chỉnh sẽ giúp kiểm chứng khả năng ứng dụng thực tế của hệ thống.

- **Triển khai hệ thống thời gian thực**

Trong tương lai, hệ thống có thể được triển khai dưới dạng:

- API semantic search xuyên ngữ.
- Hệ thống kiểm tra tương đồng văn bản cho môi trường học thuật.

- Dashboard theo dõi chất lượng QA theo thời gian thực.

Khi đó cần tối ưu thêm về:

- ANN index (IVF, HNSW).
- Tối ưu GPU cho embedding.
- Batch processing cho reranking.

Tổng thể, hướng phát triển tập trung vào ba trục chính: mở rộng dữ liệu, nâng cấp mô hình ngữ nghĩa và đưa hệ thống từ môi trường thực nghiệm sang môi trường ứng dụng thực tế.

Tài liệu tham khảo

- [1] Stephen Robertson and Hugo Zaragoza. “The Probabilistic Relevance Framework: BM25 and Beyond”. In: *Foundations and Trends in Information Retrieval* (2009).
- [2] Nils Reimers and Iryna Gurevych. “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks”. In: *arXiv preprint arXiv:1908.10084* (2019).
- [3] Alexis Conneau, Kartikay Khandelwal, Naman Goyal, et al. “Unsupervised Cross-lingual Representation Learning at Scale”. In: *arXiv preprint arXiv:1911.02116* (2020).
- [4] Jeff Johnson, Matthijs Douze, and Herve Jegou. *Billion-scale Similarity Search with GPUs*. <https://github.com/facebookresearch/faiss>. 2017.
- [5] Rodrigo Nogueira and Kyunghyun Cho. “Passage Re-ranking with BERT”. In: *arXiv preprint arXiv:1901.04085* (2019).
- [6] Vladimir Karpukhin, Barlas Oguz, Sewon Min, et al. “Dense Passage Retrieval for Open-Domain Question Answering”. In: *EMNLP*. 2020.
- [7] Phuc Vu, Dat Quoc Nguyen, and Anh Tuan Nguyen. *PhoBERT: Pre-trained Language Models for Vietnamese*. <https://huggingface.co/VinAIRResearch/phobert-base>. VinAI Research. 2020.
- [8] Phuc Van Vu et al. “Vietnamese Sentence-BERT based on PhoBERT and SimCSE”. In: *Proceedings of VLSP* (2021).
- [9] Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *arXiv preprint arXiv:2005.11401* (2020).
- [10] Colin Raffel, Noam Shazeer, Adam Roberts, et al. “Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer”. In: *Journal of Machine Learning Research* 21.140 (2020), pp. 1–67.

- [11] Valhalla. *T5 Base Question Generation Model*. <https://huggingface.co/valhalla/t5-base-qg-h1>. 2021.
- [12] Omar Khattab and Matei Zaharia. “ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT”. In: *SIGIR*. 2020.
- [13] Nandan Thakur et al. “BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models”. In: *NeurIPS*. 2021.
- [14] Tianyu Gao, Xingcheng Yao, and Danqi Chen. “SimCSE: Simple Contrastive Learning of Sentence Embeddings”. In: *EMNLP* (2021).
- [15] Jacob Devlin et al. “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”. In: *NAACL* (2019).
- [16] Tri Nguyen, Mir Rosenberg, Xia Song, et al. “MS MARCO: A Human Generated MACHine Reading COMprehension Dataset”. In: *arXiv preprint arXiv:1611.09268* (2016).
- [17] Yu. A. Malkov and D. A. Yashunin. “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2018).
- [18] Shitao Xiao, Zheng Liu, et al. “BGE: Large-scale Dense Retrieval Model”. In: *arXiv preprint arXiv:2309.07597* (2023).
- [19] Liang Wang, Nan Yang, et al. “Text Embeddings by Weakly-Supervised Contrastive Pre-training”. In: *arXiv preprint arXiv:2212.03533* (2023).
- [20] Jimmy Lin, Xueguang Ma, et al. “A Survey of Dense Retrieval for Information Retrieval”. In: *ACM Computing Surveys* (2022).
- [21] Keshav Santhanam et al. “ColBERTv2: Effective and Efficient Retrieval via Lightweight Late Interaction”. In: *arXiv preprint arXiv:2112.01488* (2021).